CS608 HANDOUTS UPDATED

(FINAL TERM SYLABUS)

(BY VURANK ADMIN)

Module 85:
Test Planning

- While doing software testing through QA team in a software house, we need to do planning to:
  - determine **effort** needed to validate quality of SUT
  - Engage key stakeholders such as developers, business managers, customers etc.
  - Output of test planning is Test Plan document
  - Test planning is essential to:
    - ensure testing is a traceable process
    - Make efficient use of resources delivering quantifiable results
  - Steps involved:
  - Analyze product and design test strategy
  - Define test objectives
  - Define test criteria (suspension/exit criteria)
  - Resource planning
  - Plan test environment
  - Determine test deliverables
- Product analysis:
  - Product family i.e. website, web service, client-server application, component etc.
- Strategy ingredients:
  - Test management plan
  - Test plan

        - Scope(in and out), key risks, key issues, logistics (when, who, how)
- Test Criteria
    - Suspension criteria: e.g.,
        - more than X number of crashes,
        - Y number of test cases reported to have failed
    - Exit Criteria denotes successful completion of a test phase e.g.,
        - Run rate: number test cases executed/total test cases, has to be 100%
        - Pass rate: numbers test cases passed / test cases executed

Module 86:

Test Plan Document

- **According to IEEE Std 829-1983:**
    - A document describing the scope, approach, resources, and schedule of intended testing activities.
    - It identifies test items, the features to be tested, the testing tasks, who will be doing each task, and any risks requiring contingency planning.
- **Kaner et al 1999:**
    - *A test plan is a valuable tool to the extent that it helps you manage your testing project and find bugs. Beyond that, it is a diversion of resources.*
- **IEEE Standard for Software Documentation**
    - Test Plan Identifier
    - Introduction
    - Test Items
    - Features to be tested
    - Features not to be tested
    - Approach
    - Item pass / fail criteria
    - Suspension criteria and resumption criteria
    - Test deliverables

    - Testing tasks
    - Environmental needs
    - Responsibilities
    - Staffing and training needs
    - Schedule
    - Risks and contingencies
    - Approvals
- Test Items identify items that are to be tested within the scope of this test plan:
    - Functions of the software
    - Requirements stated in the Design stage
    - This is important while reporting test results, outputs etc.
- **Specifications**
    - Test Design Specification
        - Test conditions, test items, high level test cases
    - Test Case Specification
- **Test Case Ingredients**
    - For different aspects of the test plan, we have different test case ingredients
    - ID – unique identifier of a test case;
    - steps / input values
    - Expected result / output values
    - Actual result – what you really get from application;
    - Pass / Fail
- Verbal description indicative of test case objective;
- Goal / objective – primary verification point of the test case;
- Functional area
- Bug numbers for Failed test cases
- Positive / Negative class – for test execution planning;
- Manual / Automatable / Automated parameter etc – for planning purposes;
- …

- **Test approach**
    - This section identifies strategy for test plan, differing depending on the level of test plan (Unit, Integration, Acceptance)
    - The level of detail of this section differs depending on the level of test plan. For example, a Unit test plan will go into much detail on individual unit tests and test data.
- **Risks**
    - All risks associated with the software and its testing need to be identified in this section. Why??
        - Plan for risks and contingencies
    - This could include complex functions, new versions of cooperating software, etc...
    - Test planners should be aware of:
        - Vague, unclear or un-testable requirements
        - Misunderstanding of requirements
- **Features to be tested**
    - Identifies the features to be tested from a particular development phase point of view.
- Example:
    - Imagine we are working on an enhancement in our product ABC, where only H/R module is added and finance module is updated. We only test
        - H/R Module
        - Finance module
- Features not to be tested lists the features not to be included in the testing process,
- We also identify reasons behind its exclusion.
    - Has no effect from newly developed part of the system
    - No intention of releasing with software?
- Example, considering our previous example:
    - Sales and Spares system

Module 87:

Test Plan Objectives and Motivation

- **Test Plan Objectives**
- Test objectives and success criteria should be developed based on
    - client's business and technical goals
    - SLAs associated with applications or services.
- Test objectives should simply be to measure the outcome of the test case
- This helps test plan developer define relevant test cases with clearly identifiable success or failure metrics that can be agreed upon by tester / client.
- **Test Plan Motivation**
    - Testers motivation is to find faults or defects
    - Test plan motivation is to provide
        - A mechanism to identify all aspects of testing that are essential to test SUT considering
            - Application domain
            - SRS document
- A matrix containing test cases segregated according to their type, nature
    - Example: we need separate sections containing test cases pertaining to functionality, load, stress, admin pages, user pages, content managers etc.
- A test result reporting mechanism to satisfy key stakeholders e.g., financer, management, development teams etc.

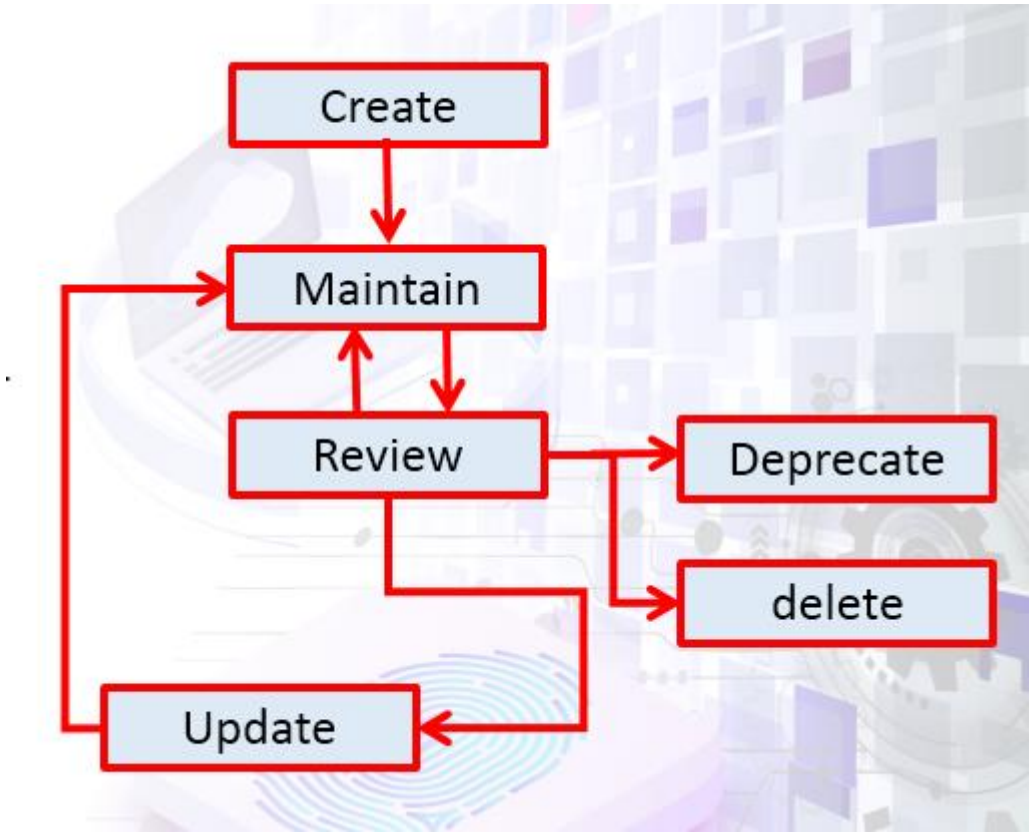
Module 88:

Test Case Lifecycle

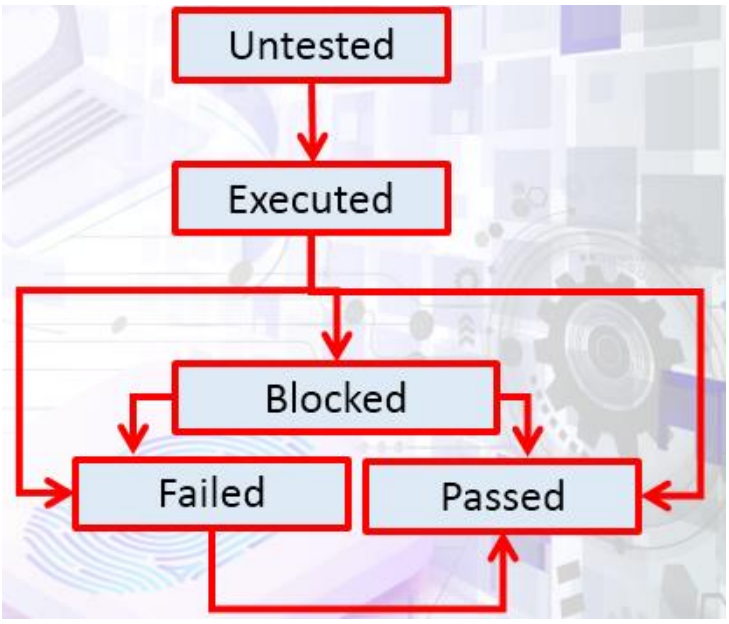
- A test case undergoes a series of steps from its creation to being deleted or deprecated.
- A test case is a process rather than a single activity i.e. designing a scenario, preparing for execution and evaluating status till the test closure.
- A test case activity can be divided into fundamental test process with following steps:
- Planning and Control step determine scope and risks and identify objectives of testing.
- Analysis and Design step is used to review the information we need in order to start the test analysis and create our own test cases.
- Implementation and Execution represent set of conditions under which a tester will determine whether an application is working correctly or not and test case is pass or fail

- Evaluating exit criteria and Reporting is set the criteria for each test level against which we will measure the “enough testing”.
- Test Closure activities are done when software is delivered to qa testing companies. The testing can be closed when all the information has been gathered which are needed for the testing.

- **Test case lifecycle**
    - Created with an ID and other details
    - Maintained in the repository for use
    - Reviewed periodically or upon a release
        - Either kept as is
        - Or updated and kept for use
        - Or deprecated / deleted depending upon situation



- State transition of a test case

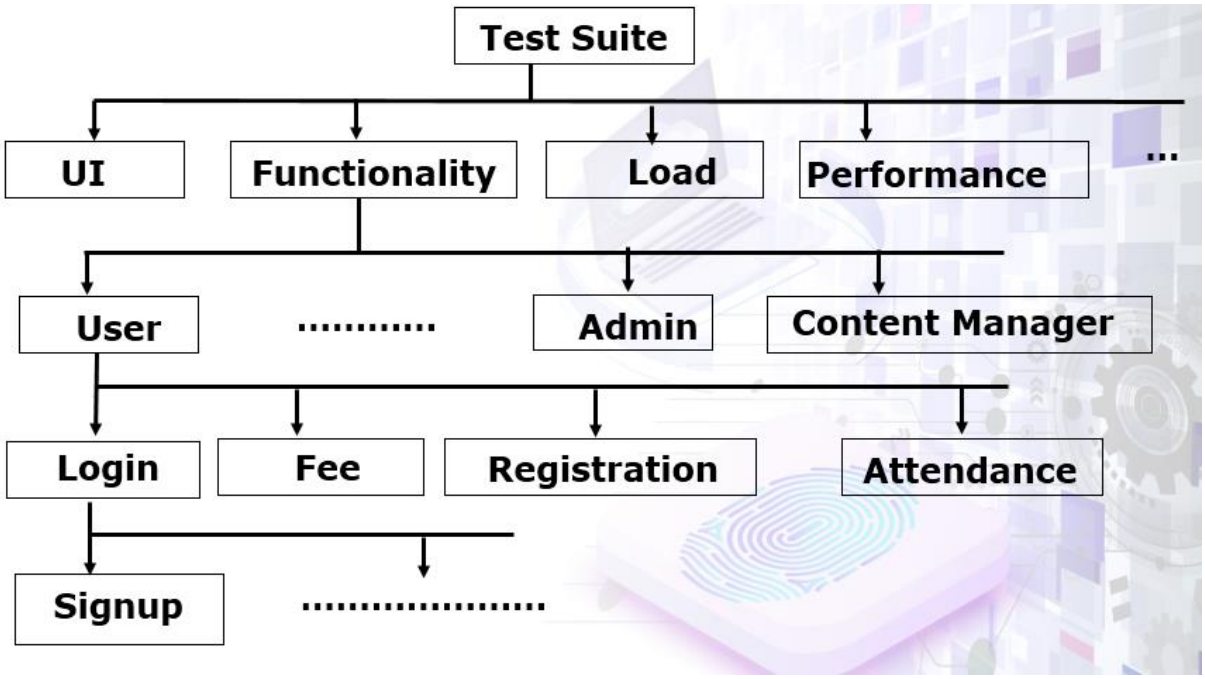


Module 89:

Test Suite Structure

- **Test suite**: this is a set of test cases organized in a hierarchal manner for a certain software release.
- There are two important characteristics of a test suite:
    - Test cases in a test suite relate to a certain release of the system.
    - Test cases are organized in a hierarchal manner for three reasons:

Test Suite Structure:



- **Traceability**
    - We need to have a mechanism for tracking each software feature from SRS to Test docs (Test cases, Test suites);
    - We need to make sure each requirement is covered in FS and Test cases;
    - This helps in establishing a measure of completeness;
        - Make sure each feature is represented;

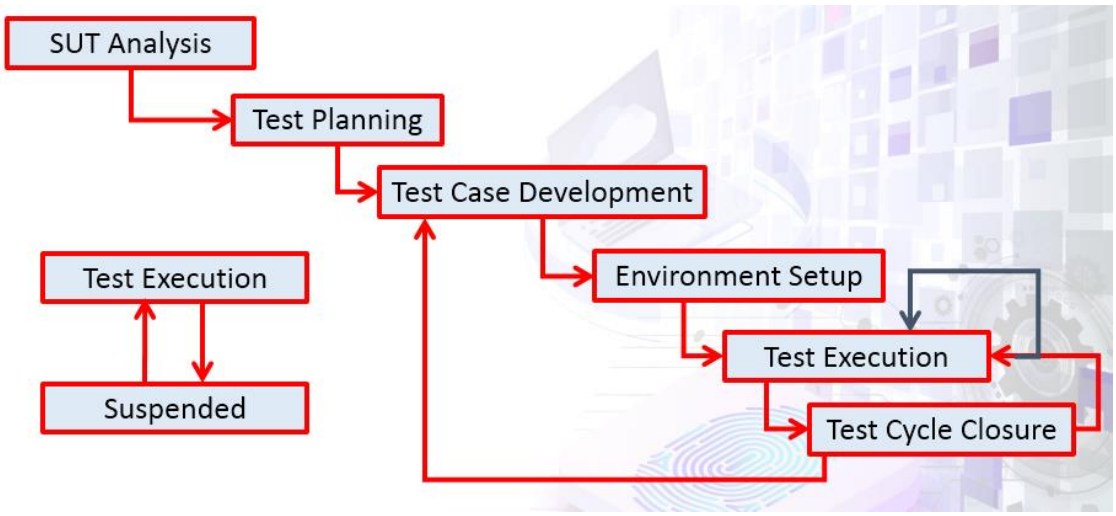
- Highlight gaps.

SRS Section	SDS Section	Test case	Notes
1.1. Validation of user login credentials.	4.1. login validation.	6.1.4. User login with proper credentials.	
		6.1.5. User login with invalid username.	
		6.1.6. User login with invalid password.	
1.2. Validation of credit card information.	7.2.4. Credit card information verification.	10.1.1. Valid credit card information input.	
		10.1.2. Invalid credit card number.	
		10.1.3. Invalid credit card name.	
		...	

Module 90:

Software Testing Lifecycle Phases

- Software Testing Life Cycle (STLC) is defined as a sequence of activities conducted to perform Software Testing.
- Contrary to popular belief, Software Testing is not a just a single activity.
- It consists of a series of activities carried out methodologically to help certify a particular software product.



- **STLC phases**
  - System under test (SUT) Analysis
  - Test Planning
  - Test case development
  - Test Environment setup
  - Test Execution
  - Test Cycle closure

Module 91:

Test Result Reporting

- Once testing activity (cycle) is compete, we need to share the results. Inputs are:
- Test Plan Document
  - Test Design Specification
  - Test Case Specification
  - Test Input Data and Output Data, Test Procedure Specification
- Test Incident Report
  - Test Log, Test Summary Report, Test Metrics, Coverage Analysis
- Program Quality Estimate

Overall progress of the QA cycle - 013	On time
Total number of test cases	227
Test cycle duration	5 days
Progress for 20/04/19	
Number of test cases planned	20
Number of test cases executed	18
Number of test cases executed overall	78
Number of defects encountered	2
Number of defect encountered so far	10
Number of critical defects- still open	3
Overall status	
Number of test cases planned	100
Number of test cases executed	78
Pass Percentage of the defects	98%
Defects density	2.5 per day
Critical defects percentage	20%

- **Consolidated test report form**
  - This may be output from bug reporting database
  - We conduct a meeting in which development representative, testing team representative and project management discusses each of the rows in test execution report (test status report)

Module	Scenarios	Sub Levels	Complexity	Date	Status	Defect ID	Severity	Status
Login	Successful login	Signup	Simple	17/3/19	Fail	#4573 - Session data doesn't persist on different tabs	1- High	Open
		Forgot password	Moderate		Blocked	1028		
		Sign in	Simple		Pass			

BUG LOG FORM						
5655	Widget not visible on UI	1. Login to the site using the admin credentials 2. Click on E-course management. 3. Select e-course 4. Share e-course using social media widgets Expected result: The widgets should be visible on each e-course page Actual Result: Widget not visible	1- High	Open	Tester X	Dev representative

- Screen shot of bug

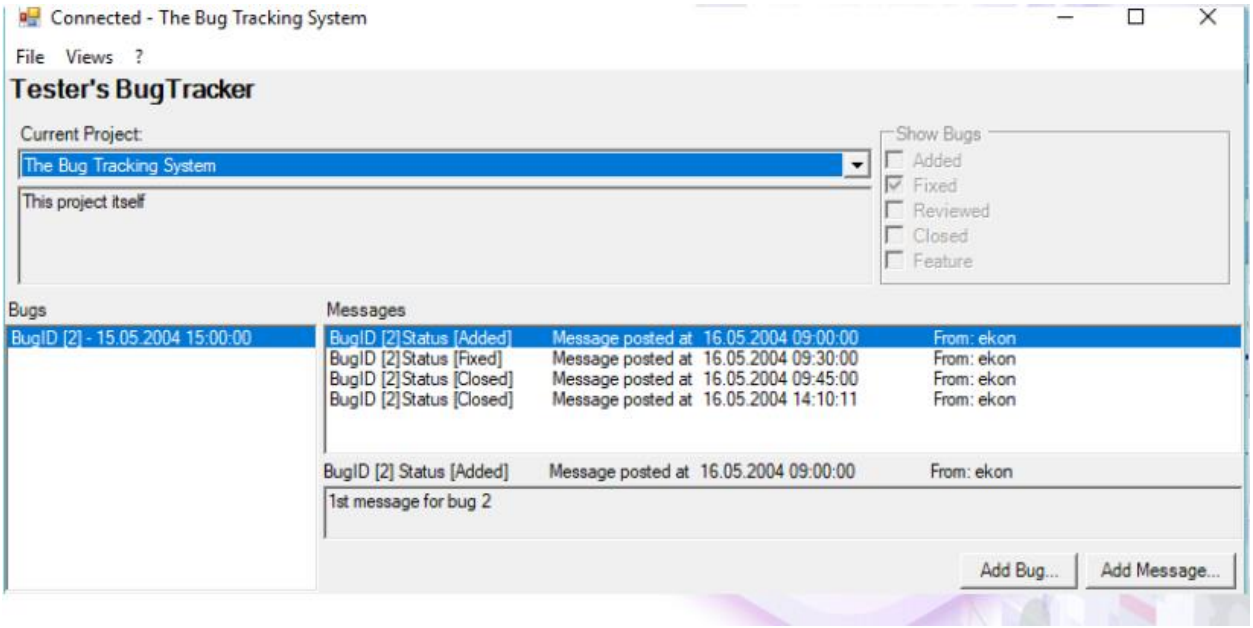


Module 92:

Bug Tracking Databases

- We cannot use documents or spreadsheets for recording the test execution results
- At the simplest, we have small scale bug tracking, open source, software having a backend in some DBMS.
- On the professional side, we have test management software e.g.,
  - Microsoft Test Manager (MTM)
  - JIRA
  - IBM Rational Test Manager

- Example open source bug tracking software, available as BTSys



- These systems allow testers to report
  - Faults
  - Issues
- Allow developers to view these faults
  - Fix these faults
- Allow management to review the progress.
- Allow analysis

Module 93:

Test Data Management

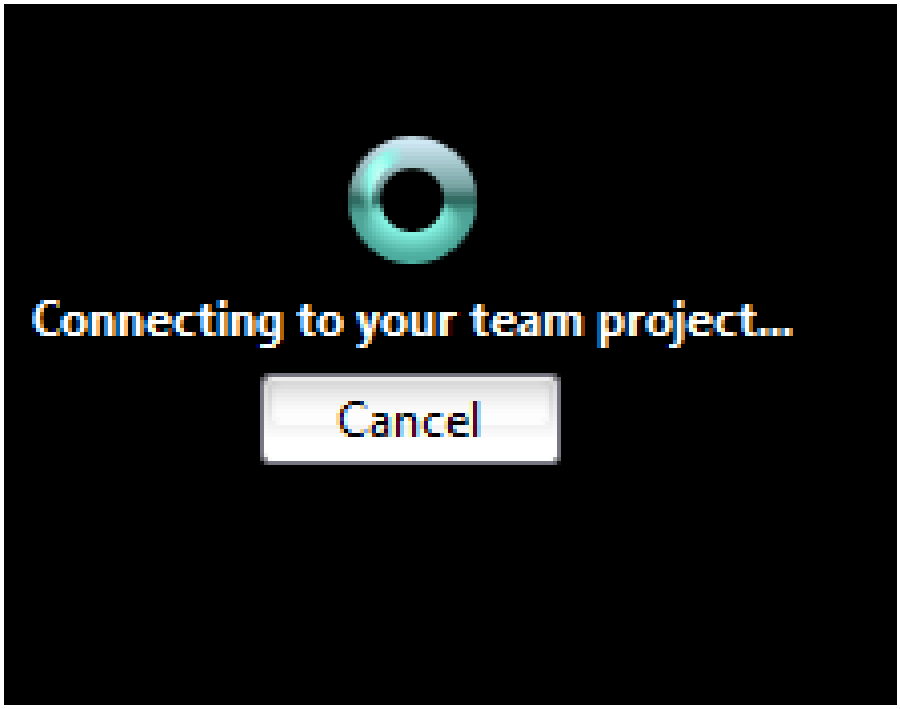
- While testing, we input data to make test cases execution possible
- We keep track of test cases in a traceability matrix and know which test cases are related to which part of SRS, SDS etc.
- Once test cases are executed, we also collect test execution data, how many test cases were executed, how many passed, failed, suspended or blocked
- Therefore, we have
    - Test oracles
    - Test execution resulting data in the shape of test execution logs
    - Traceability matrix
- Once testing is complete, we maintain clean-up information
- Another important aspect is the storage of automated test cases, bug logs, screenshots etc.
- We keep record of all of these to infer valuable information such as:
    - Product quality metric
    - Expected completion time
- We emphasize on aspects of test data management considering automated testing tools providing support for:
    - Test case recording
    - Test case execution
    - Bug log recording
    - Analysis of results
- we study usage of Microsoft Test Manager (MTM)

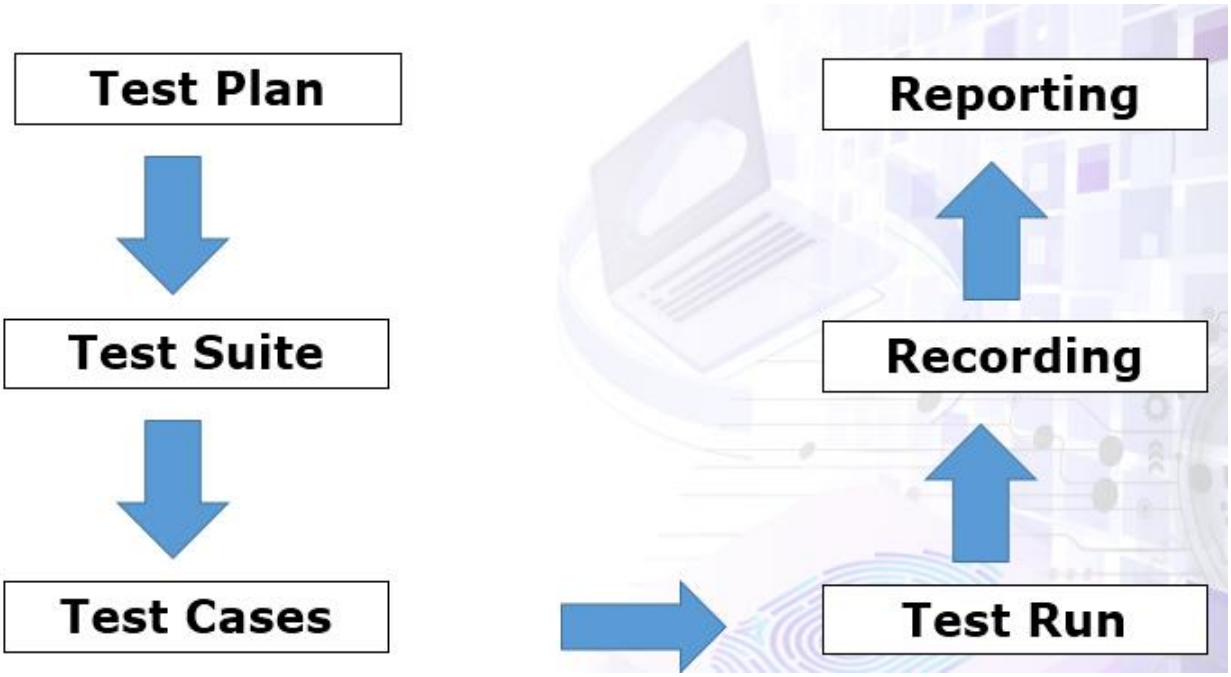
Module 94:

Test Data Management with MTM (Microsoft Test Manager)

- Test data management is important for many reasons
    - Traceability
    - In process fault arrival for quality assessment
    - Fault predictions
    - Fault removal effectiveness
    - Expected test cycle completion
- We study a tool for test data management

- **System Under Test**
    - We have a learning management system which is under development and we treat it is SUT.
    - We write test cases, execute them and record
- Login MTM using Team Foundation Server





Module 95:

Test Planning with MTM (Microsoft Test Manager)

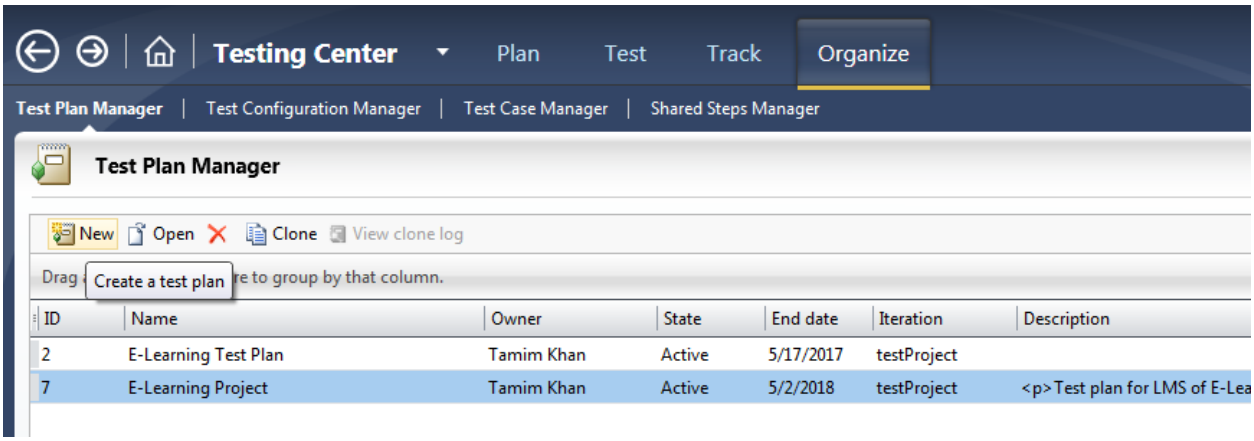
- We undertake steps involved in Software Testing Lifecycle (STLC) as follows:
  - System under test (SUT) Analysis
  - Test Planning
  - Test case development
  - Test Environment setup
  - Test Execution
  - Test Cycle closure

- **Application analysis:**

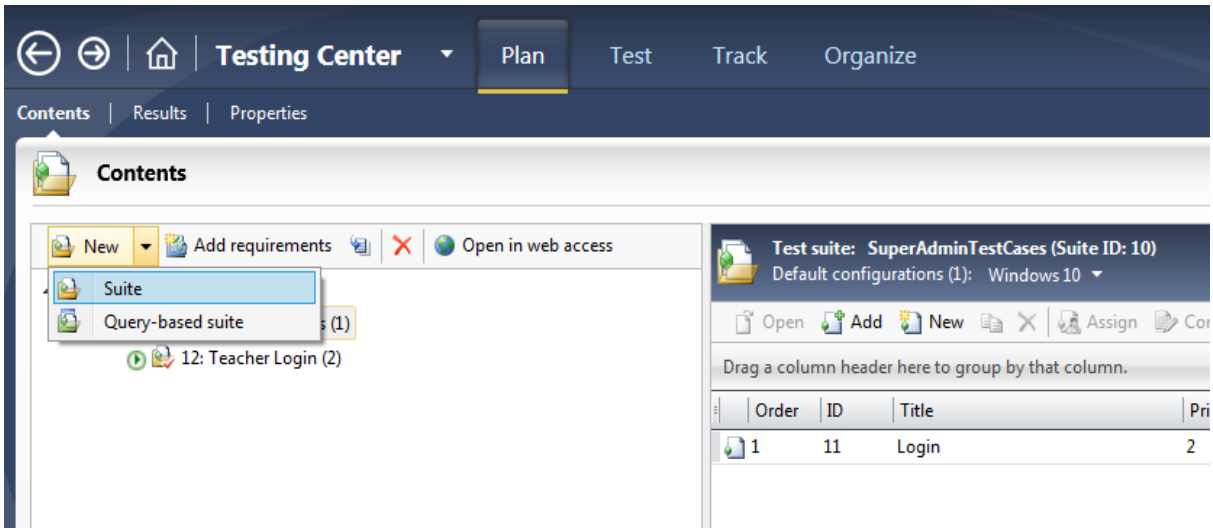
- We have a web based system under test that includes:
    - Learning management system offering online lectures, assessments and practice sessions
    - Teachers can plan their schedule.
    - Students can access online content for learning
    - Campus management system to provide support for related activities

- **Test planning**
    - We develop
        - Test plan
        - Test suites
        - Test cases
    - We make use of Microsoft Test Manager (MTM)

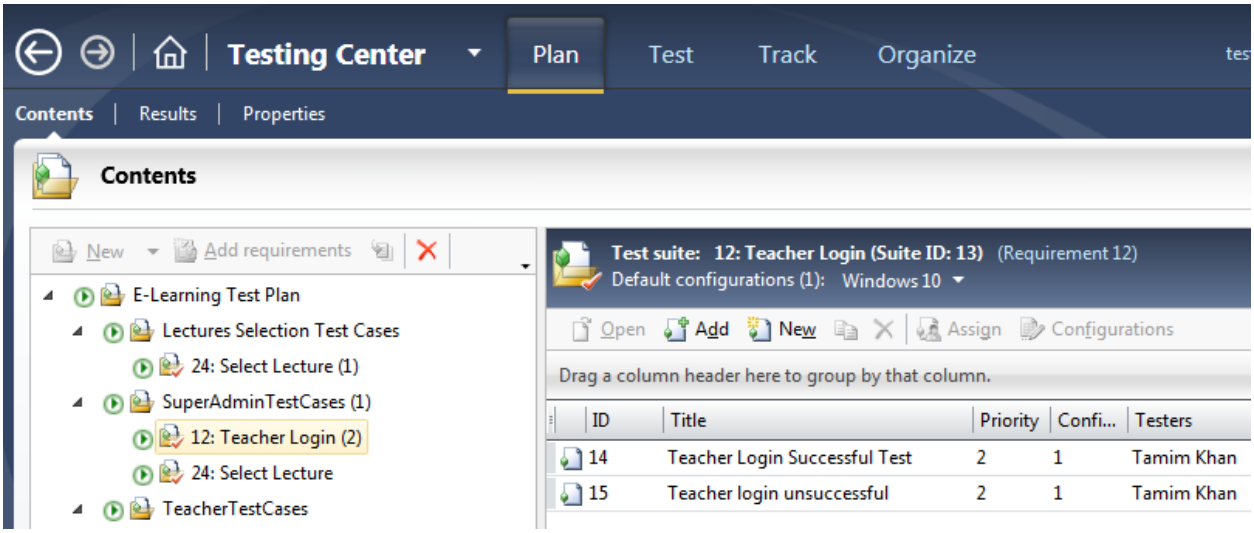
- **Create A New Test Plan**



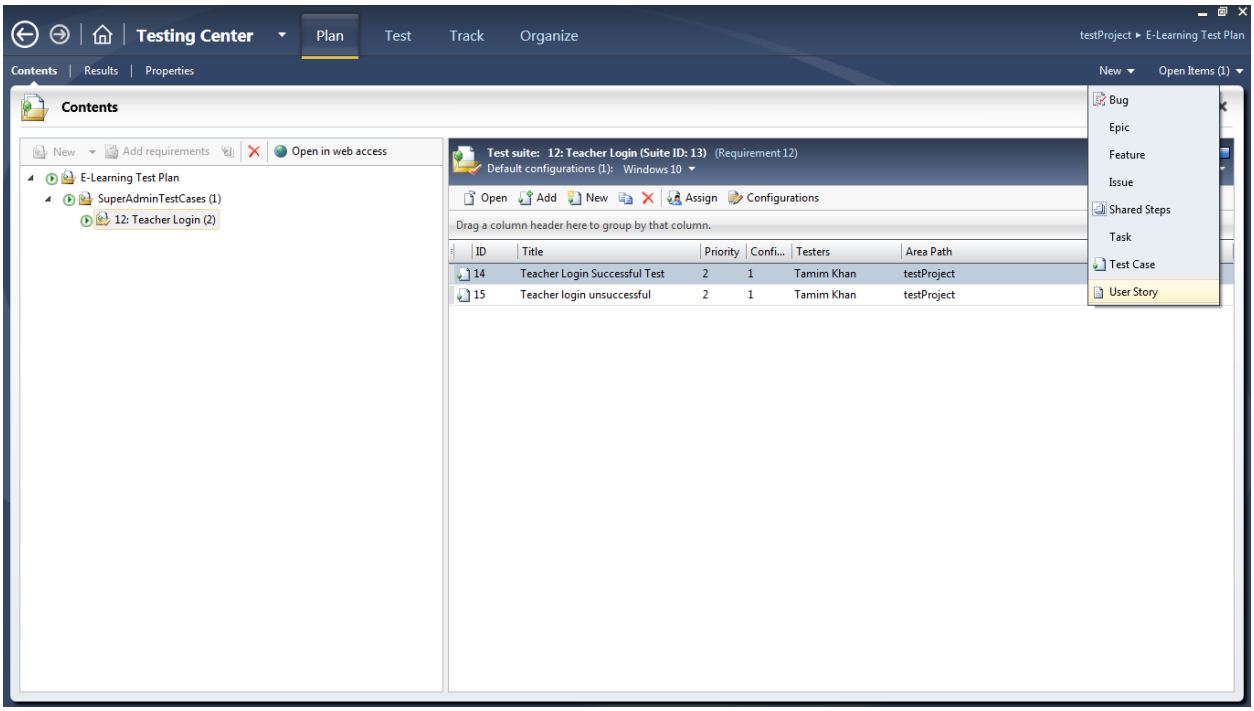
- **Create A New Test Suite**



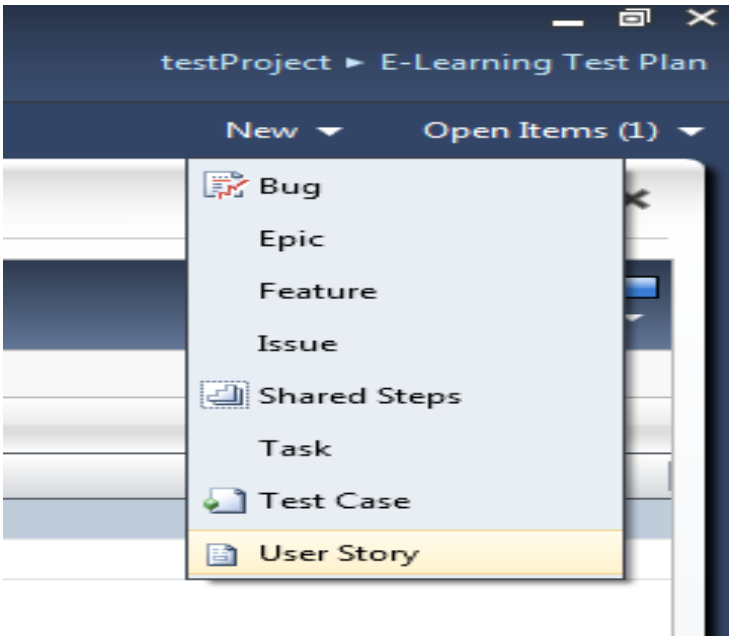
- **Create A New Test Suite**



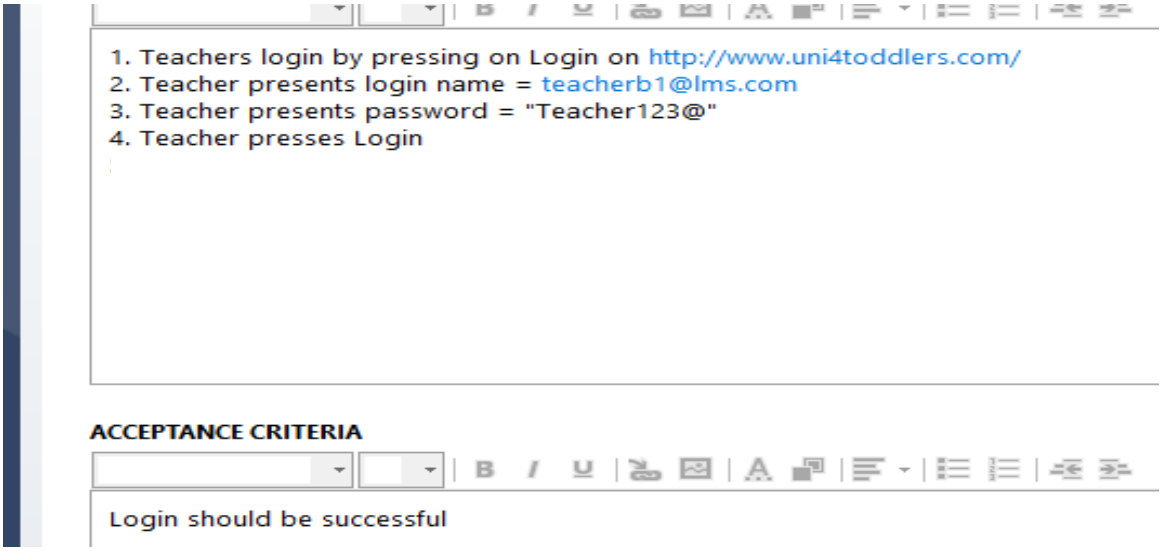
- **Create A Test Case**



- **Create A Test Case**
- **Steps:**
  - We add a test requirement
  - Add a test case to test that requirement



- **Adding a test requirement**

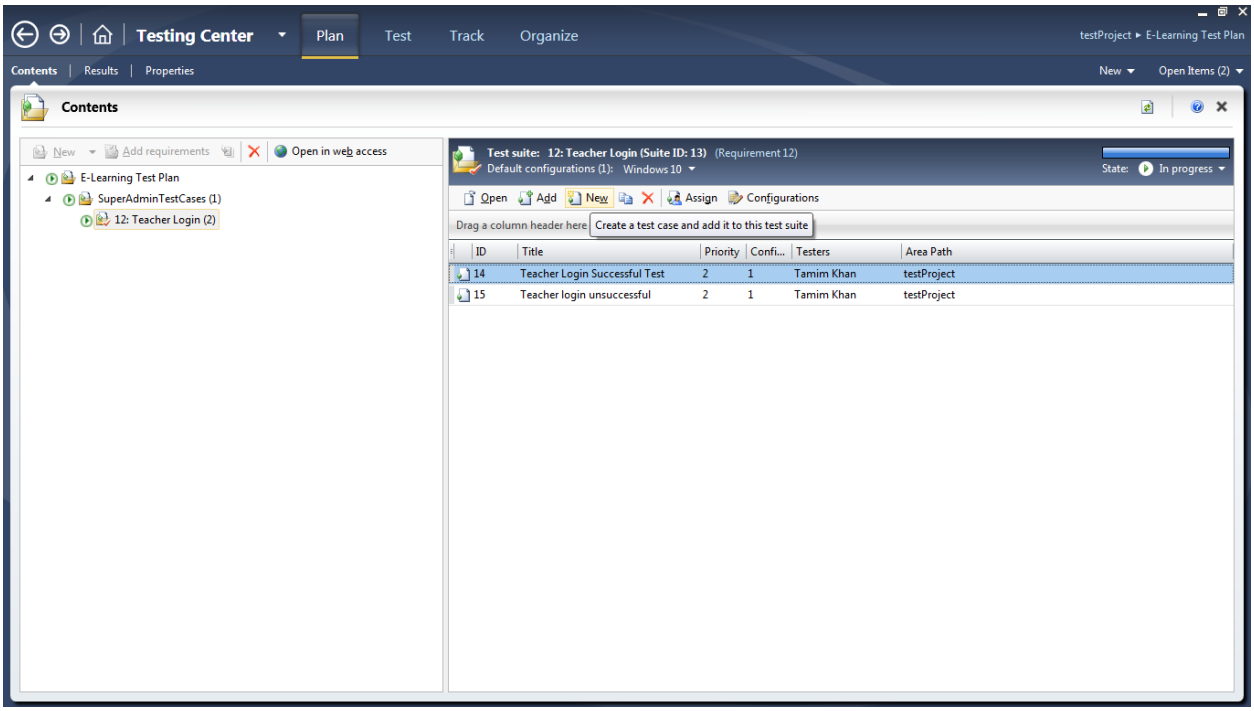


Module 96:

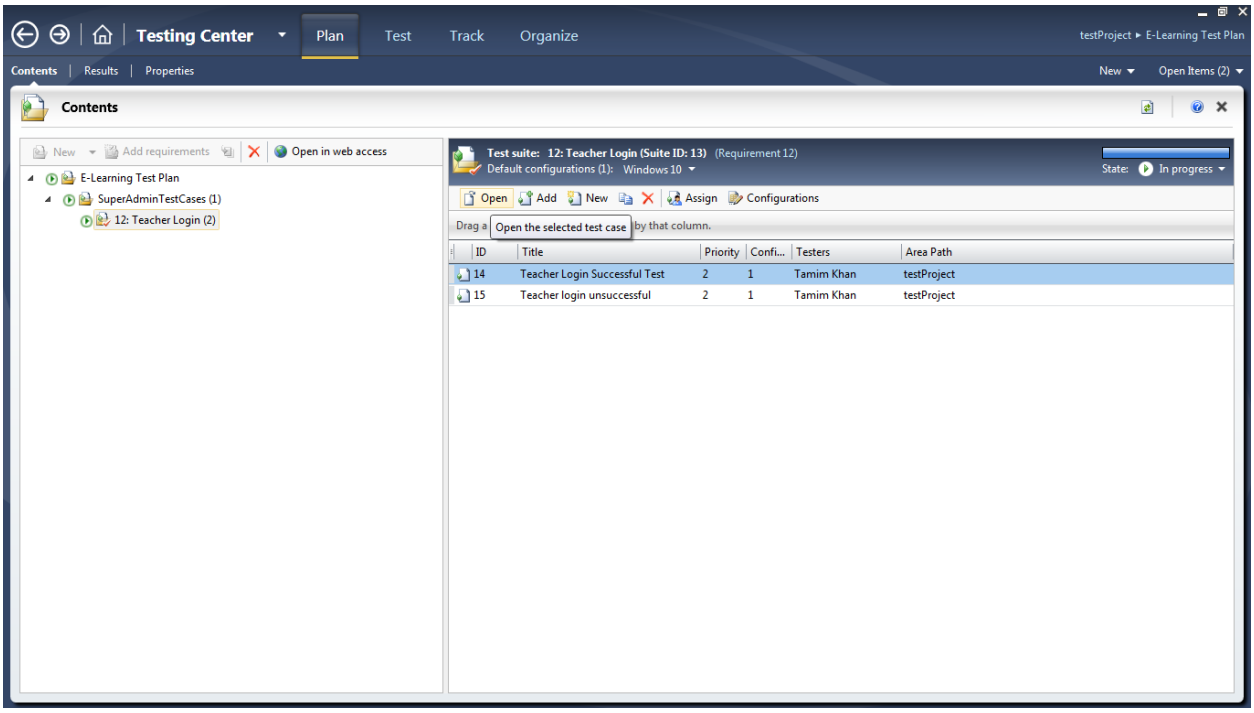
Test Case Development with MTM (Microsoft Test Manager)

- We undertake steps involved in Software Testing Lifecycle (STLC) as follows:
  - ✓ System under test (SUT) Analysis
  - ✓ Test Planning
  - ✓ Test case development
  - ✓ Test Environment setup
  - ✓ Test Execution
  - ✓ Test Cycle closure

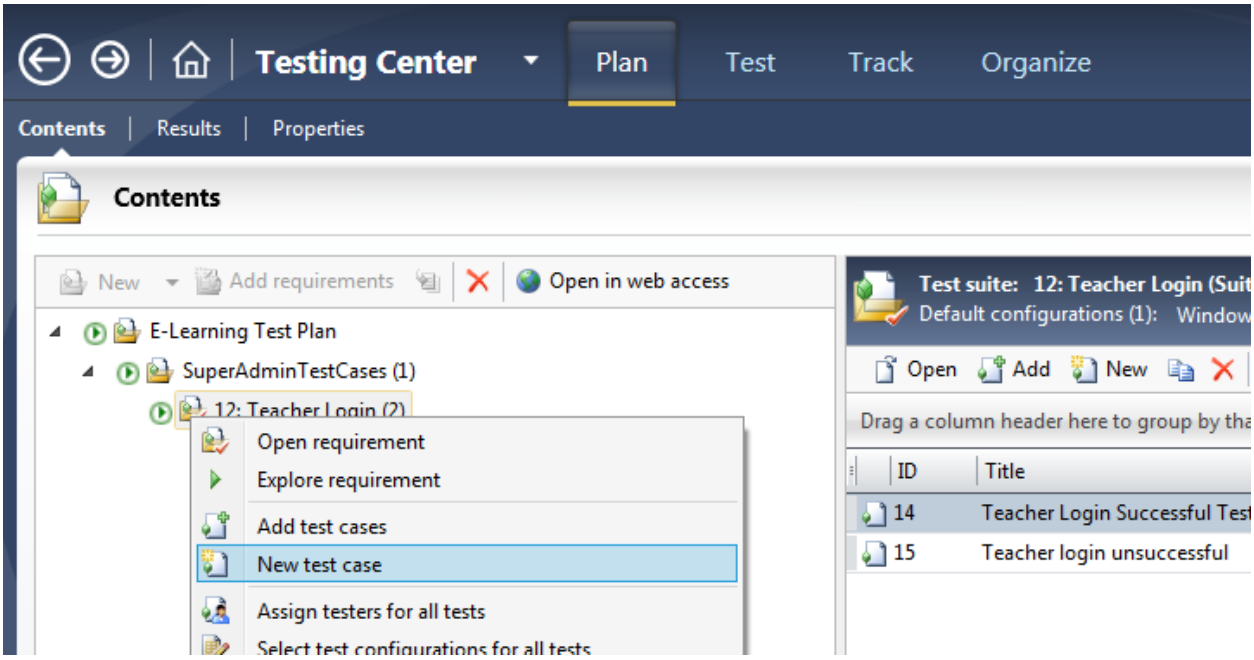
- **Adding a test case**



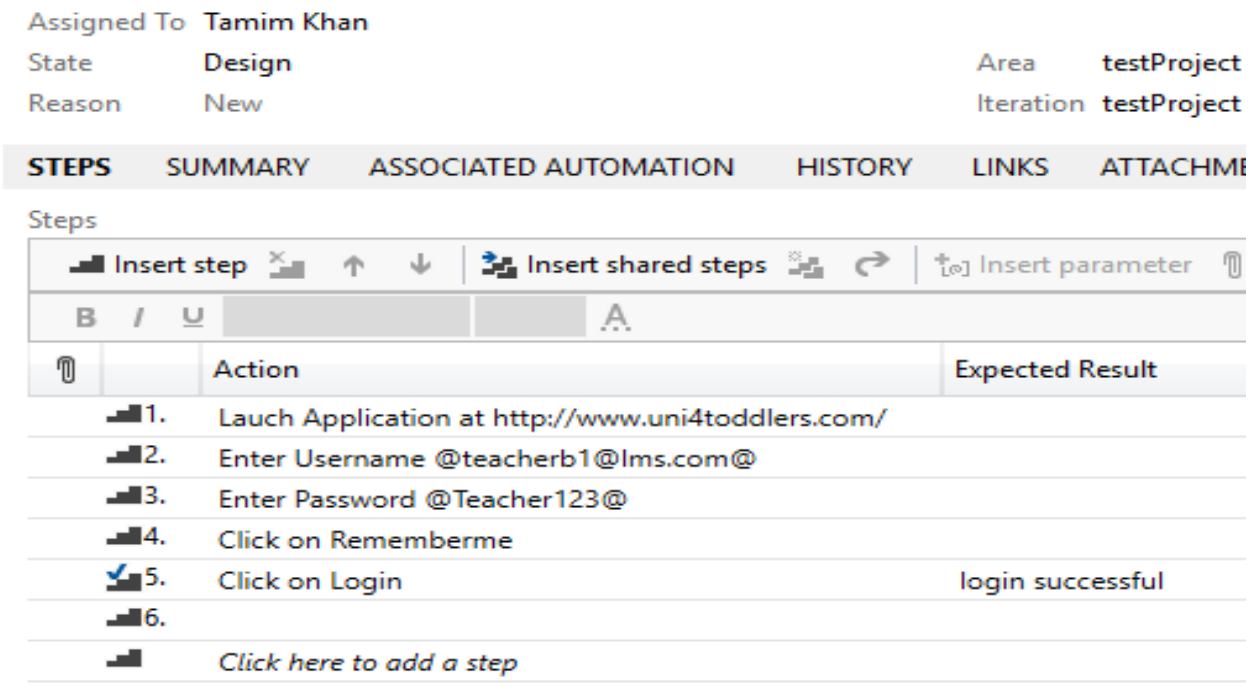
- **Opening a test case**



- **Opening a test case**



- **Recording a test case**

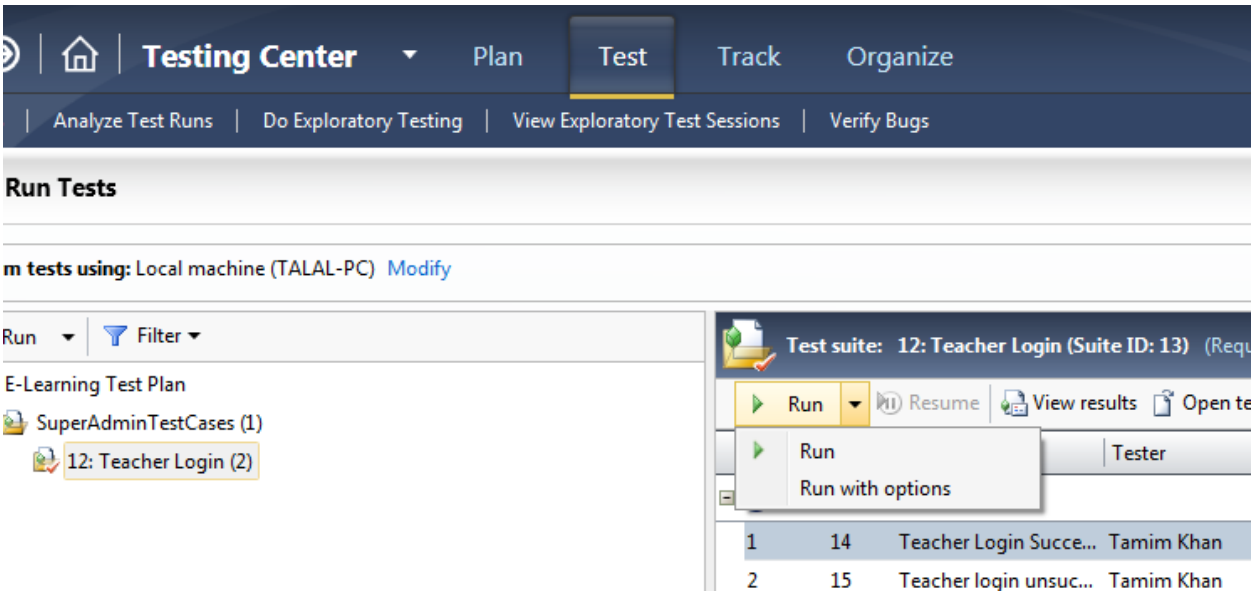


Module 97:

Test Execution with MTM (Microsoft Test Manager)

- We undertake steps involved in Software Testing Lifecycle (STLC) as follows:
  - ✓ System under test (SUT) Analysis
  - ✓ Test Planning
  - ✓ Test case development
  - ✓ Test Environment setup
  - ✓ Test Execution
    - Recording
    - Reporting
    - Bug Logs
  - ✓ Test Cycle closure

- **Test execution with MTM**



Module 98:

Test Execution Recording with MTM

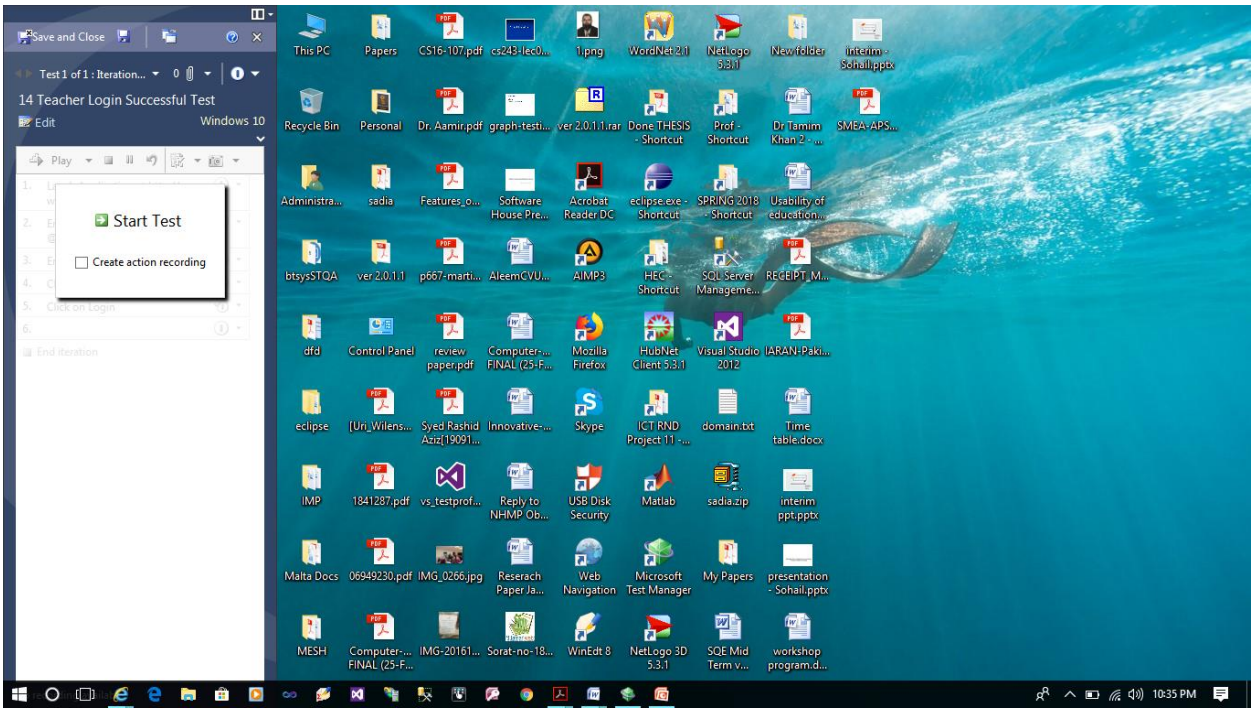
- We undertake steps involved in Software Testing Lifecycle (STLC) as follows:
  - ✓ System under test (SUT) Analysis
  - ✓ Test Planning
  - ✓ Test case development
  - ✓ Test Environment setup
  - ✓ Test Execution
  - ✓ Test Cycle closure

- We undertake steps involved in Software Testing Lifecycle (STLC) as follows:
  - ✓ System under test (SUT) Analysis
  - ✓ Test Planning
  - ✓ Test case development
  - ✓ Test Environment setup
  - ✓ Test Execution
    - Execution Recording
    - Execution Reporting
    - Execution Bug Logs
  - ✓ Test Cycle closure

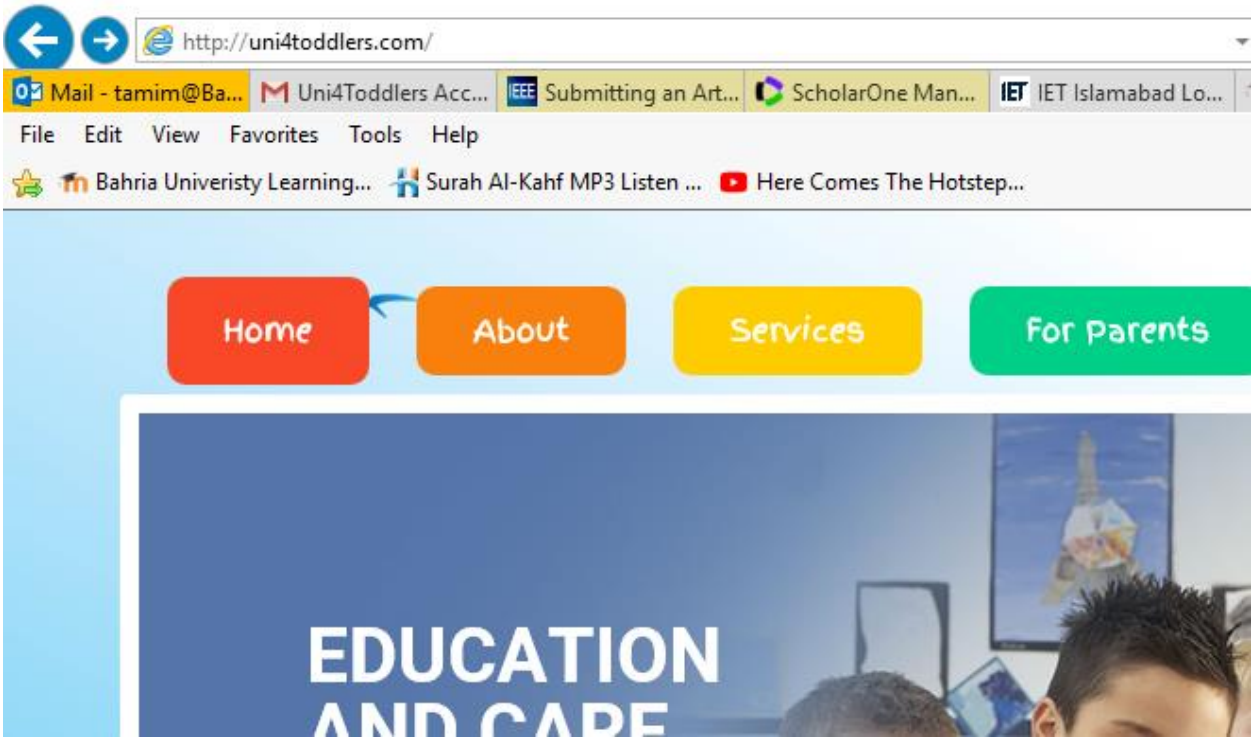
- We undertake steps involved in Software Testing Lifecycle (STLC) as follows:

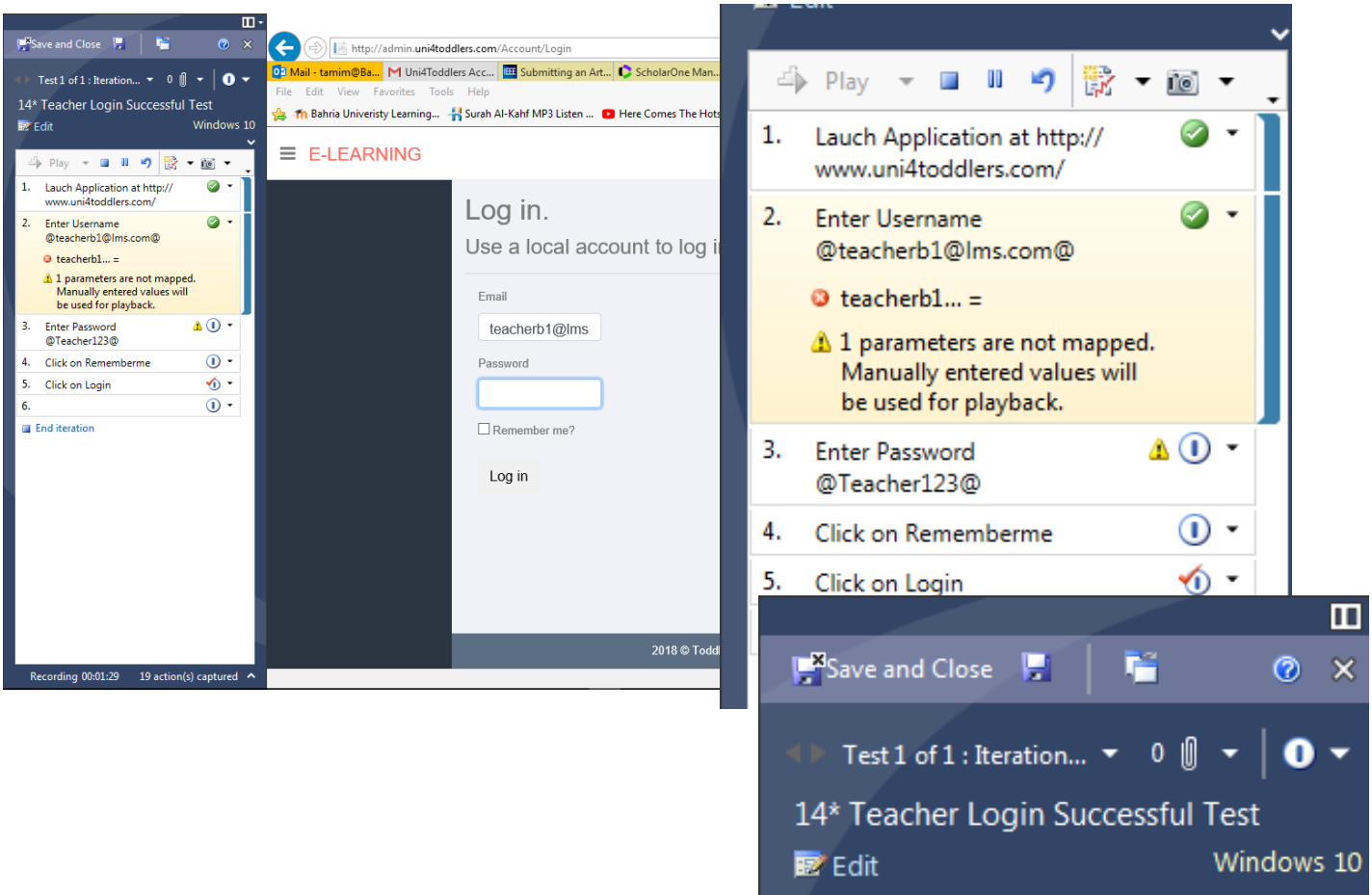
- ✓ System under test (SUT) Analysis
- ✓ Test Planning
- ✓ Test case development
- ✓ Test Environment setup
- ✓ Test Execution
  - Execution Recording
  - Execution Reporting
  - Execution Bug Logs
- ✓ Test Cycle closure

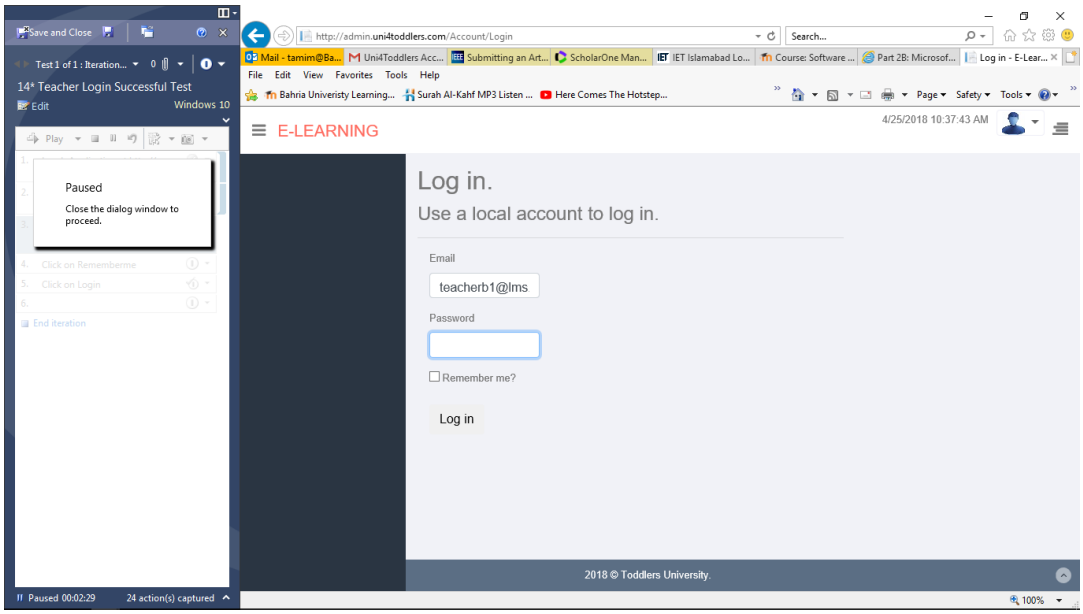
- **Test Case Execution Recording**

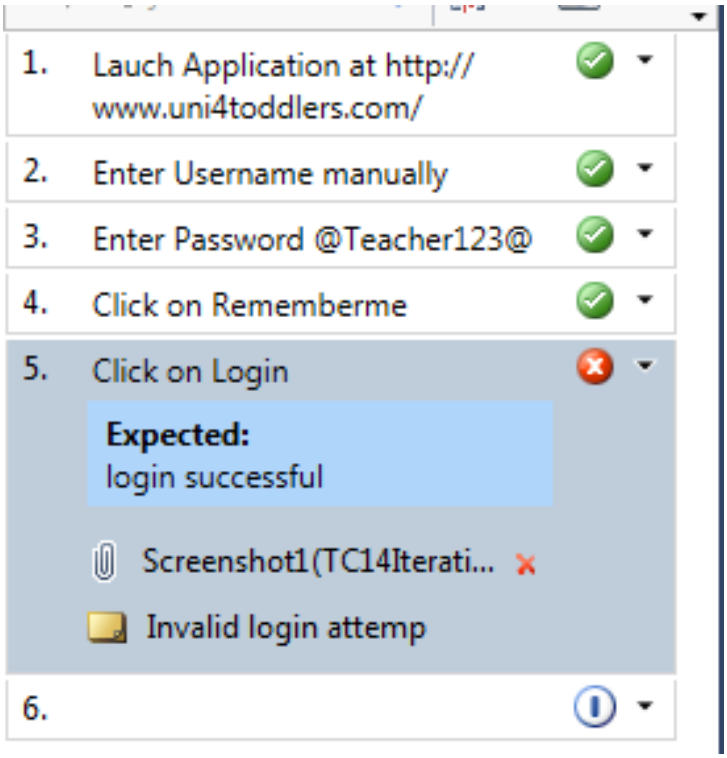


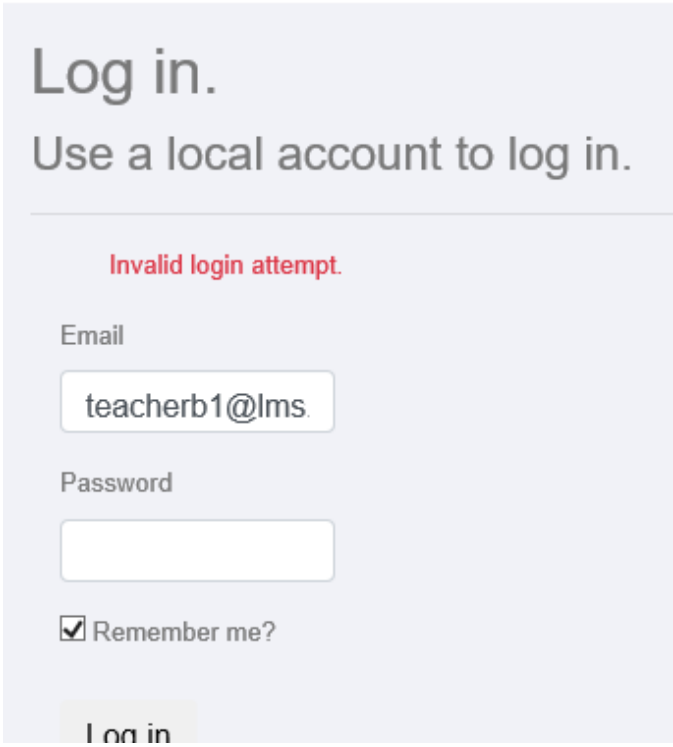
- **Test Case Execution Recording**

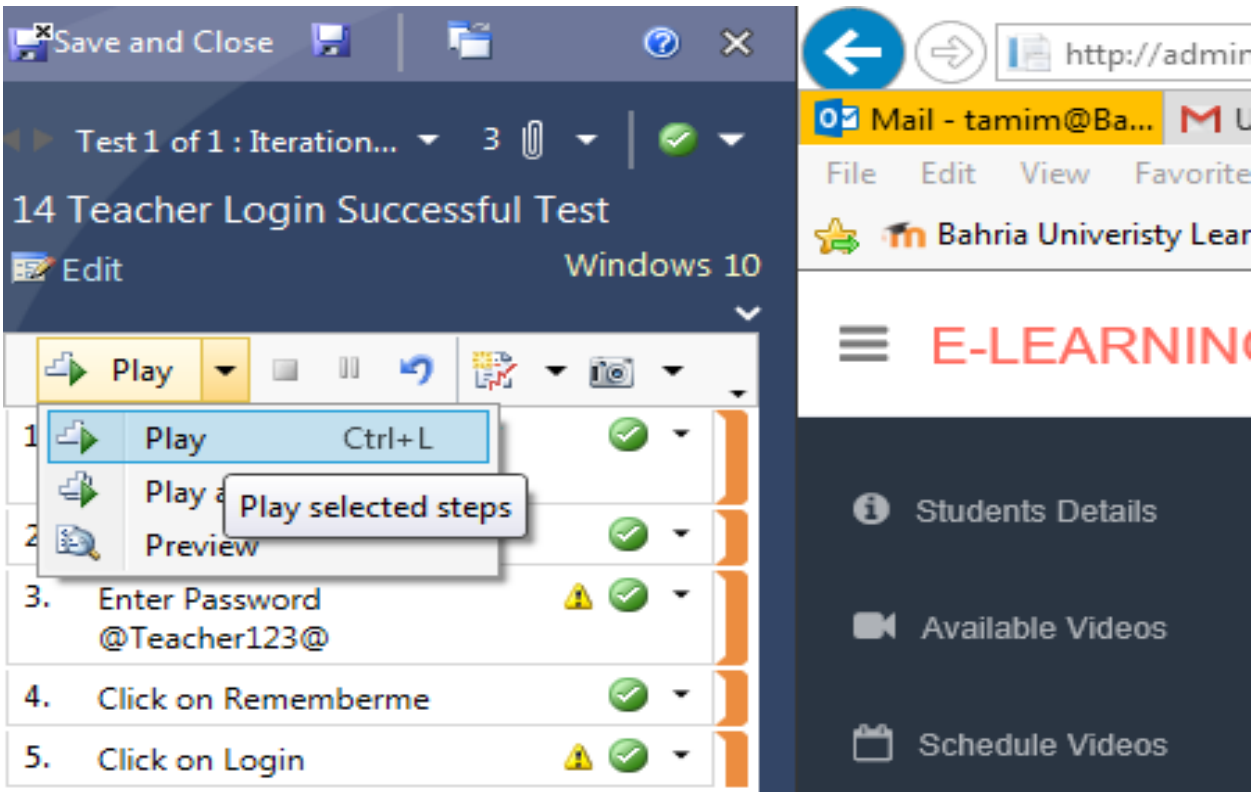

http://uni4toddlers.com/
Mail - tamim@Ba...
Uni4Toddlers Acc...
Submitting an Art...
ScholarOne Man...
IET Islamabad Lo...
File Edit View Favorites Tools Help
Bahria Univeristy Learning...
Surah Al-Kahf MP3 Listen ...
Here Comes The Hotstep...
Home
About
Services
For Parents
EDUCATION


Save and Close
Test 1 of 1 : Iteration...
14* Teacher Login Successful Test
Edit
Windows 10
Play
1. Lauch Application at http://www.uni4toddlers.com/
2. Enter Username @teacherb1@lms.com@
teacherb1... =
1 parameters are not mapped. Manually entered values will be used for playback.
3. Enter Password @Teacher123@
4. Click on Rememberme
5. Click on Login
6.
End iteration
Recording 00:01:29
19 action(s) captured
http://admin.uni4toddlers.com/Account/Login
E-LEARNING
Log in.
Use a local account to log i
Email
teacherb1@lms
Password
Remember me?
Log in
2018 © Todd


Save and Close
Test 1 of 1 : Iteration...
14* Teacher Login Successful Test
Edit
Windows 10
Play
Paused
Close the dialog window to proceed.
Paused 00:02:29
24 action(s) captured
http://admin.uni4toddlers.com/Account/Login
Search...
File Edit View Favorites Tools Help
E-LEARNING
4/25/2018 10:37:43 AM
Log in.
Use a local account to log in.
Email
teacherb1@lms
Password
Remember me?
Log in
2018 © Toddlers University.
100%


1. Lauch Application at http://www.uni4toddlers.com/
2. Enter Username manually
3. Enter Password @Teacher123@
4. Click on Rememberme
5. Click on Login
Expected:
login successful
Screenshot1(TC14Iterati...
Invalid login attemp
6.

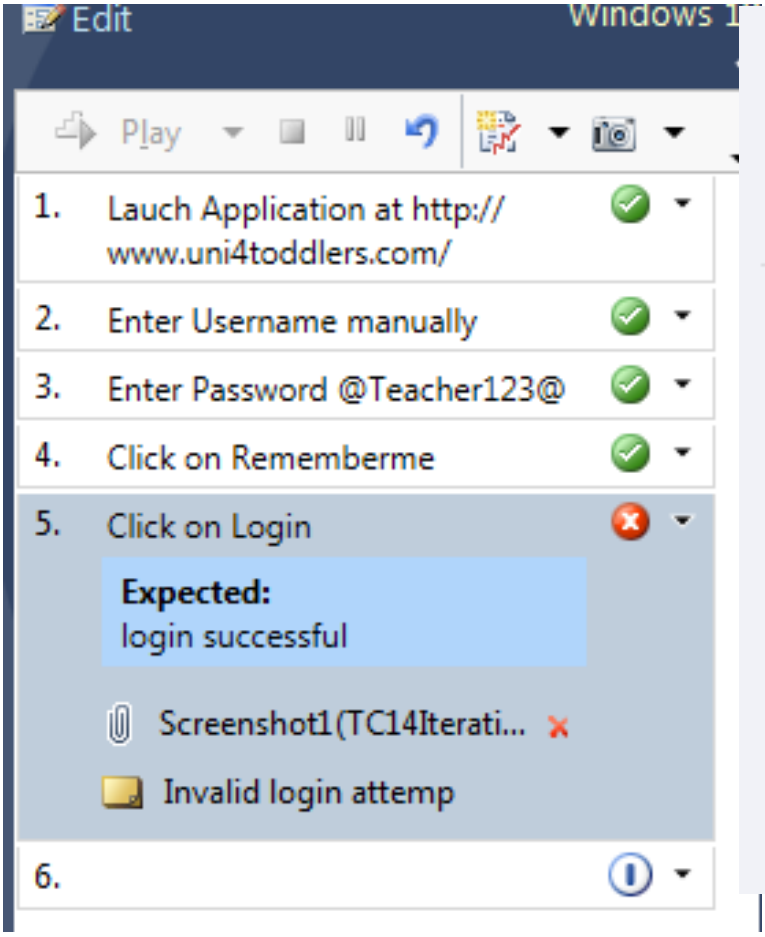

Log in.
Use a local account to log in.
Invalid login attempt.
Email
teacherb1@lms
Password
Remember me?
Log in

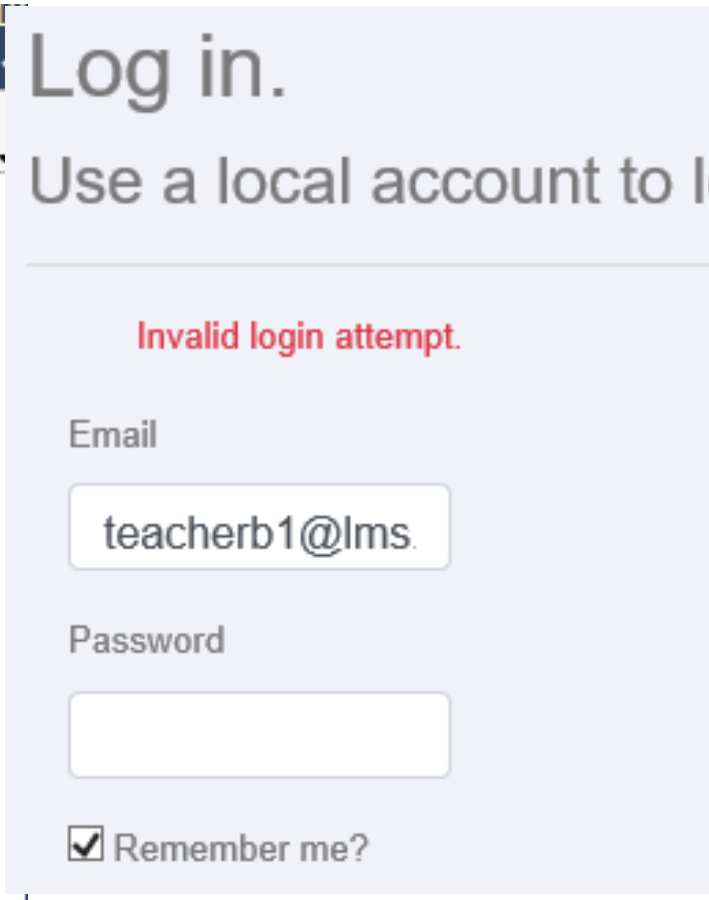


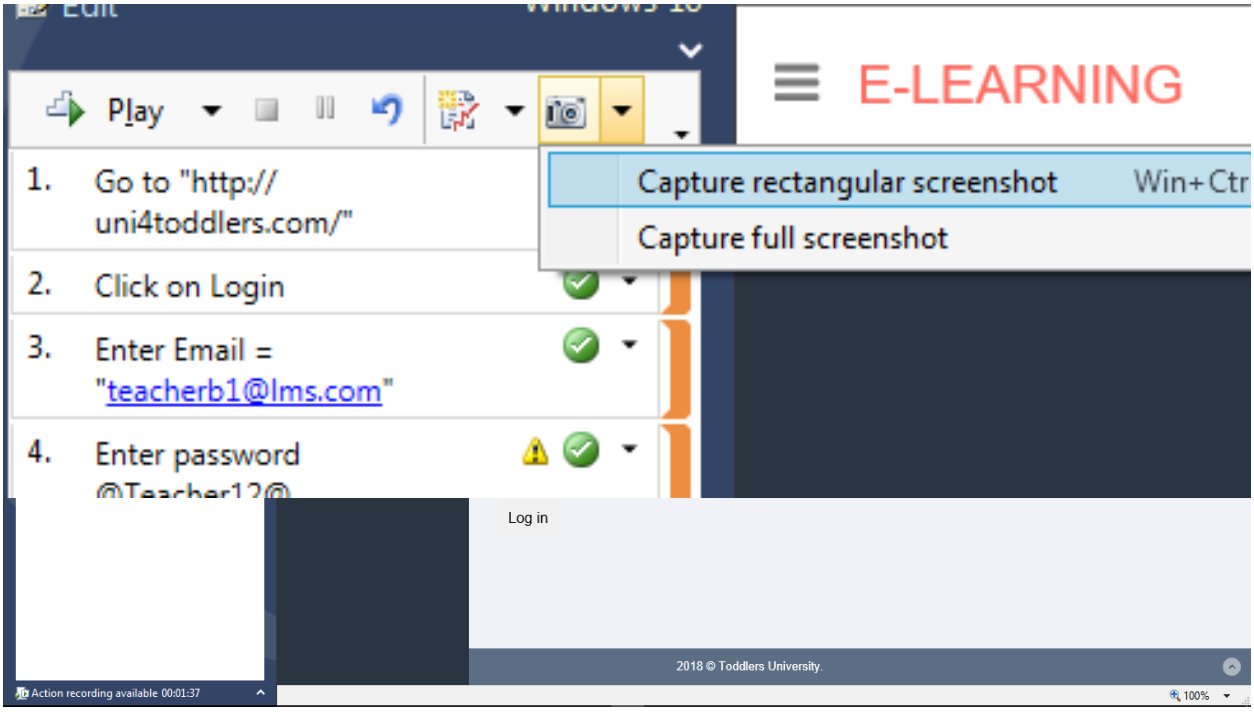
Module 99:

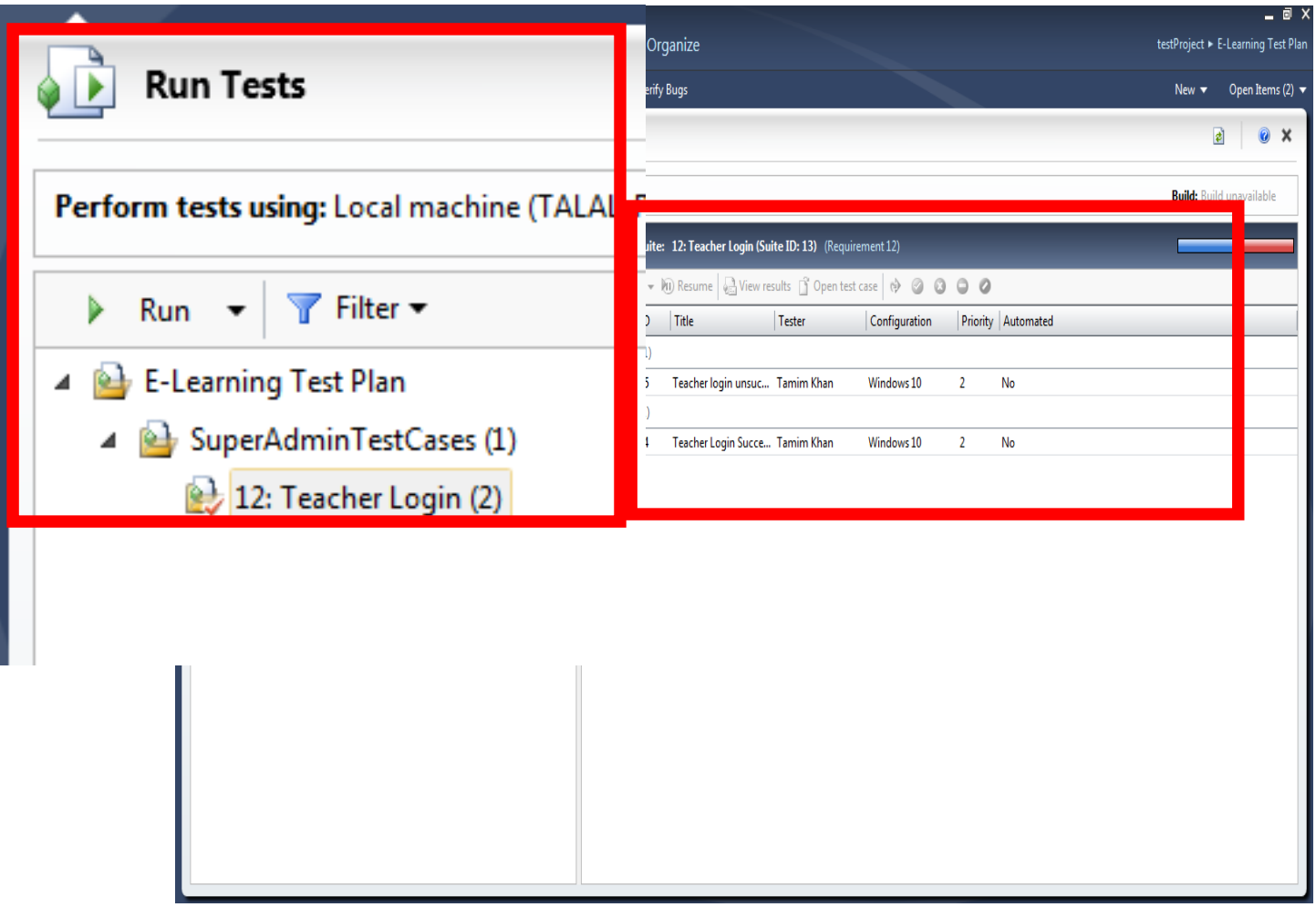
Test Execution Reporting with MTM

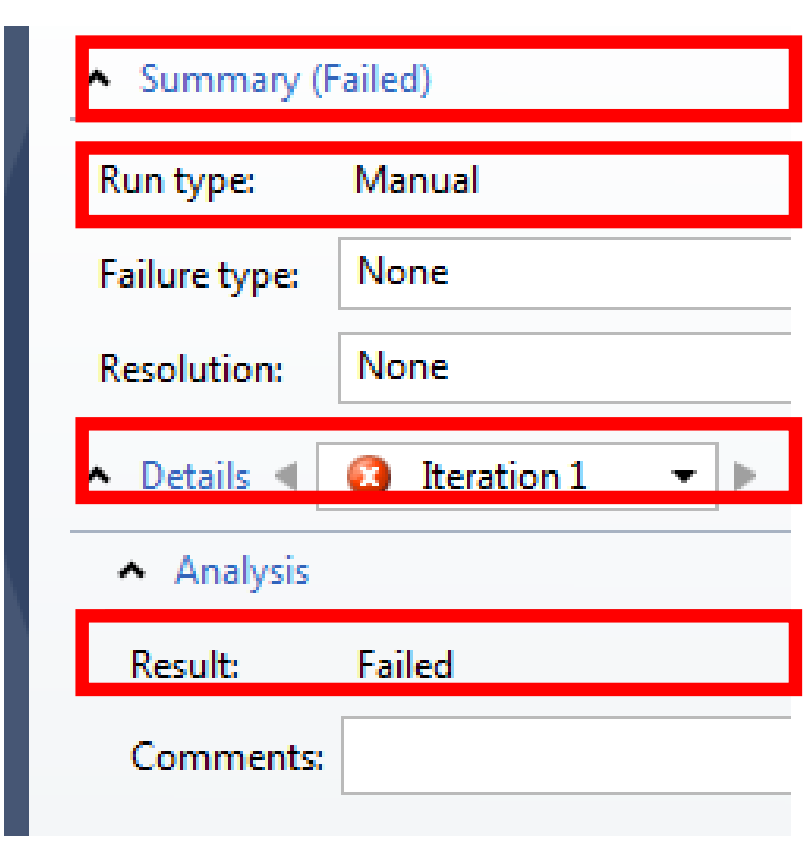
- We undertake steps involved in Software Testing Lifecycle (STLC) as follows:
  - ✓ System under test (SUT) Analysis
  - ✓ Test Planning
  - ✓ Test case development
  - ✓ Test Environment setup
  - ✓ Test Execution
    - ✓ Execution Recording
    - ✓ Execution Reporting
    - ✓ Execution Bug Logs
  - ✓ Test Cycle closure
- **Test Case Execution Reporting**

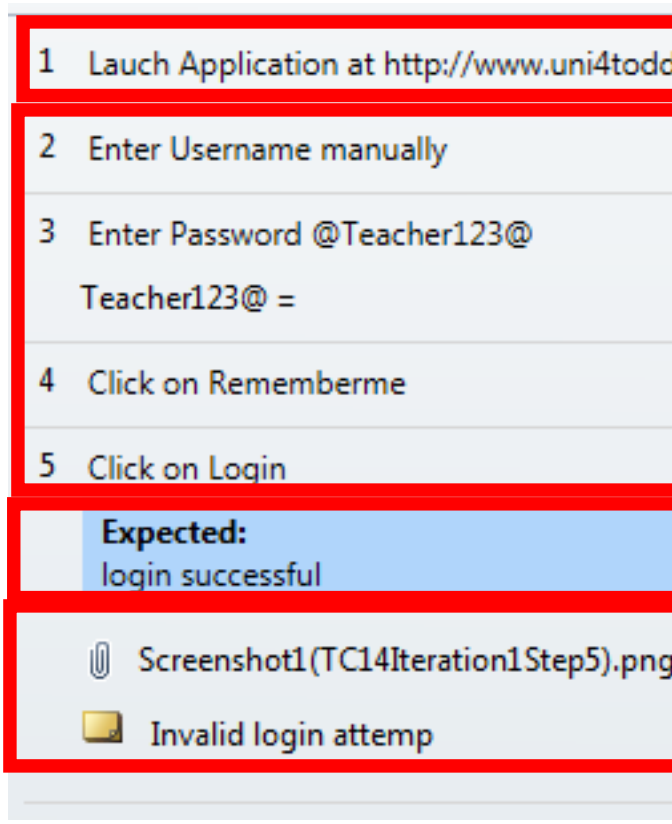

Edit
Play
1. Lauch Application at http://www.uni4toddlers.com/
2. Enter Username manually
3. Enter Password @Teacher123@
4. Click on Rememberme
5. Click on Login
Expected:
login successful
Screenshot1(TC14Iterati...
Invalid login attemp
6.


Log in.
Use a local account to l
Invalid login attempt.
Email
teacherb1@lms
Password
Remember me?


Play
E-LEARNING
1. Go to "http://
uni4toddlers.com/"
2. Click on Login
3. Enter Email =
"teacherb1@lms.com"
4. Enter password
Capture rectangular screenshot
Win+Ctr
Capture full screenshot
Log in
2018 © Toddlers University.
Action recording available 00:01:37
100%


Run Tests
Perform tests using: Local machine (TALAL
Run
Filter
E-Learning Test Plan
SuperAdminTestCases (1)
12: Teacher Login (2)
Organize
testProject ▸ E-Learning Test Plan
New
Open Items (2)
Build: Build unavailable
12: Teacher Login (Suite ID: 13) (Requirement 12)
Resume
View results
Open test case
Title
Tester
Configuration
Priority
Automated
Teacher login unsuc... Tamim Khan Windows 10 2 No
Teacher Login Succe... Tamim Khan Windows 10 2 No


Summary (Failed)
Run type: Manual
Failure type: None
Resolution: None
Details
Iteration 1
Analysis
Result: Failed
Comments:

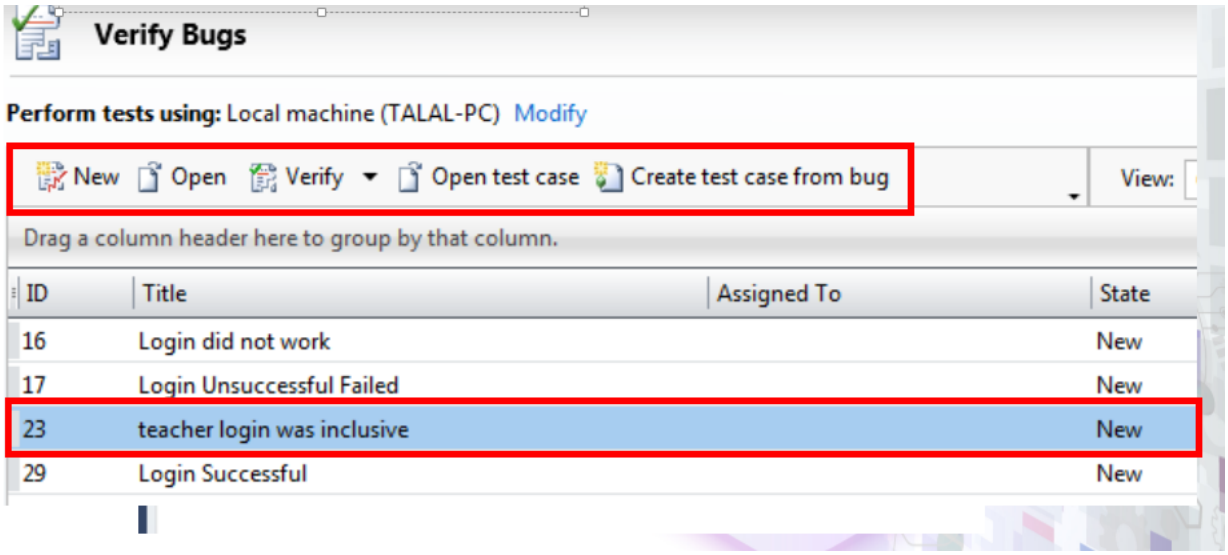

1 Lauch Application at http://www.uni4todo
2 Enter Username manually
3 Enter Password @Teacher123@
Teacher123@ =
4 Click on Rememberme
5 Click on Login
Expected:
login successful
Screenshot1(TC14Iteration1Step5).png
Invalid login attemp

Module 100:

Bug Logs with MTM

- We undertake steps involved in Software Testing Lifecycle (STLC) as follows:
  - ✓ System under test (SUT) Analysis
  - ✓ Test Planning
  - ✓ Test case development
  - ✓ Test Environment setup
  - ✓ Test Execution
    - ✓ Execution Recording
    - ✓ Execution Reporting
    - ✓ Execution Bug Logs
  - ✓ Test Cycle closure

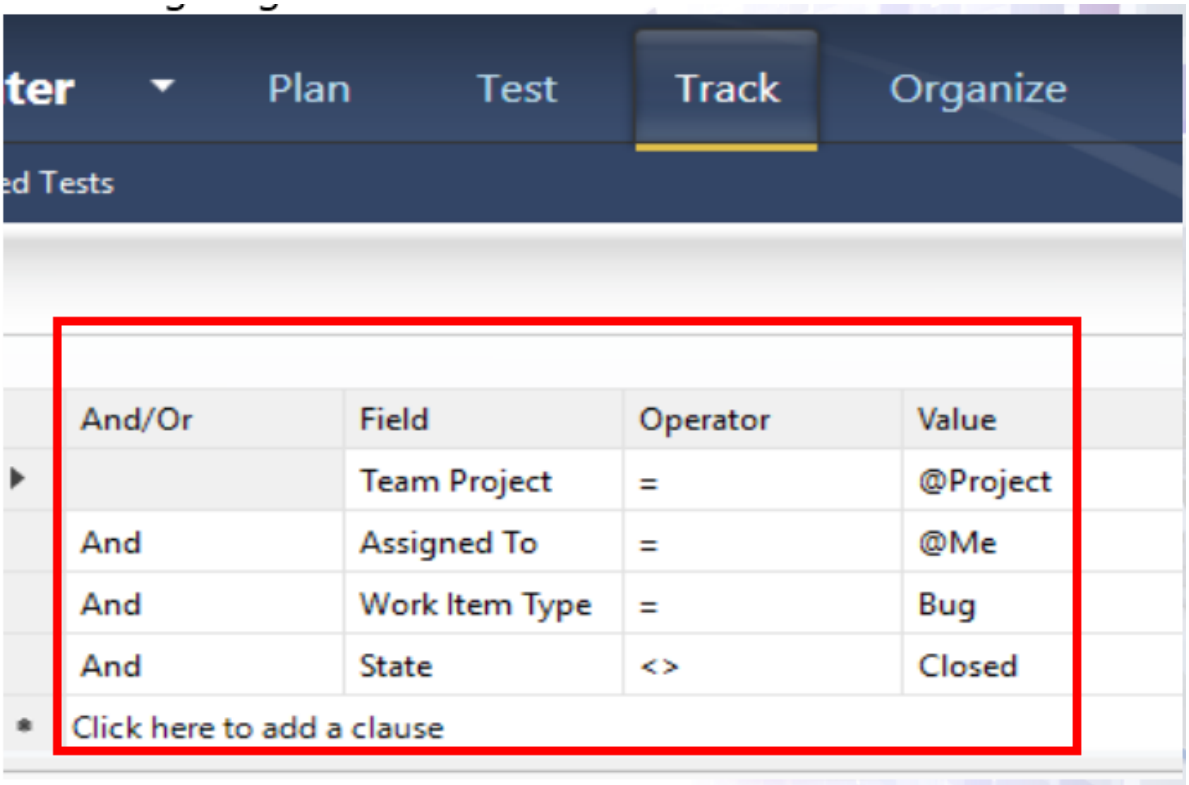
- **Execution Bug Log**

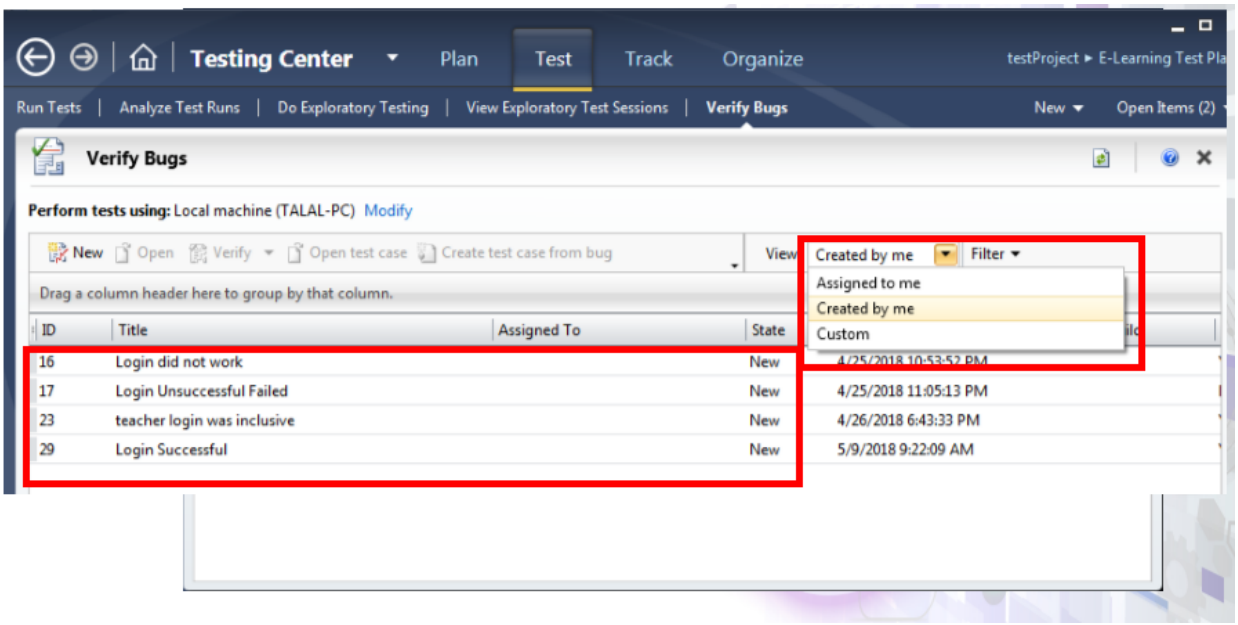


mstsc.exe , TabTip.exe , mtm.exe , ois.exe , mspaint.exe , WLXPhotoGallery.exe

Diagnostic Data Adapter
Action Log
System Information



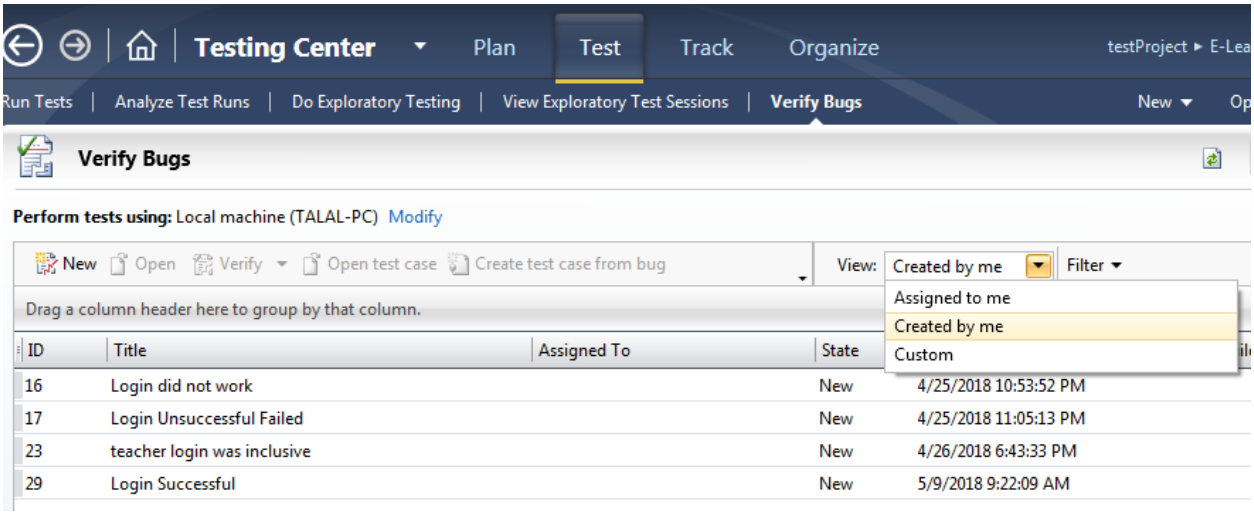




Module 101:

Test Cycle Closure with MTM

- We undertake steps involved in Software Testing Lifecycle (STLC) as follows:
  - ✓ System under test (SUT) Analysis
  - ✓ Test Planning
  - ✓ Test case development
  - ✓ Test Environment setup
  - ✓ Test Execution
    - ✓ Execution Recording
    - ✓ Execution Reporting
    - ✓ Execution Bug Logs
  - ✓ Test Cycle closure



Module 102:

Regression Testing

- **Regression Testing**
  - Testing activities occur after software changes.
  - Regression testing usually refers to testing activities during software maintenance phase.
- **Major testing objectives:**
  - Retest changed components (or parts)
  - Check the affected parts (or components)

- **Regression Testing**
  - Software evolve over time.
  - Reasons may include:
    - Requirements / design changes due to:
      - Addition of new functionality
      - Modification in existing functionality
    - Design changes
    - Bug fixing
    - Maintenance
  - We need to validate if the changes have not caused any regression in the rest of the system
- **Regression testing at different levels:**
  - Regression testing at the unit level
  - Re-integration
  - Regression testing at the function level
  - Regression testing at the system level
- Definition
- Let P be a procedure or program, let P' be a modified version of P, and let T be a test suite for P.
- A typical regression test proceeds as follows:
  - Select *T' ≤ T, a set of test cases to execute on P'*.
  - Test *P'* with *T'*, establishing *P'*'s correctness with respect to *T'*
  - If necessary, create *T''*, a set of new functional or structural test cases for *P'*
  - Test *P' with T'' establishing P''s correctness with respect to T''* .
  - Create *T'''*, a new test suite and test execution profile for *P'*, from *T, T'*, and *T''*
- Step 1 is regression test selection problem
- Step 3 addresses the coverage identification problem
- Steps 2 and 4 address the test suite execution problem
- Step 5 addresses the test suite maintenance problem: the problem of updating and storing test information.

Module 103:

Regression Testing Requirement

- Changes are inevitable
    - We need to verify and validate if changes in one part of the system have not impacted non-changed parts of the system
- Who performs software Regression:
    - Developers - regression testing at the unit level or integration
    - Test engineers - regression testing at the function level
    - QA and test engineers - regression testing at the system level
- What artifacts are needed to perform software regression testing?
    - Software change information (change notes).
    - Updated software Requirements and Design specifications, and user manuals.
    - Software regression testing process and strategy.
    - Software regression testing methods and criteria.

Module 104:

How changes effect System under Test (SUT)

- Major challenge is to find out how changes effected SUT:
- Major problems in software regression testing:
- How to identify software changes in a systematic way?
    - Specification changes
    - Design specification changes
    - Implementation (or source code) changes
    - User manual changes
    - Environment or technology changes
    - Test changes

- How to identify software change impacts in a systematic way?
    - REQ impacts
    - Design impacts
    - Implementation impacts
    - User impacts
    - Test impacts
- How to select subset of test cases from existing test suite:
    - Re-testable
    - Reusable
    - Obsolete
- Adding new test cases

- Therefore, Major regression testing problems:
    - How to use a systematic method or tool to identify changed software parts?
    - How to use a systematic method or tool to identify software change impacts?
    - How to use a systematic method or tool to identify affected software test cases?

- Therefore, Major regression testing problems:
    - How to reduce the re-test suites?
    - How to select the test cases in a test suite?
    - Major challenge in software regression testing:
    - How to minimize re-testing efforts, and achieve the adequate testing coverage?

Module 105:

Effects of Requirement Change in Regression Testing

- Effects of changes in requirements and activities that may induce change:
    - Add new functional features
    - Change current function features
    - Delete existing function features
    - System Design
    - System architecture changes
- Effects of changes in requirements and activities that may induce change:
    - Change component interactions
    - Add new components or subsystems
    - Update existing components
    - Delete existing components

Module 106:

Effects of Maintenance in Regression Testing

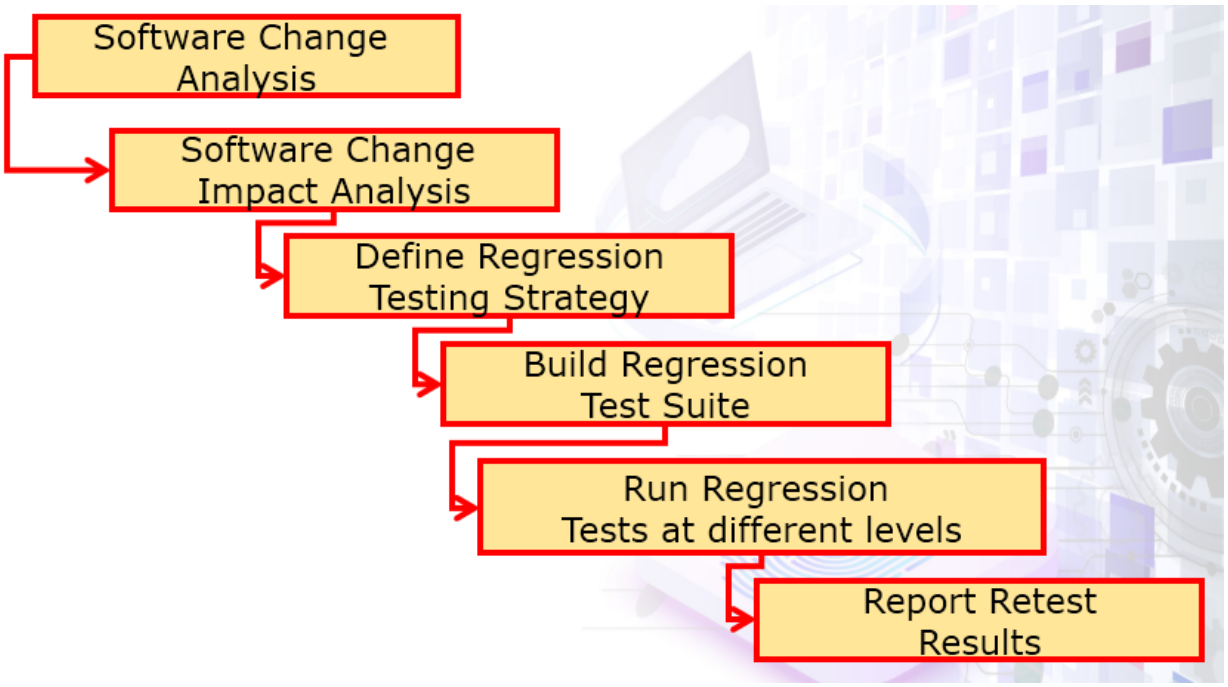
- Effects of maintenance:
    - change state-based behaviors
    - change component interfaces
    - change database design
    - change GUI design
    - change function design
    - low-level design doc. Changes
    - change algorithm logic
    - change component structure

- Component changes
- internal data types and names
- internal structures, such as
- class relationships
- control flow and data flow
- internal functions
- Component interface changes
- call signatures
- message interactions
- protocol messages and formats
- Technology and/or language changes

Module 107:

Impact of Bug Fixing in Regression Testing

- Developers do bug fixing and they may induce further bugs into the system
- Example
    - Developer X needed to make a change in the backend
    - He required to add a column in one of the tables he was working with
    - Development manager approved that change and forgot to disseminate information to the rest of the team
    - Developer Y had tested his code with respect to that particular table
    - While Developer X work is working, code of Developer Y has stopped working
        - Without his knowledge
        - With a sound reason e.g., a bug
        - Would again start working as soon as he handles effect of a change impacting his work
        - Without any side affect



- Therefore,
- Methods of regression testing work on the basis of:
  - Change identification
  - Test selection through:
    - Test case minimization
    - Test case prioritization
- These are our next two topics

Module 108:

Methods in Regression Testing

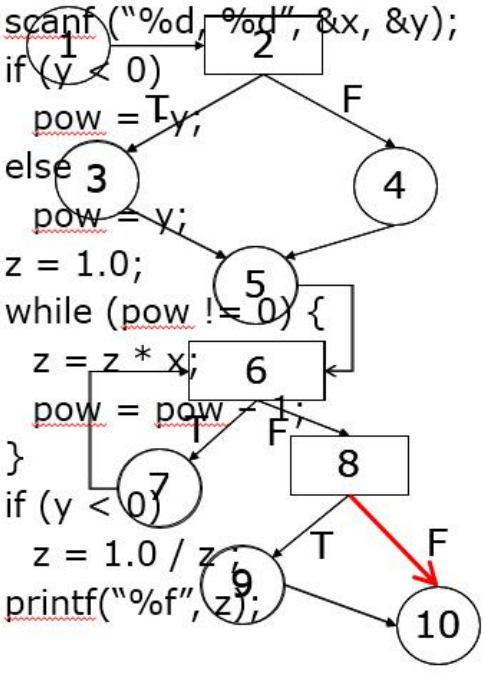
- We study a couple of methods for regression test selection:
- Test case selection for regression testing requires prior knowledge of application type
- Methods include:
    - Re-test all (one extreme)
    - Modification based selection methods
    - Coverage based selection methods
    - Random retesting (other extreme)
- Why application specific:
    - Imagine:
        - you are working on nuclear power plant
        - Medical equipment
        - Banking application
        - Attendance system
    - Your range of application is suggesting if regression testing be based on:
        - Retest all
        - Selective retesting based on a selection methods
- Extended finite state machine considering control and data dependence to construct a state dependence graph which is used to identify the effect of modification (Chen et al, 2006)
- Model-based regression testing using data dependence analysis where Extended finite state machine is used for dependence analysis in object-oriented systems for reducing regression test suites (Korel et al., 2002)
- UML designs are considered where pre- and post-conditions are given as OCL constraints and the changes in model are analyzed by analyzing their XML representation (Briand et al. 2009)
- Analyze visual contracts to identify impact of change and select test cases into subsets and perform coverage analysis to see if the resulting test suite retains coverage
- Analyze the information in terms of visual contracts where our focus is how the change affects the model. We also need to know which of the code artifacts were accessed during update. Our focus is different as we consider one web service and not compositions (Khan TA et al, 2012)

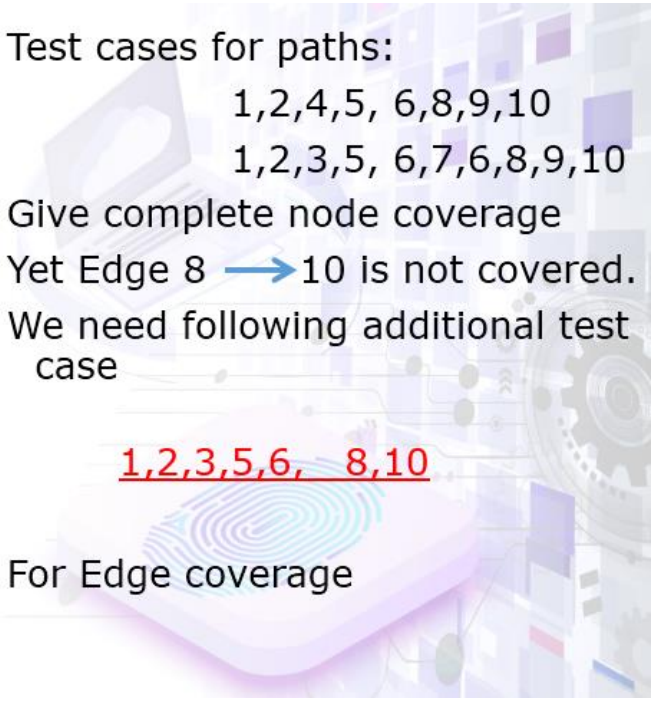
- Evaluate the change in model through visual contracts and do not consider control dependence
- Analysis is based on visual contracts analysis and not on extended finite state machines and the application domain is web services
- Consider visual contracts to represent pre- and post-conditions formalizing them as rules in a TAGTS for analysis and propose a regression testing approach for web services

Module 109:

Role of Test Case Prioritization for Regression Testing

- Test case prioritization helps us order previously run test cases.
- According to our requirement.
- Let our requirement considering unit test cases be:
  - We need to reorder test cases such that the one providing maximum coverage
  - With respect to nodes
  - be selected first
- Considering a previous example from Module 35





- Our test cases are:
  - 1,2,4,5, 6,8,9,10
  - 1,2,3,5, 6,7,6,8,9,10
  - 1,2,3,5,6, 8,10
- If we prioritize them according to number of nodes covered, they would become
  - 1,2,3,5, 6,7,6,8,9,10
  - 1,2,4,5, 6,8,9,10
  - 1,2,3,5,6, 8,10

- Other test case prioritization techniques:
  - Sort test cases in order of increasing cost per additional coverage
  - Select the first test case
  - Repeat the above two steps until n test cases are selected
- Decision regarding cost can be based on
  - Individual test case cost
  - Cumulative formula

Module 110:

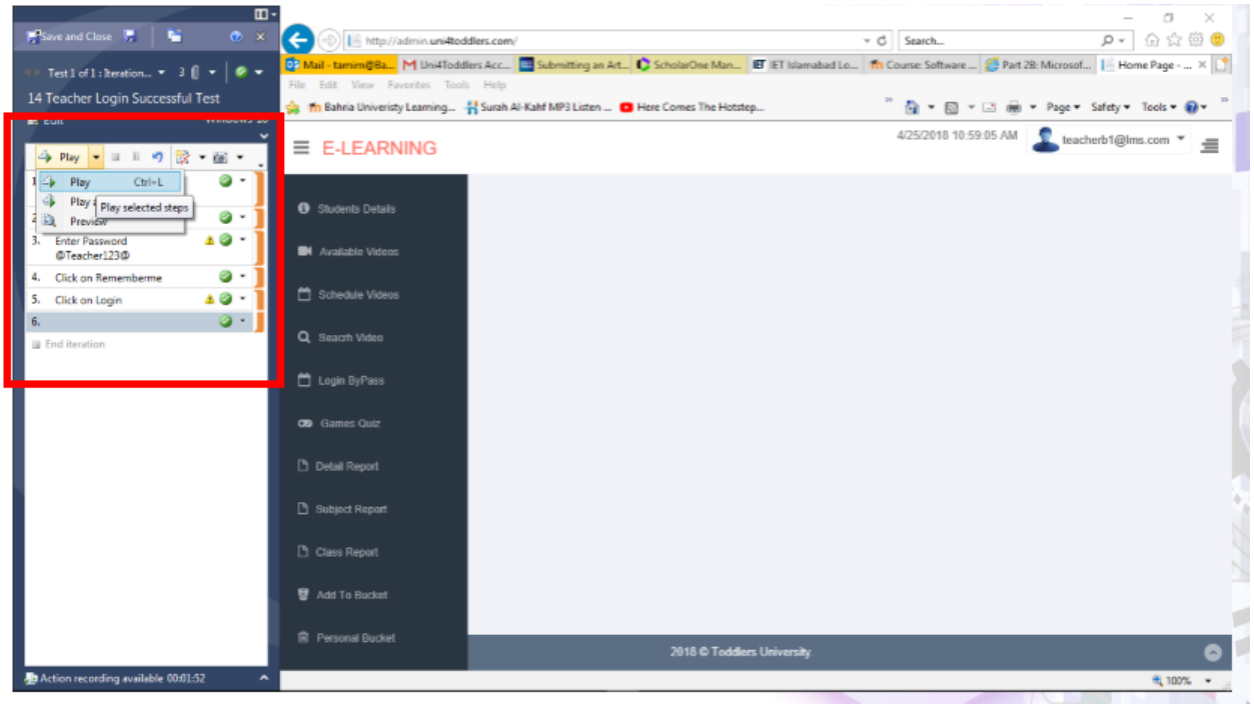
Test Case Recording and Regression Testing

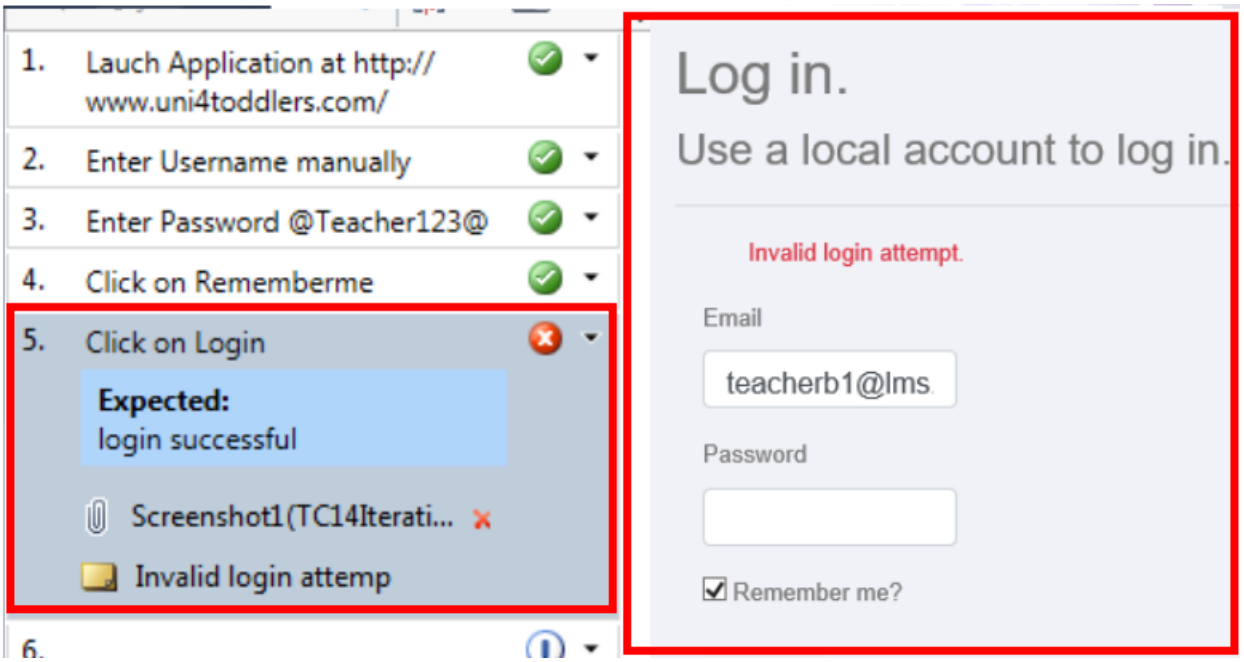
- We have seen so far:
    - Test suite minimization techniques
    - Test case Prioritization techniques
- Based on our knowledge about our test suite
- AND based on our information about the affected area or the target area of the software under test i.e., SUT
- We select test cases for regression testing
- Now we consider case where we do minimal effort on finding out
    - Affected area of SUT
    - Test case prioritization or minimization
- Record test case execution, *somehow*
    - Well!, why somehow
    - We cover in next few modules
- And rerun them whenever required

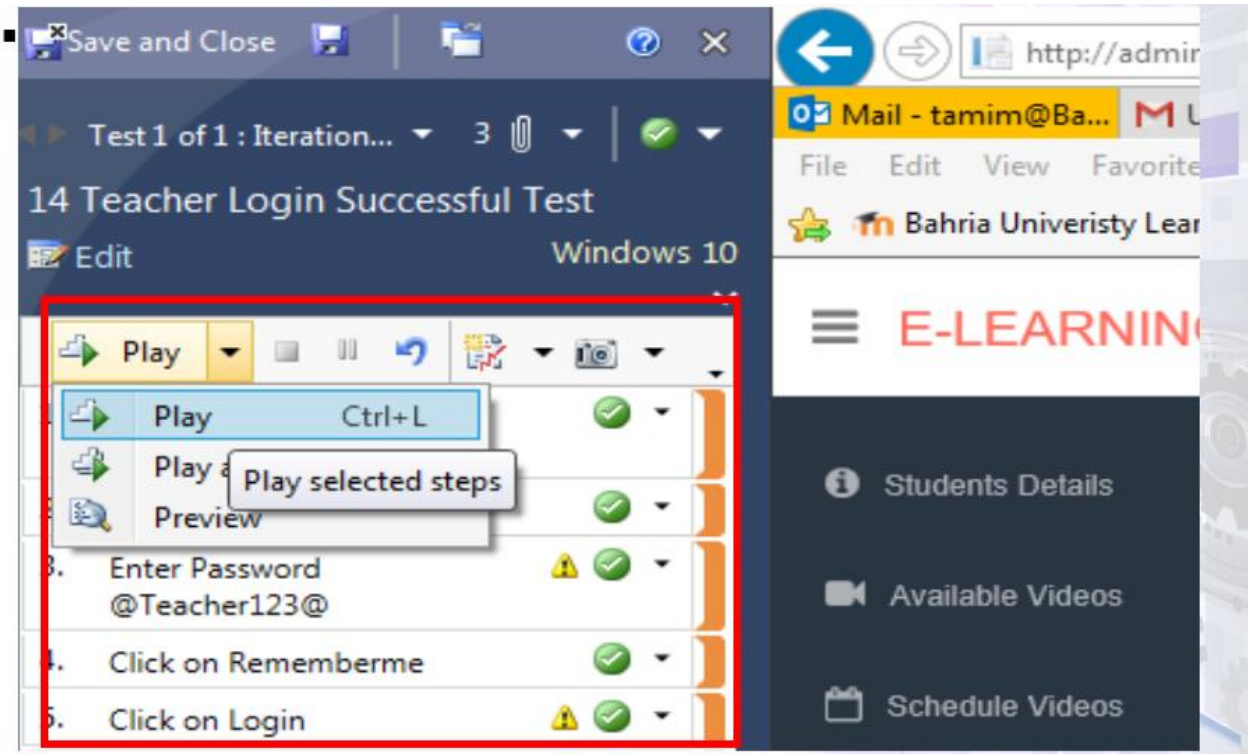
Module 111:

Test Case Recording with Microsoft Test Manager (MTM)

- We have already covered test case recording with MTM.
- We have seen in Module 97 how to record and rerun a test case.
- As a reminder, we revise a couple of slides from Module 97.


Save and Close
Test 1 of 1 : Iteration...
14 Teacher Login Successful Test
Play
Play Ctrl+L
Play selected steps
Preview
3. Enter Password @Teacher123@
4. Click on Rememberme
5. Click on Login
6.
End iteration
Action recording available 00:01:52
E-LEARNING
4/25/2018 10:59:05 AM
teacherb1@lms.com
Students Details
Available Videos
Schedule Videos
Search Video
Login ByPass
Games Quiz
Detail Report
Subject Report
Class Report
Add To Bucket
Personal Bucket
2018 © Toddlers University
100%


1. Lauch Application at http://www.uni4toddlers.com/
2. Enter Username manually
3. Enter Password @Teacher123@
4. Click on Rememberme
5. Click on Login
Expected:
login successful
Screenshot1(TC14Iterati...
Invalid login attemp
6.
Log in.
Use a local account to log in.
Invalid login attempt.
Email
teacherb1@lms
Password
Remember me?

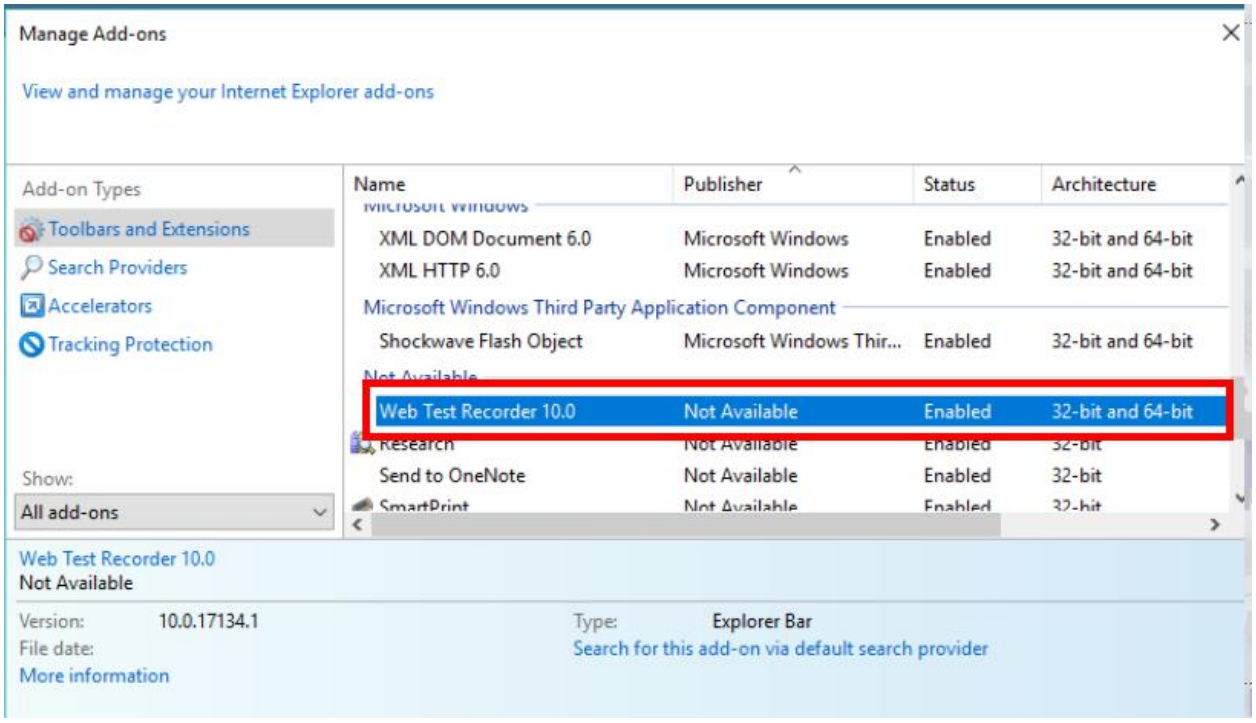
- 

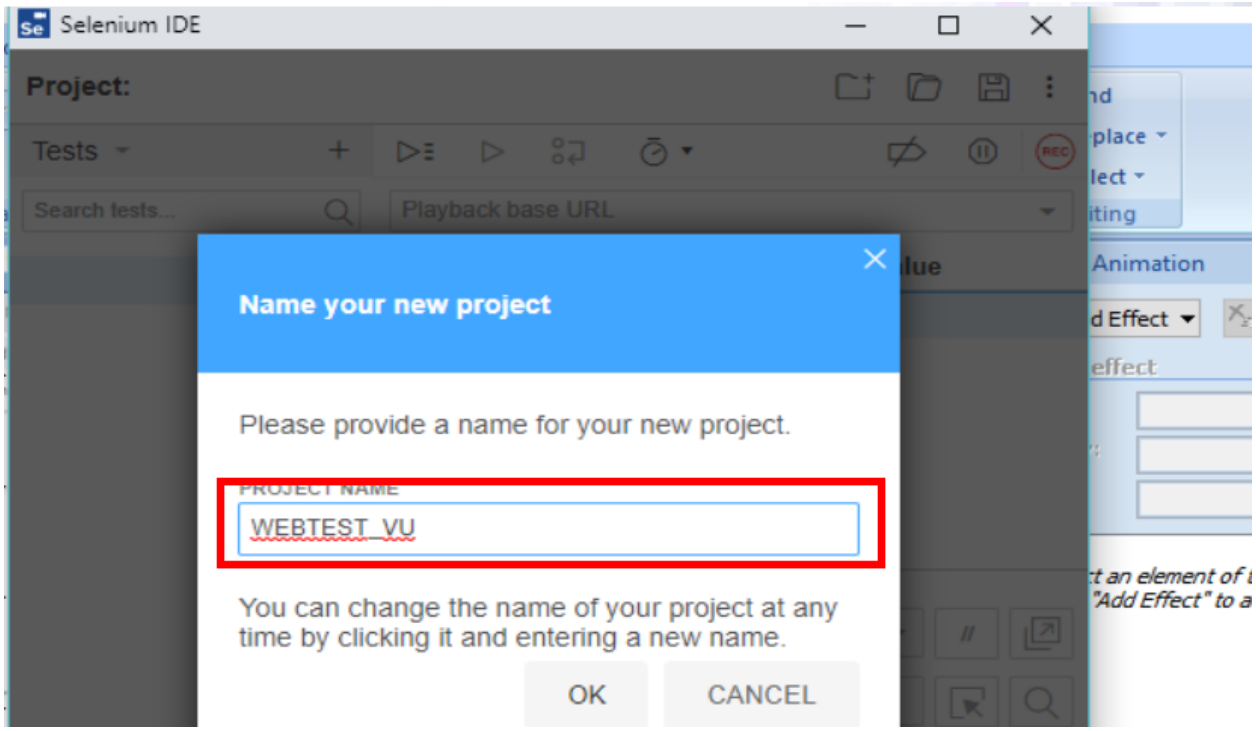
- Once recorded, these test cases are useful for Regression Testing.
- However, we still need to reduce the number of test cases so that:
  - We only rerun required test cases.
  - Rerunning a test case is economically a burden with respect to time, human resource, cost etc.

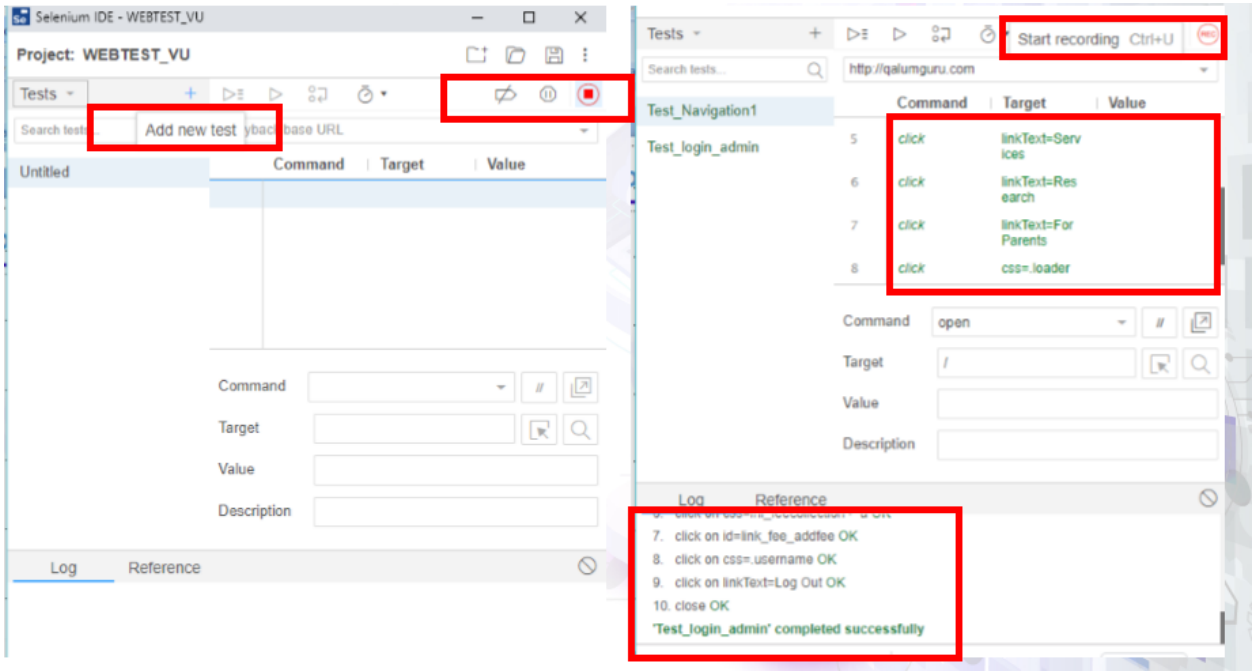
Module 112:

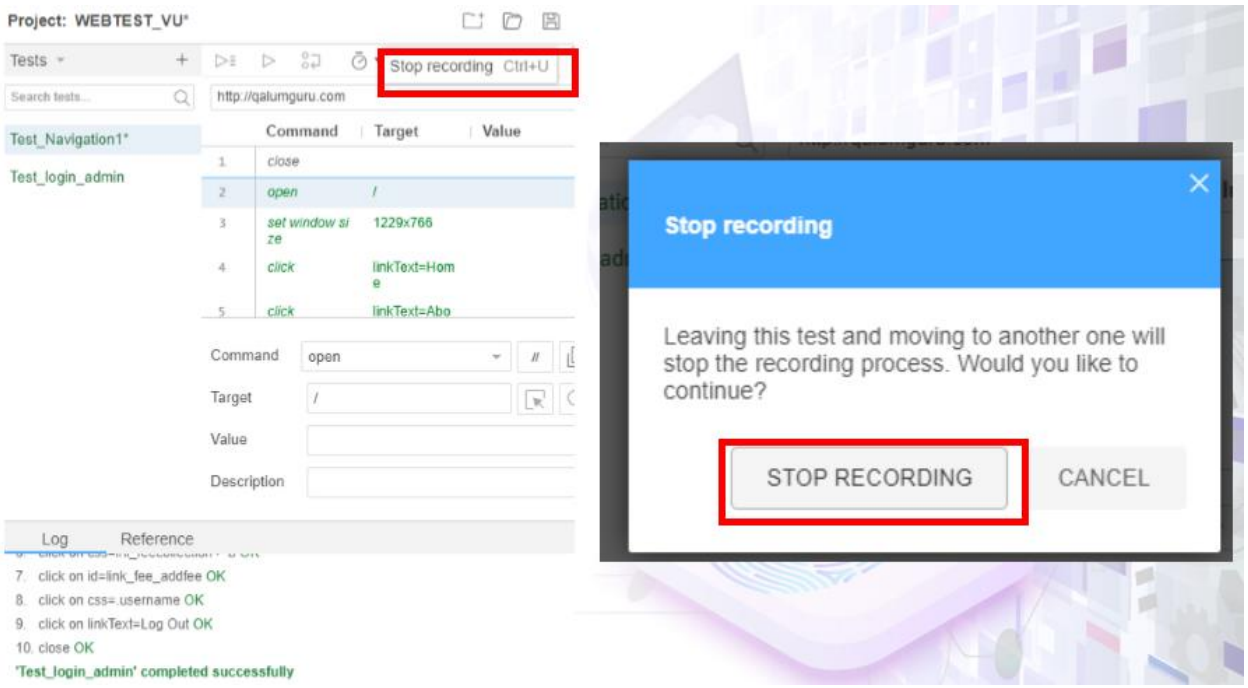
Test Case Recording with Selenium

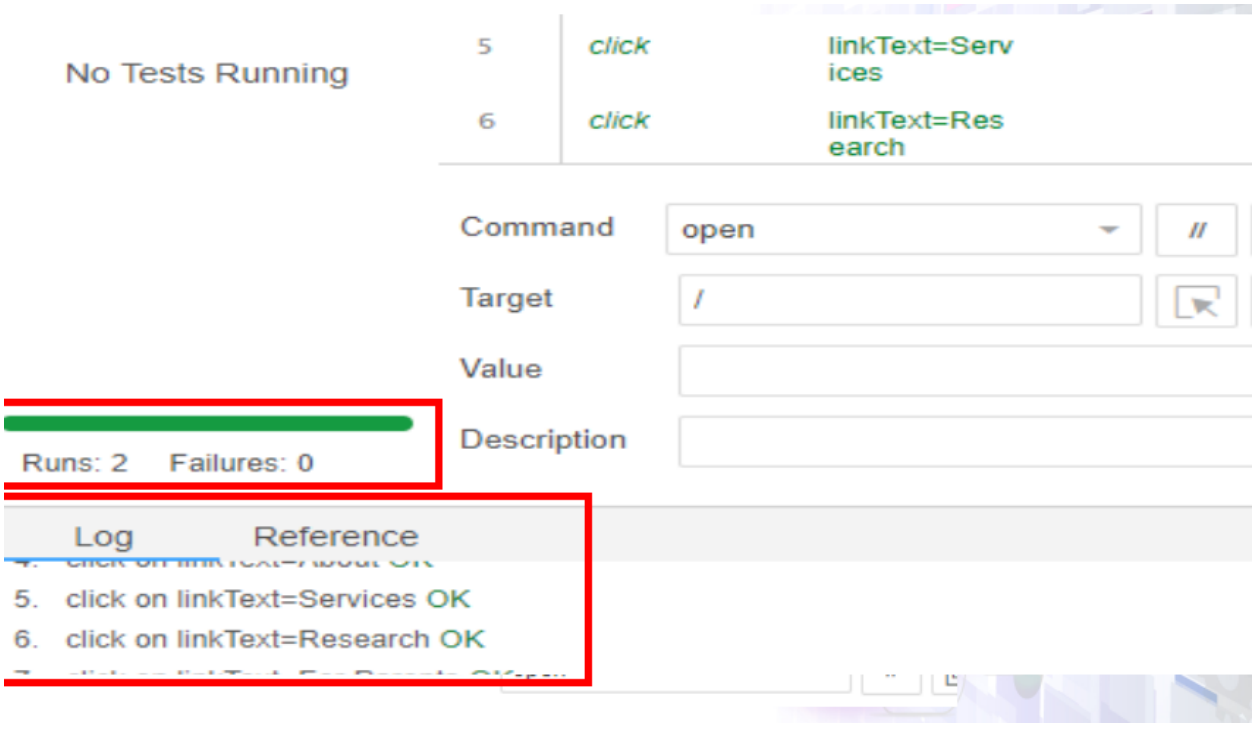
- Selenium is a set of tools that supports rapid development of test automation for web-based applications.
- Works anywhere JavaScript is supported
- Supports languages e.g., Java, Ruby, Python
- Can simulate a user navigating through pages and then assert for specific marks on the pages




Selenium IDE
Project:
Tests
Search tests...
Playback base URL
Name your new project
Please provide a name for your new project.
PROJECT NAME
WEBTEST_VU
You can change the name of your project at any time by clicking it and entering a new name.
OK
CANCEL


Selenium IDE - WEBTEST_VU
Project: WEBTEST_VU
Tests
Add new test
Untitled
Command
Target
Value
Description
Log
Reference
Start recording Ctrl+U
http://qalumguru.com
Test_Navigation1
Test_login_admin
click
linkText=Services
linkText=Research
linkText=For Parents
css=.loader
open
7. click on id=link_fee_addfee OK
8. click on css=.username OK
9. click on linkText=Log Out OK
10. close OK
'Test_login_admin' completed successfully


Project: WEBTEST_VU*
Tests
Stop recording Ctrl+U
Search tests...
http://qalumguru.com
Test_Navigation1*
Test_login_admin
Command
Target
Value
1 close
2 open /
3 set window size 1229x766
4 click linkText=Home
5 click linkText=Abo
Command open
Target /
Value
Description
Log
Reference
7. click on id=link_fee_addfee OK
8. click on css=.username OK
9. click on linkText=Log Out OK
10. close OK
'Test_login_admin' completed successfully
Stop recording
Leaving this test and moving to another one will stop the recording process. Would you like to continue?
STOP RECORDING
CANCEL

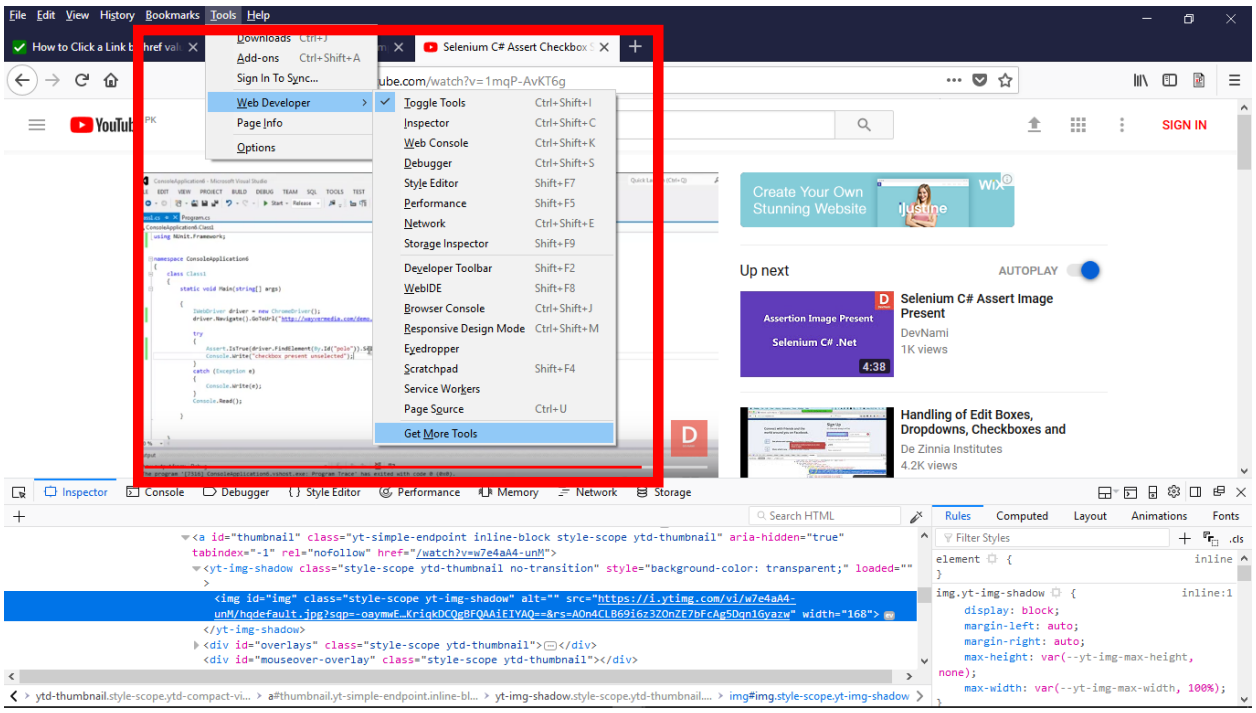

No Tests Running
5 click linkText=Services
6 click linkText=Research
Command open
Target /
Value
Description
Runs: 2 Failures: 0
Log
Reference
5. click on linkText=Services OK
6. click on linkText=Research OK

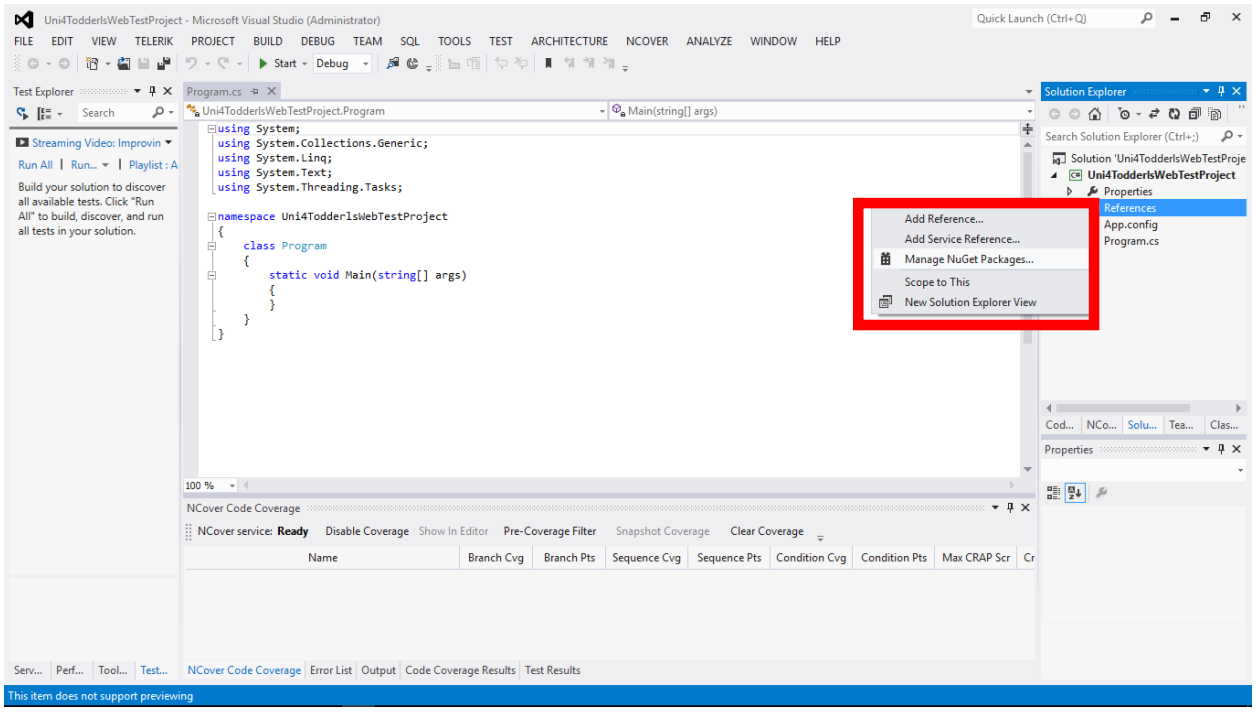
- Now these test cases are available as test suite
- They can be run on a click
- We can rerun them anytime and see if there is any regression in the overall system quality.

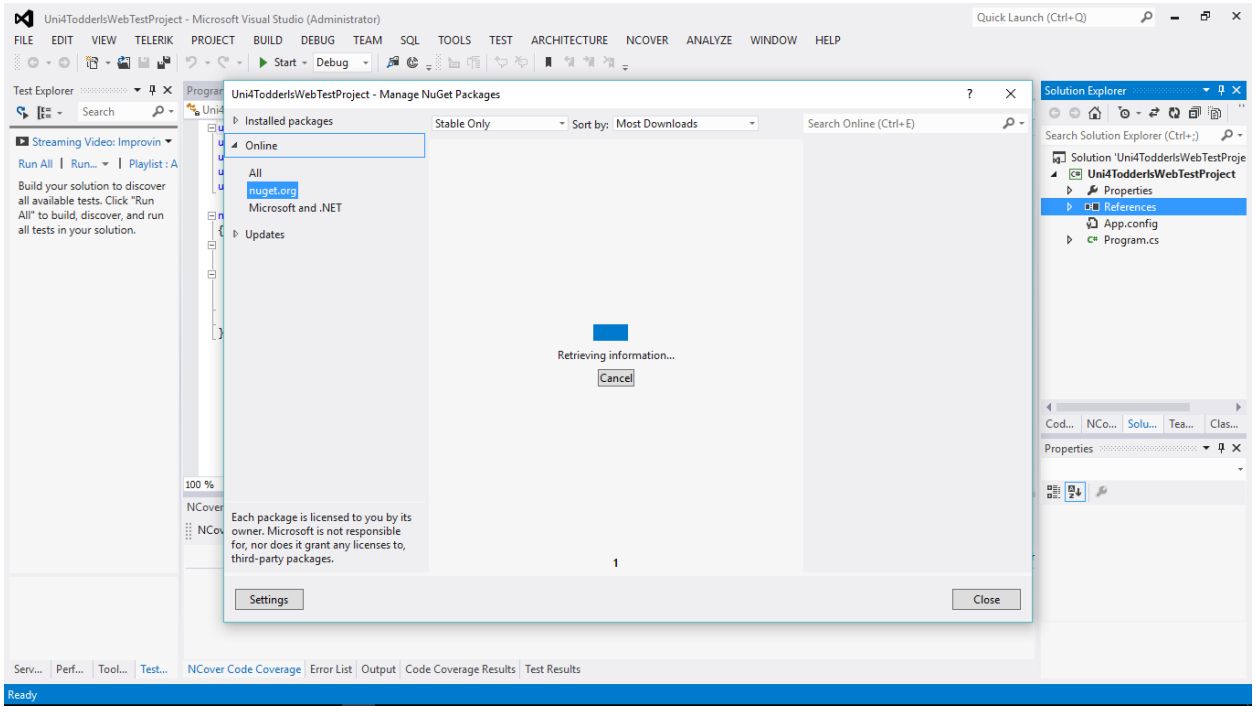
Module 113:

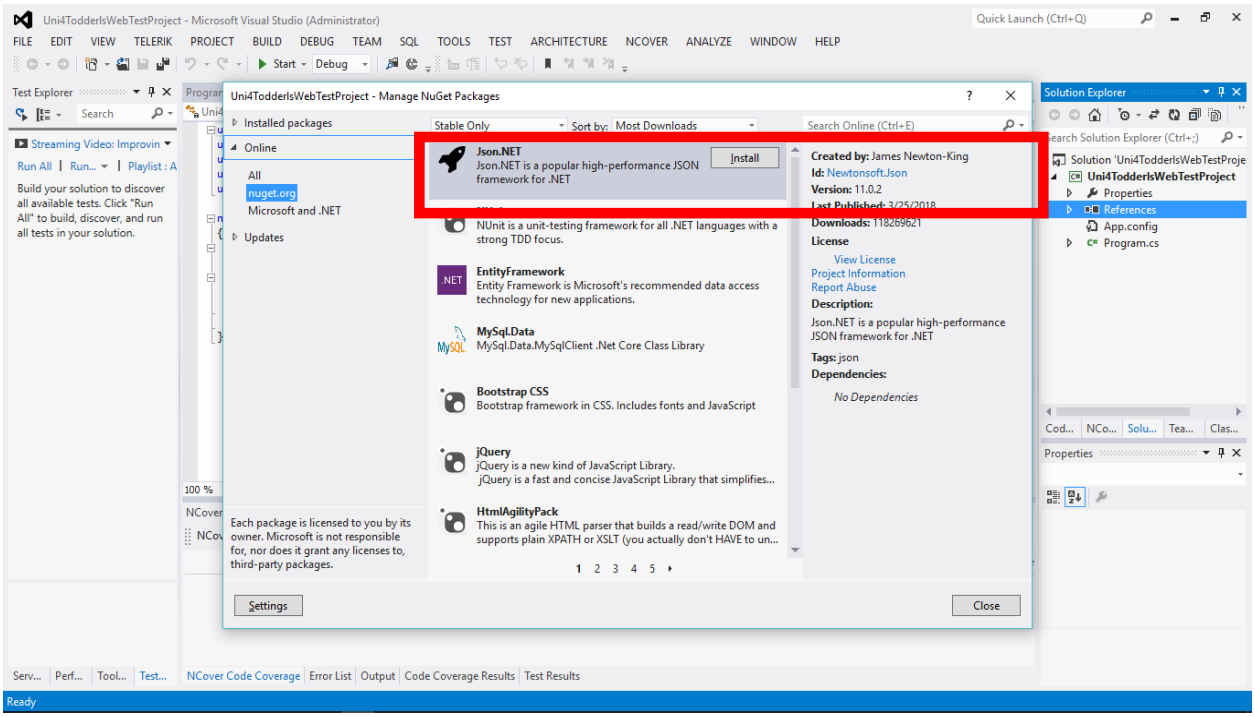
Selenium as an Addon for IDE

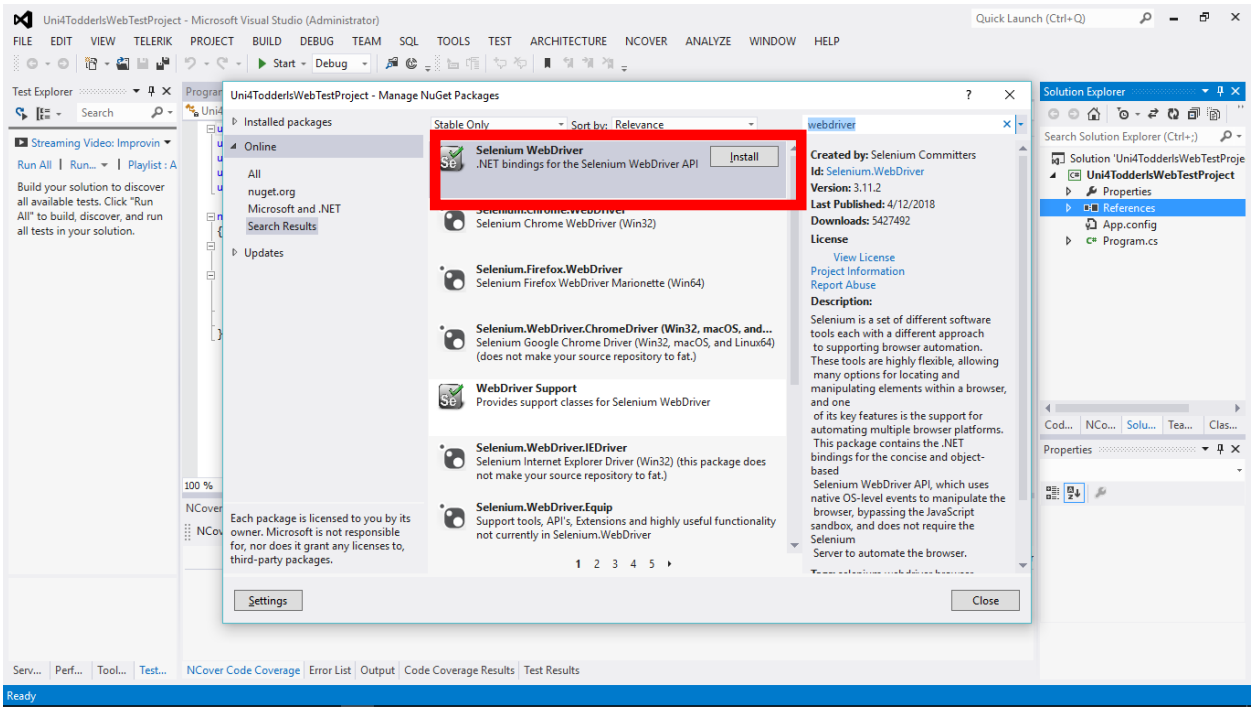
- We can run record test cases using IDE e.g., Visual Studio and develop a test suite
- We can run these test cases using Selenium driver from inside a test case written from IDE.
- We take example of a test case written in C#

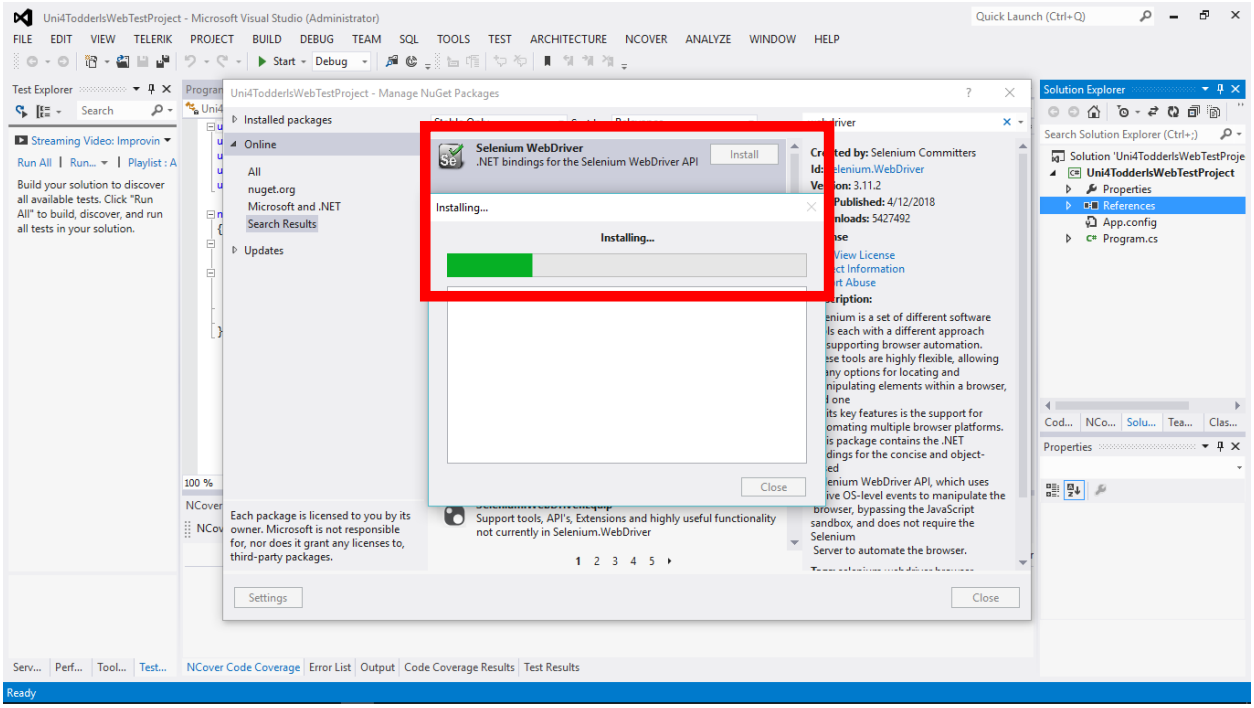


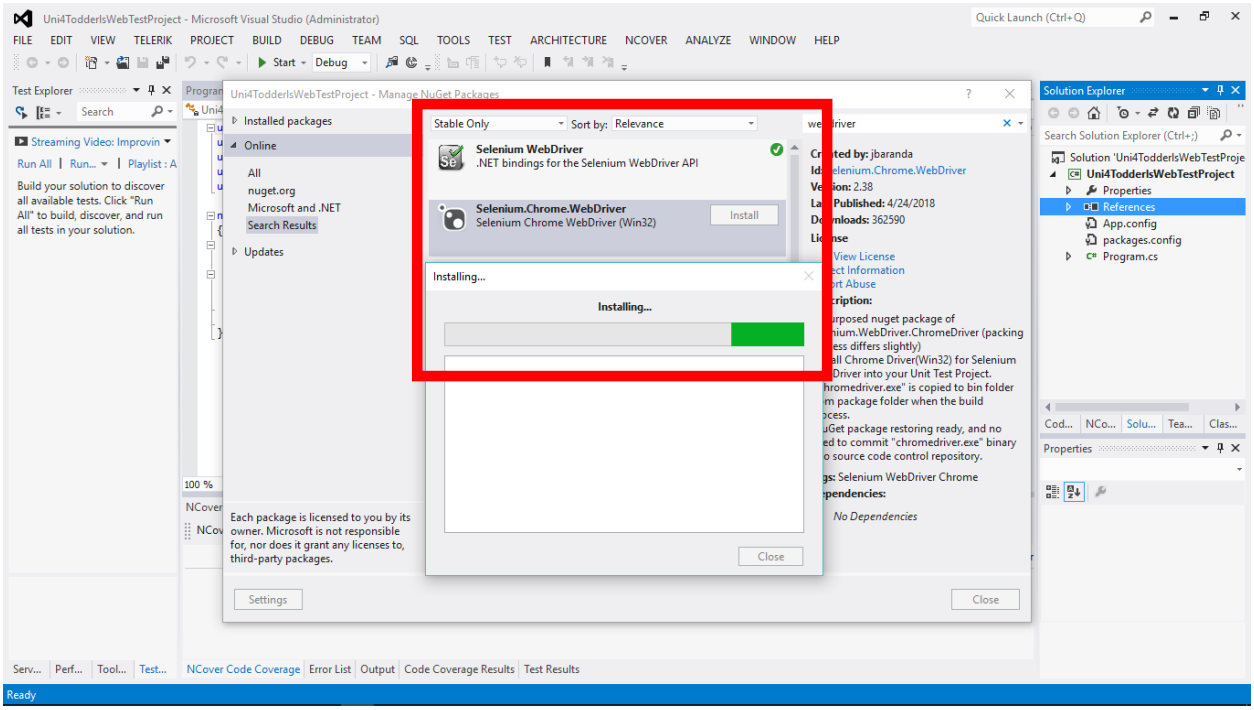

Uni4TodderIsWebTestProject - Microsoft Visual Studio (Administrator)
Quick Launch (Ctrl+Q)
FILE EDIT VIEW TELERIK PROJECT BUILD DEBUG TEAM SQL TOOLS TEST ARCHITECTURE NCOVER ANALYZE WINDOW HELP
Start Debug
Test Explorer
Search
Streaming Video: Improvin
Run All | Run... | Playlist : A
Build your solution to discover all available tests. Click "Run All" to build, discover, and run all tests in your solution.
Program.cs
Uni4TodderIsWebTestProject.Program
Main(string[] args)
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Uni4TodderIsWebTestProject
{
class Program
{
static void Main(string[] args)
{
}
}
}
Add Reference...
Add Service Reference...
Manage NuGet Packages...
Scope to This
New Solution Explorer View
Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'Uni4TodderIsWebTestProje
Uni4TodderIsWebTestProject
Properties
References
App.config
Program.cs
100 %
NCover Code Coverage
NCover service: Ready
Disable Coverage
Show In Editor
Pre-Coverage Filter
Snapshot Coverage
Clear Coverage
Name
Branch Cvg
Branch Pts
Sequence Cvg
Sequence Pts
Condition Cvg
Condition Pts
Max CRAP Scr
Properties
Serv... Perf... Tool... Test...
NCover Code Coverage
Error List
Output
Code Coverage Results
Test Results
This item does not support previewing

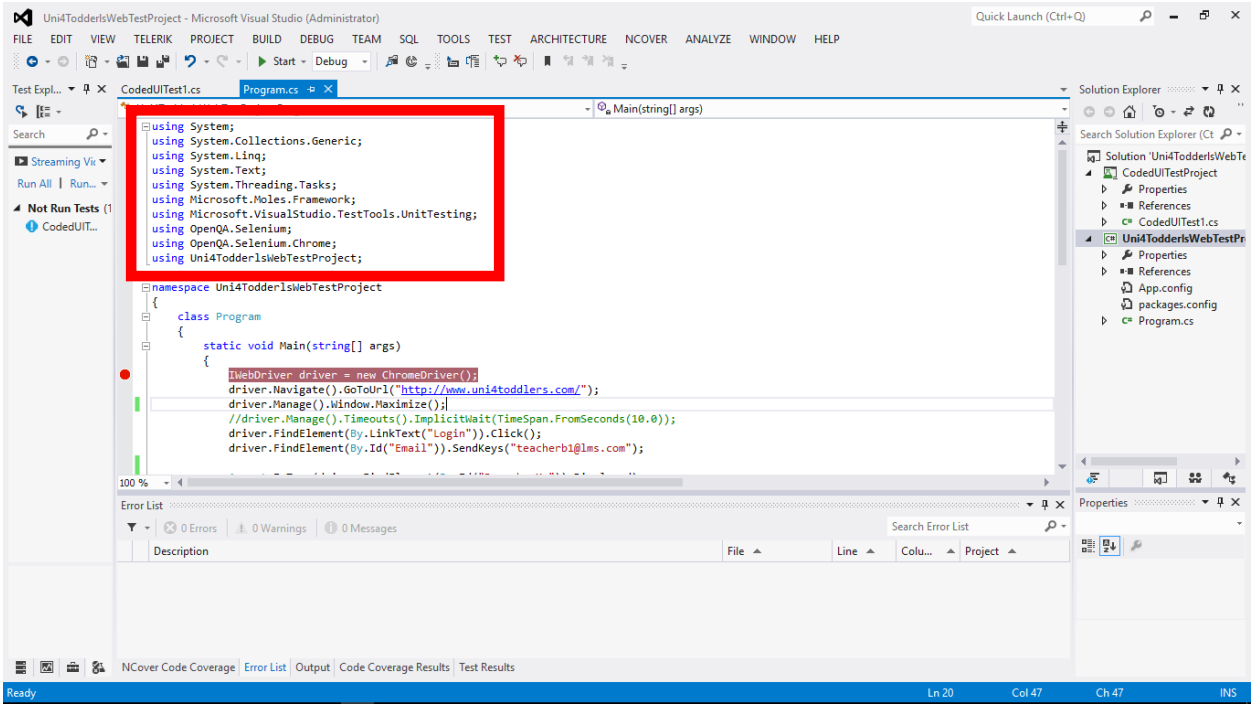

Uni4TodderIsWebTestProject - Microsoft Visual Studio (Administrator)
Quick Launch (Ctrl+Q)
FILE EDIT VIEW TELERIK PROJECT BUILD DEBUG TEAM SQL TOOLS TEST ARCHITECTURE NCOVER ANALYZE WINDOW HELP
Start Debug
Test Explorer
Search
Streaming Video: Improvin
Run All | Run... | Playlist : A
Build your solution to discover all available tests. Click "Run All" to build, discover, and run all tests in your solution.
Uni4TodderIsWebTestProject - Manage NuGet Packages
Installed packages
Online
All
nuget.org
Microsoft and .NET
Updates
Stable Only
Sort by: Most Downloads
Search Online (Ctrl+E)
Retrieving information...
Cancel
Each package is licensed to you by its owner. Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
1
Settings
Close
Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'Uni4TodderIsWebTestProje
Uni4TodderIsWebTestProject
Properties
References
App.config
Program.cs
100 %
Properties
Serv... Perf... Tool... Test...
NCover Code Coverage
Error List
Output
Code Coverage Results
Test Results
Ready

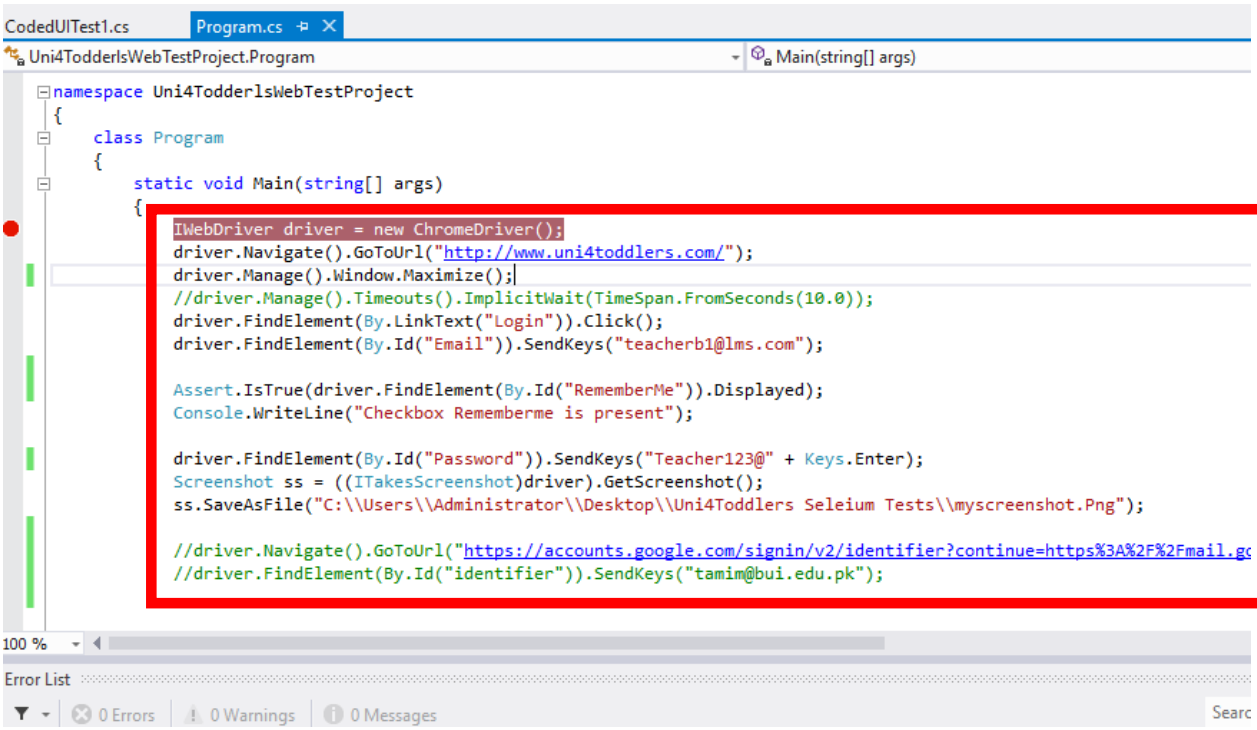

Uni4ToddlerIsWebTestProject - Microsoft Visual Studio (Administrator)
Uni4ToddlerIsWebTestProject - Manage NuGet Packages
Installed packages
Online
All
nuget.org
Microsoft and .NET
Updates
Stable Only
Sort by: Most Downloads
Search Online (Ctrl+E)
Json.NET
Json.NET is a popular high-performance JSON framework for .NET
Install
NUnit is a unit-testing framework for all .NET languages with a strong TDD focus.
EntityFramework
Entity Framework is Microsoft's recommended data access technology for new applications.
MySql.Data
MySql.Data.MySqlClient .Net Core Class Library
Bootstrap CSS
Bootstrap framework in CSS. Includes fonts and JavaScript
jQuery
jQuery is a new kind of JavaScript Library.
jQuery is a fast and concise JavaScript Library that simplifies...
HtmlAgilityPack
This is an agile HTML parser that builds a read/write DOM and supports plain XPATH or XSLT (you actually don't HAVE to un...
1 2 3 4 5
Each package is licensed to you by its owner. Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Settings
Close
Created by: James Newton-King
Id: Newtonsoft.Json
Version: 11.0.2
Last Published: 3/25/2018
Downloads: 118269621
License
View License
Project Information
Report Abuse
Description:
Json.NET is a popular high-performance JSON framework for .NET
Tags: json
Dependencies:
No Dependencies
Solution Explorer
Solution 'Uni4ToddlerIsWebTestProje
Uni4ToddlerIsWebTestProject
Properties
References
App.config
Program.cs
Test Explorer
Streaming Video: Improvin
Run All | Run... | Playlist : A
Build your solution to discover all available tests. Click "Run All" to build, discover, and run all tests in your solution.
NCover Code Coverage
Error List
Output
Code Coverage Results
Test Results
Ready

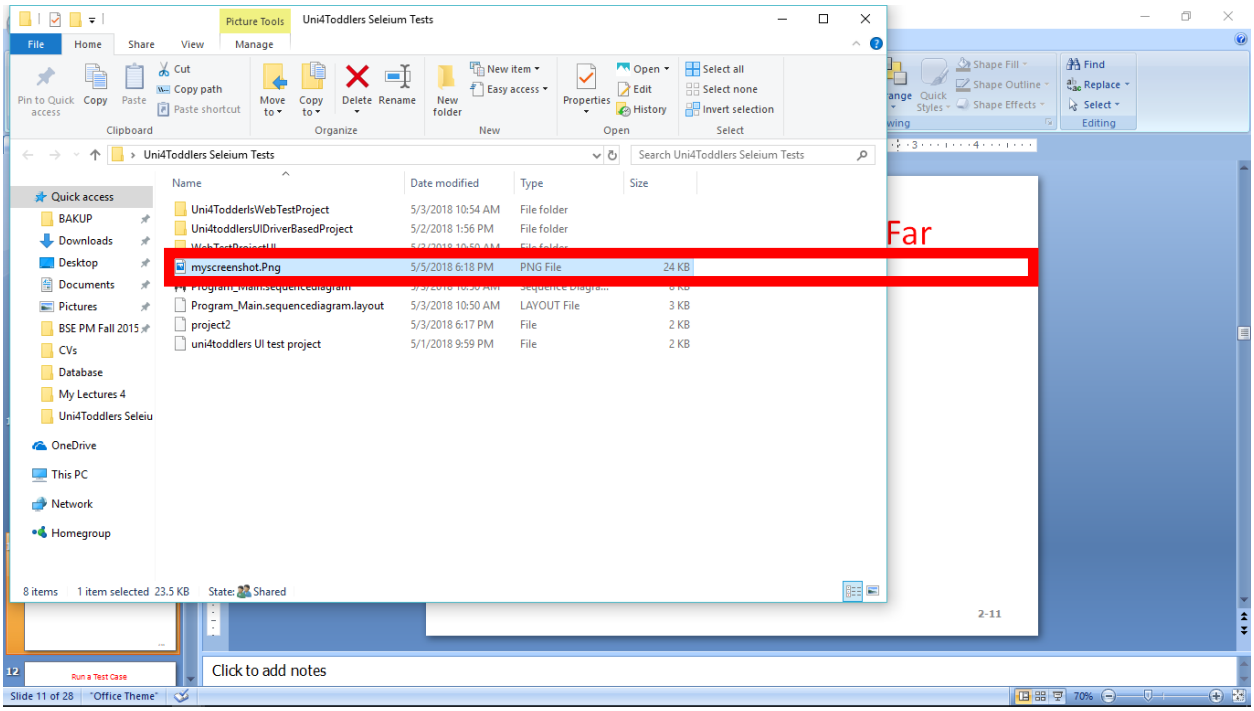

Uni4ToddlerIsWebTestProject - Microsoft Visual Studio (Administrator)
Uni4ToddlerIsWebTestProject - Manage NuGet Packages
Installed packages
Online
All
nuget.org
Microsoft and .NET
Search Results
Updates
Stable Only
Sort by: Relevance
webdriver
Selenium WebDriver
.NET bindings for the Selenium WebDriver API
Install
Selenium Chrome WebDriver (Win32)
Selenium.Firefox.WebDriver
Selenium Firefox WebDriver Marionette (Win64)
Selenium.WebDriver.ChromeDriver (Win32, macOS, and...
Selenium Google Chrome Driver (Win32, macOS, and Linux64) (does not make your source repository to fat.)
WebDriver Support
Provides support classes for Selenium WebDriver
Selenium.WebDriver.IEDriver
Selenium Internet Explorer Driver (Win32) (this package does not make your source repository to fat.)
Selenium.WebDriver.Equip
Support tools, API's, Extensions and highly useful functionality not currently in Selenium.WebDriver
1 2 3 4 5
Each package is licensed to you by its owner. Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Settings
Close
Created by: Selenium Committers
Id: Selenium.WebDriver
Version: 3.11.2
Last Published: 4/12/2018
Downloads: 5427492
License
View License
Project Information
Report Abuse
Description:
Selenium is a set of different software tools each with a different approach to supporting browser automation. These tools are highly flexible, allowing many options for locating and manipulating elements within a browser, and one of its key features is the support for automating multiple browser platforms. This package contains the .NET bindings for the concise and object-based Selenium WebDriver API, which uses native OS-level events to manipulate the browser, bypassing the JavaScript sandbox, and does not require the Selenium Server to automate the browser.
Solution Explorer
Solution 'Uni4ToddlerIsWebTestProje
Uni4ToddlerIsWebTestProject
Properties
References
App.config
Program.cs
Test Explorer
Streaming Video: Improvin
Run All | Run... | Playlist : A
Build your solution to discover all available tests. Click "Run All" to build, discover, and run all tests in your solution.
NCover Code Coverage
Error List
Output
Code Coverage Results
Test Results
Ready


Uni4ToddlerIsWebTestProject - Microsoft Visual Studio (Administrator)
Uni4ToddlerIsWebTestProject - Manage NuGet Packages
Installed packages
Online
All
nuget.org
Microsoft and .NET
Search Results
Updates
Selenium WebDriver
.NET bindings for the Selenium WebDriver API
Install
Installing...
Installing...
Close
Each package is licensed to you by its owner. Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Support tools, API's, Extensions and highly useful functionality not currently in Selenium.WebDriver
Server to automate the browser.
Settings
Close
Solution Explorer
Uni4ToddlerIsWebTestProject
References
App.config
Program.cs
Properties
Ready


Uni4ToddlerIsWebTestProject - Microsoft Visual Studio (Administrator)
Uni4ToddlerIsWebTestProject - Manage NuGet Packages
Installed packages
Online
All
nuget.org
Microsoft and .NET
Search Results
Updates
Stable Only
Sort by: Relevance
Selenium WebDriver
.NET bindings for the Selenium WebDriver API
Selenium.Chrome.WebDriver
Selenium Chrome WebDriver (Win32)
Install
Installing...
Installing...
Close
Each package is licensed to you by its owner. Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
No Dependencies
Settings
Close
Solution Explorer
Uni4ToddlerIsWebTestProject
References
App.config
packages.config
Program.cs
Properties
Ready


Uni4TodderlsWebTestProject - Microsoft Visual Studio (Administrator)
CodedUITest1.cs
Program.cs
Main(string[] args)
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Microsoft.Moles.Framework;
using Microsoft.VisualStudio.TestTools.UnitTesting;
using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;
using Uni4TodderlsWebTestProject;
namespace Uni4TodderlsWebTestProject
{
class Program
{
static void Main(string[] args)
{
IWebDriver driver = new ChromeDriver();
driver.Navigate().GoToUrl("http://www.uni4toddlers.com/");
driver.Manage().Window.Maximize();
//driver.Manage().Timeouts().ImplicitWait(TimeSpan.FromSeconds(10.0));
driver.FindElement(By.LinkText("Login")).Click();
driver.FindElement(By.Id("Email")).SendKeys("teacherb1@lms.com");
Solution Explorer
Error List
Properties
Ready


CodedUITest1.cs
Program.cs
Uni4TodderlsWebTestProject.Program
Main(string[] args)
namespace Uni4TodderlsWebTestProject
{
class Program
{
static void Main(string[] args)
{
IWebDriver driver = new ChromeDriver();
driver.Navigate().GoToUrl("http://www.uni4toddlers.com/");
driver.Manage().Window.Maximize();
//driver.Manage().Timeouts().ImplicitWait(TimeSpan.FromSeconds(10.0));
driver.FindElement(By.LinkText("Login")).Click();
driver.FindElement(By.Id("Email")).SendKeys("teacherb1@lms.com");
Assert.IsTrue(driver.FindElement(By.Id("RememberMe")).Displayed);
Console.WriteLine("Checkbox Rememberme is present");
driver.FindElement(By.Id("Password")).SendKeys("Teacher123@" + Keys.Enter);
Screenshot ss = ((ITakesScreenshot)driver).GetScreenshot();
ss.SaveAsFile("C:\\Users\\Administrator\\Desktop\\Uni4Toddlers Seleium Tests\\myscreenshot.Png");
//driver.Navigate().GoToUrl("https://accounts.google.com/signin/v2/identifier?continue=https%3A%2F%2Fmail.g
//driver.FindElement(By.Id("identifier")).SendKeys("tamim@bui.edu.pk");
100 %
Error List
0 Errors
0 Warnings
0 Messages



- Now these test cases are available as test suite
- They can be run on a click
- We can rerun them anytime and see if there is any regression in the overall system quality.

Module 114:

Web Testing

- **Testing** is the process of exercising software with the intent of finding of errors

  Whereas

- **Web testing** is the testing that focuses on testing of web applications

- Web Testing Checklist:
- **Functional Testing Part**
  - In Functional testing we need check the each components are functioning as expected or not, so it is also called as “Component Testing”.
- **Usability Testing Part**

In Usability testing we ensure that all possible test cases which targeted audience of the web application are doing regularly.

- **Compatibility testing**
  - It is ensuring that how application’s working in the supported environments.
- **Database Testing**
  - Web Application is thoroughly tested to ensure data reliability.

- **Crowd Testing part**
  - Crowd testing is when a large group of perfect strangers try your product then give you phenomenally helpful feedback on usability, bugs and features.
- **Interface Testing part**
  - In the Interface testing mainly three areas should be covered: Web Server, Application Server and Database Server.
- **Performance Testing**
  - Performance testing is to check the web application is working under the heavy load.

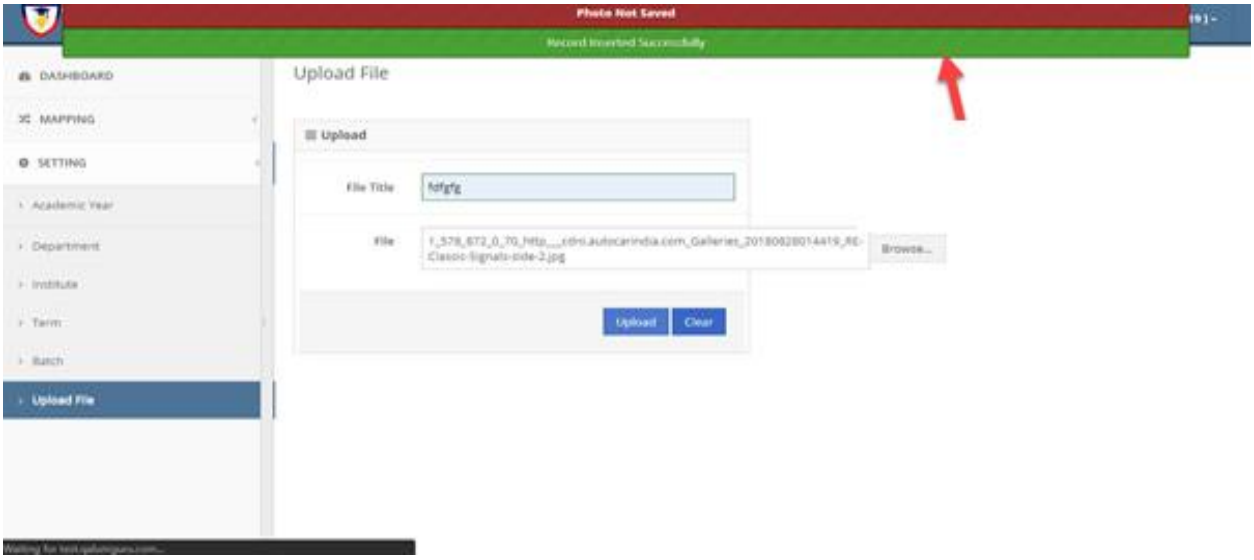
- **Security Testing**
    - The security testing is carried out to ensure that is any information leakage in terms of encrypting the data.

- **Web Testing Scenarios:**
    - User navigation to different pages of the website and completing the end-to-end workflow
    - If the user can select/deselect checkboxes
    - If the user can select values from Dropdown fields
    - If the user can select/deselect Radio buttons
    - Different navigation buttons like Submit, Next, Upload etc. buttons are working well
    - Calendars are loading properly and allowing the user to select a date
    - Calculations are happening as implemented
    - Search functionality is working if any

- Example
    - We are testing our Qalum guru example



Test case	Verify File upload with valid Data
Test case steps	1- Enter URL http://qalumguru.com/index.html. 2. Log in as a student. 3. Click 'Setting' tab 4. Click Upload File button. 5. Enter File name. 6. Select File. 7. Click upload button. 8. Verify if video is uploaded.
Expected Output	Video should be uploaded
Actual Output	Video is not uploaded
Result	Fail

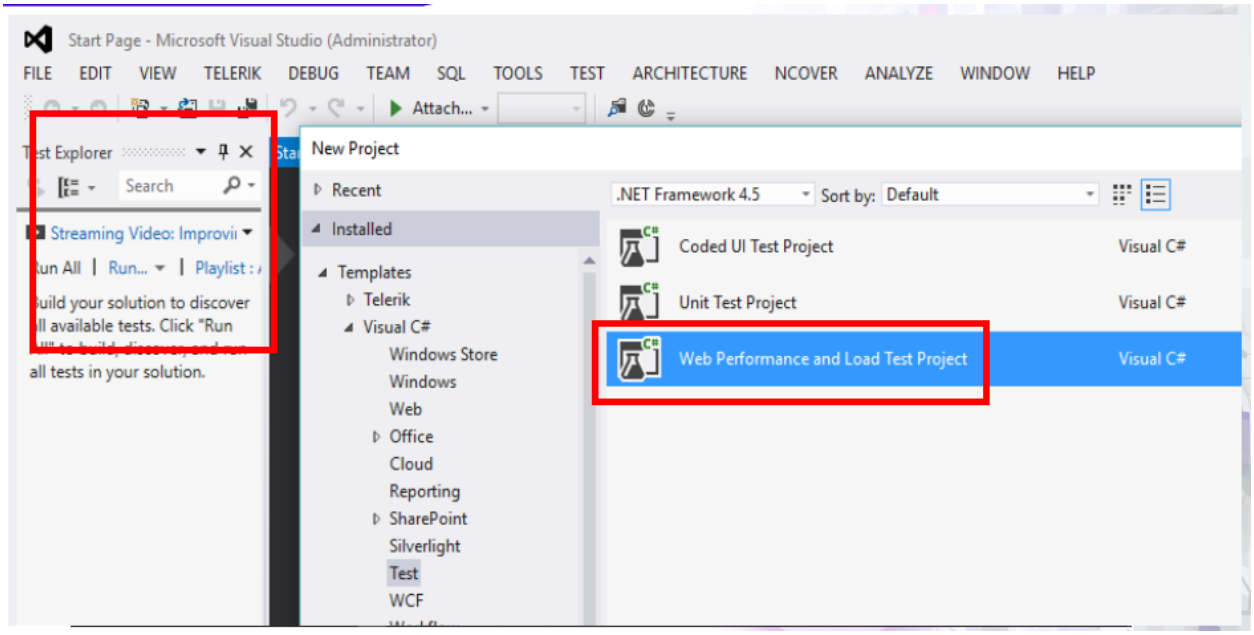


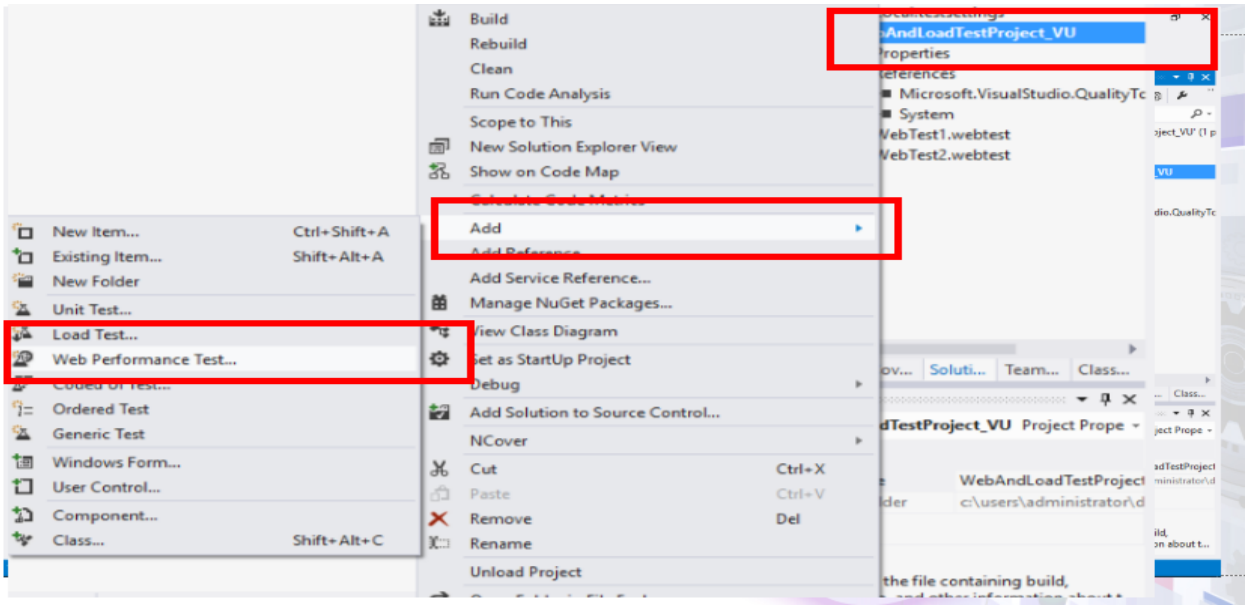
- We now study a couple of web testing tools
- We cover:
  - Web Testing
  - Regression Testing
- We try to cover both aspects in a single go.

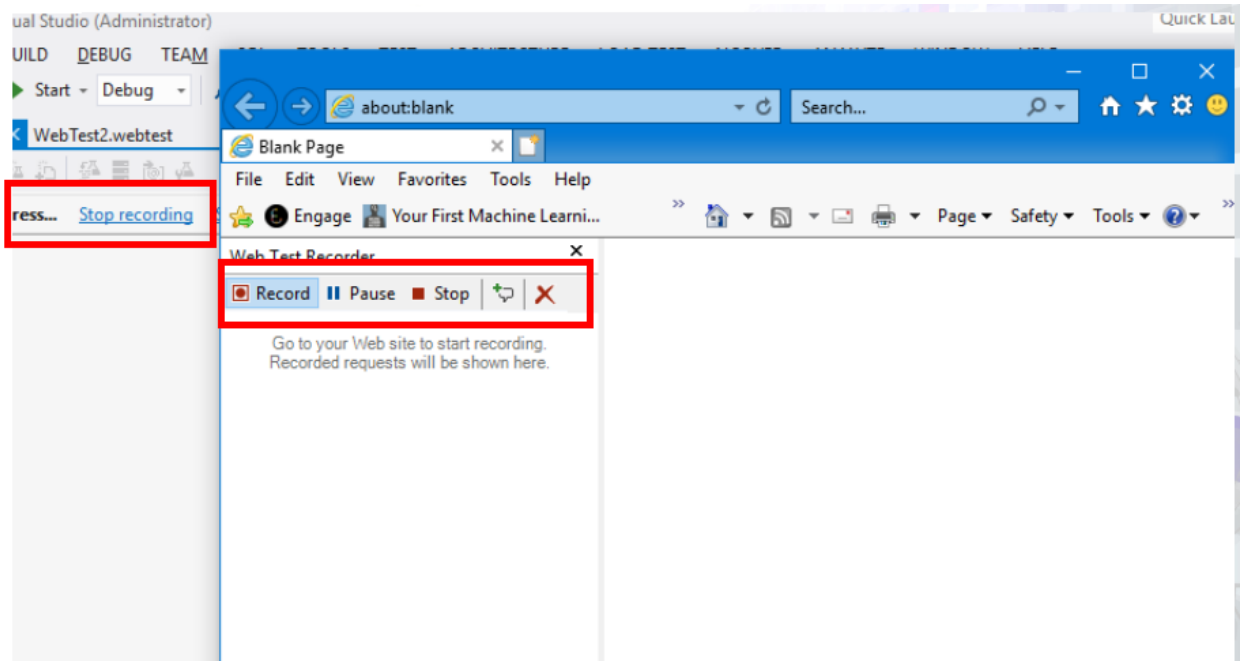
Module 115:

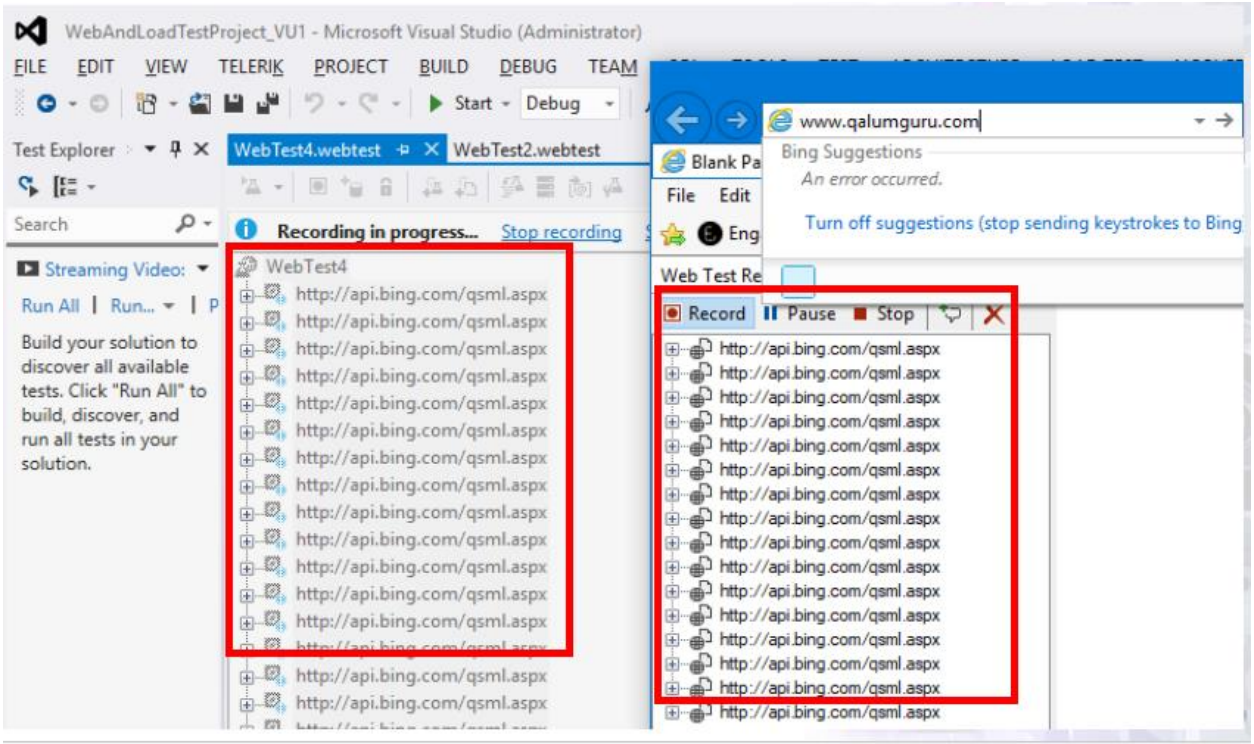
Web Test Recording with Microsoft Visual Studio

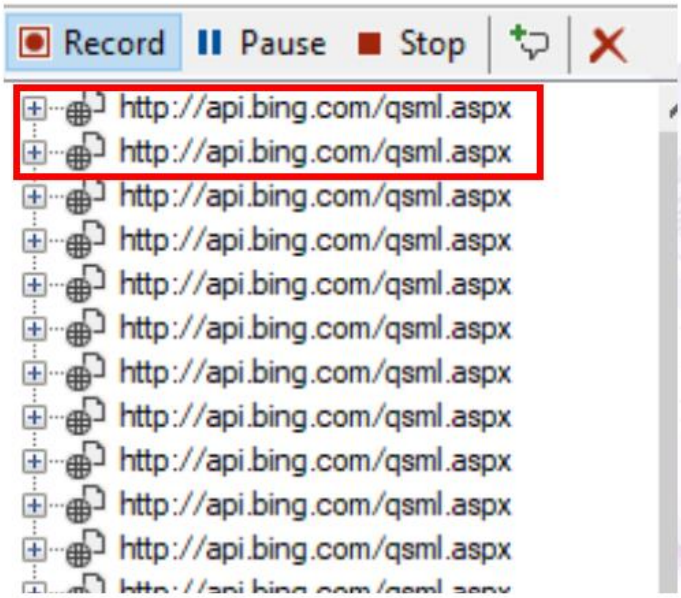
- We can record a test case as a web test case using Microsoft Visual Studio
- These test cases serve many purposes
- One purpose is a record of them for rerunning



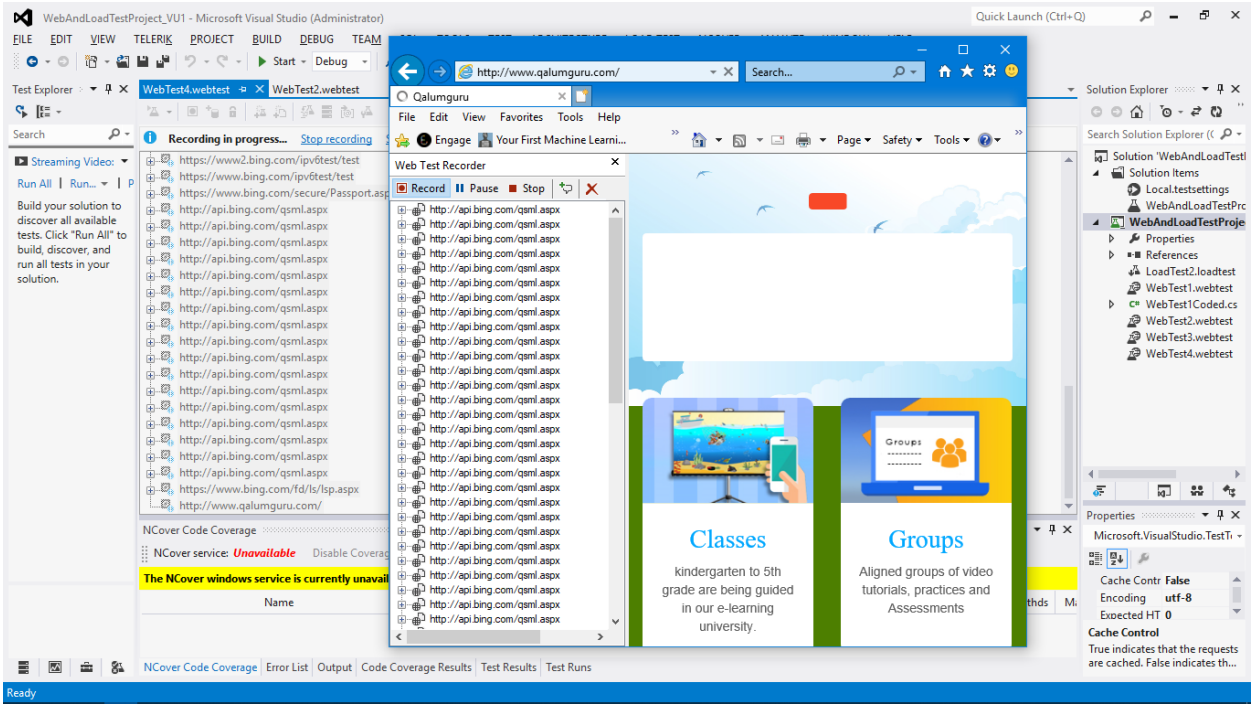


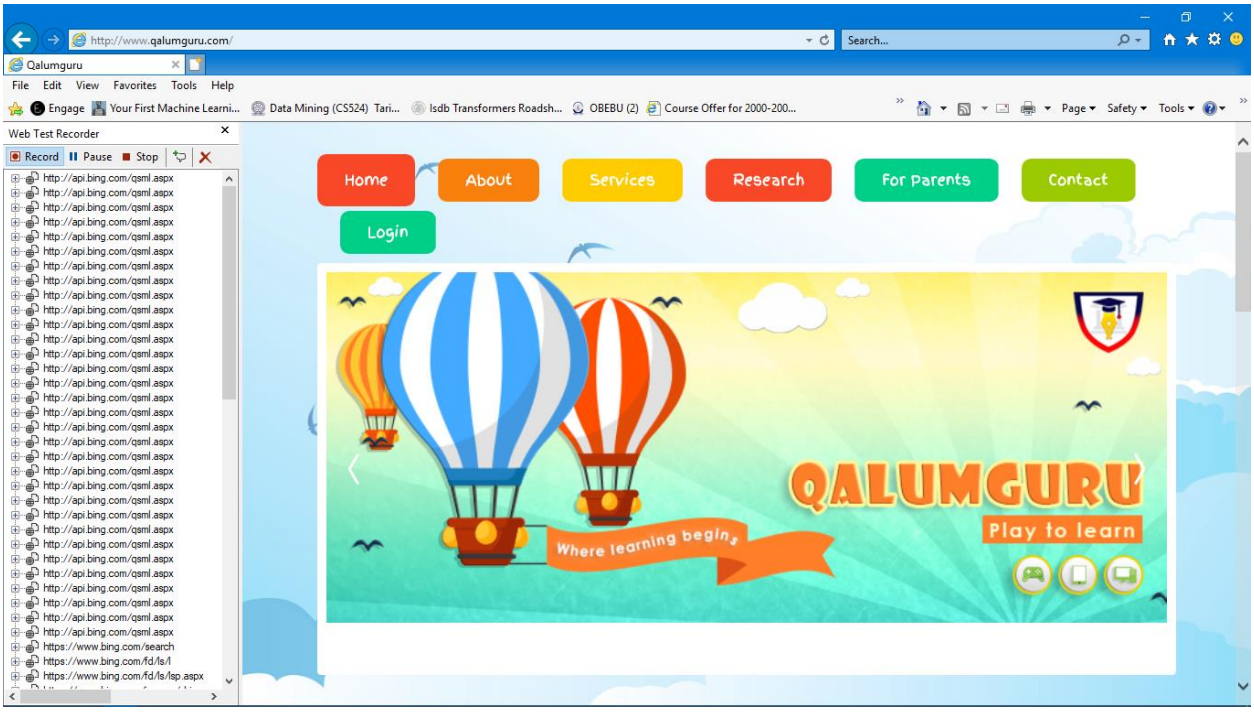

ual Studio (Administrator)
UILD DEBUG TEAM
Start Debug
WebTest2.webtest
ress... Stop recording
about:blank
Search...
Blank Page
File Edit View Favorites Tools Help
Engage Your First Machine Learni...
Page Safety Tools
Record Pause Stop
Go to your Web site to start recording.
Recorded requests will be shown here.

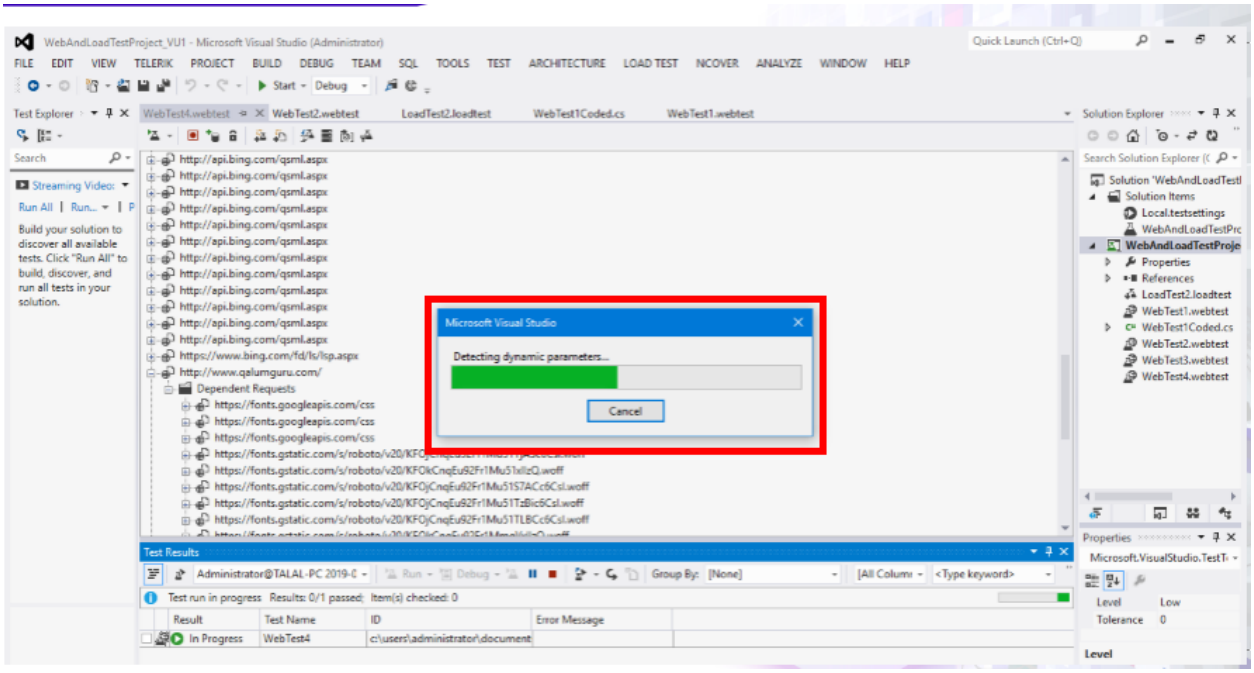

WebAndLoadTestProject_VU1 - Microsoft Visual Studio (Administrator)
FILE EDIT VIEW TELERIK PROJECT BUILD DEBUG TEAM
Start Debug
Test Explorer
WebTest4.webtest
WebTest2.webtest
Search
Recording in progress... Stop recording
Streaming Video:
Run All | Run...
Build your solution to discover all available tests. Click "Run All" to build, discover, and run all tests in your solution.
WebTest4
http://api.bing.com/qsml.aspx
www.qalumguru.com
Bing Suggestions
An error occurred.
Turn off suggestions (stop sending keystrokes to Bing
File Edit
Record Pause Stop

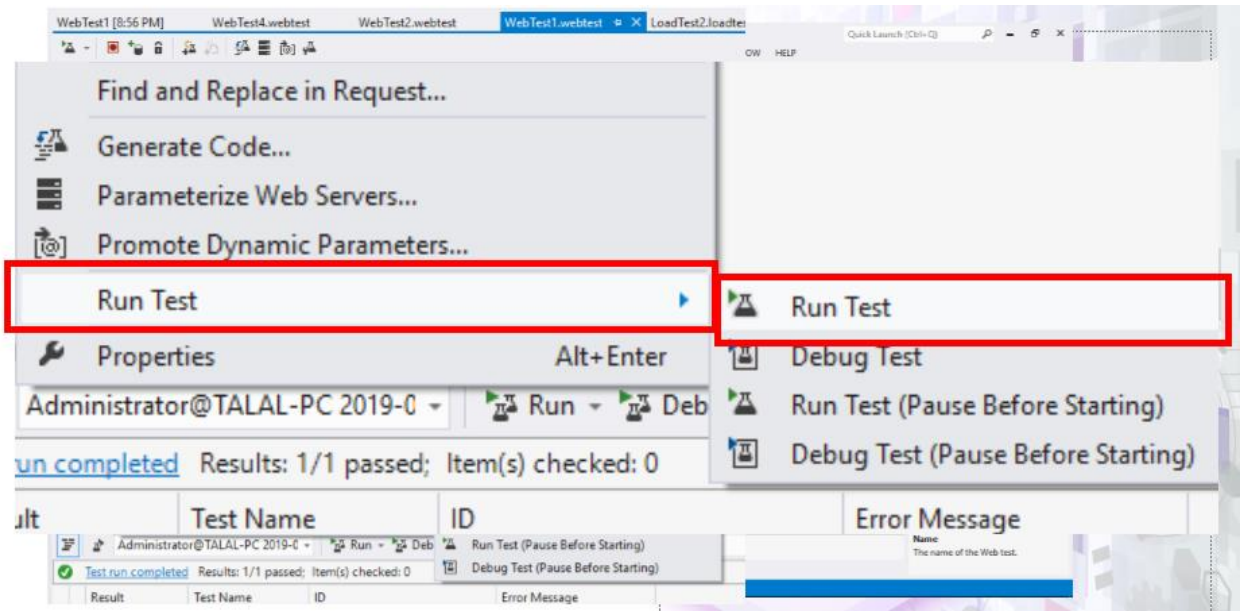


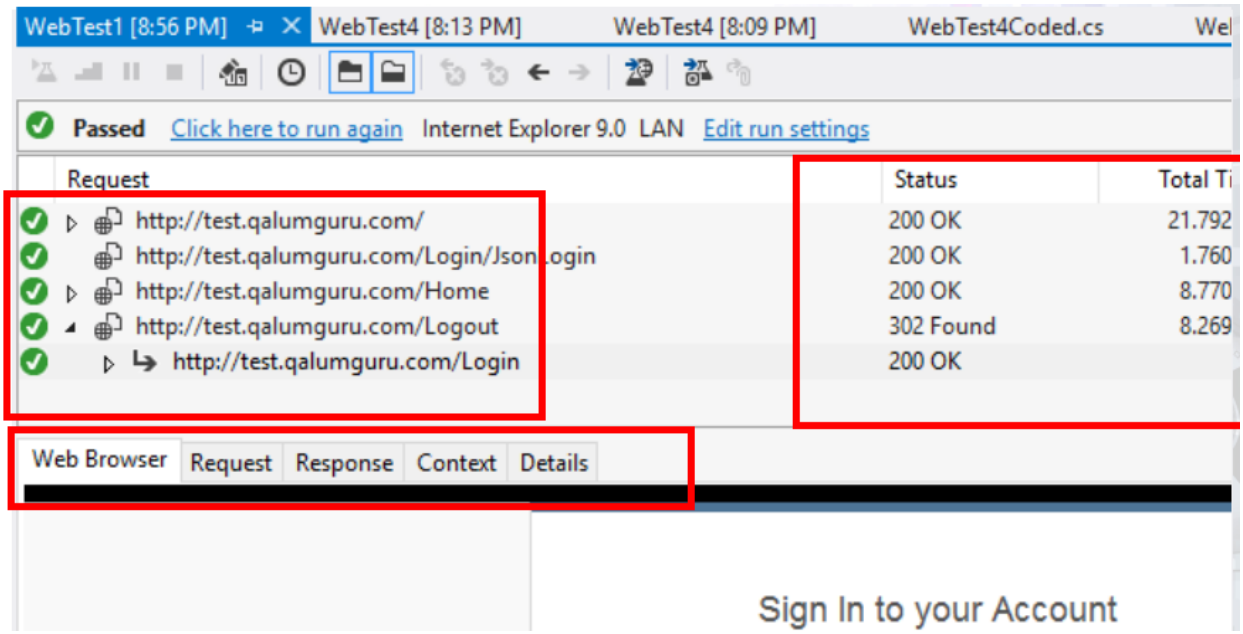
- It is worth noting that:
  - While recording, the internet went off
  - The explorer launched Bing and recorded as a step.
  - We remove them once we have all the recording steps.

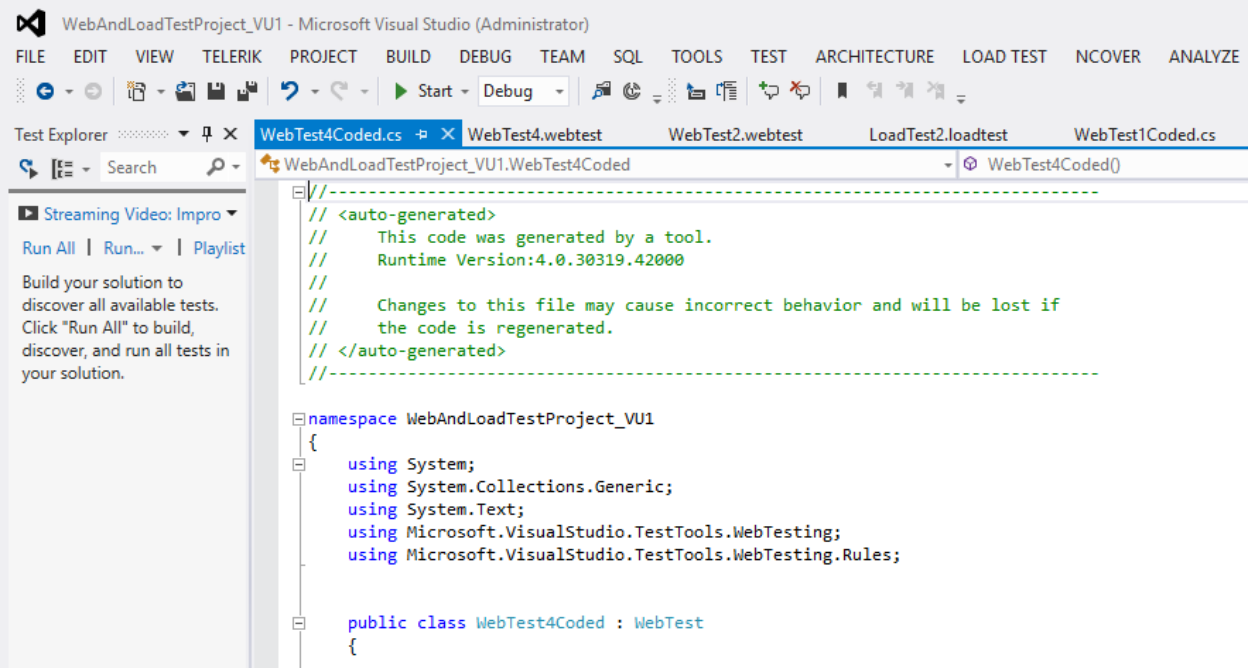



Home
About
Services
Research
For Parents
Contact
Login
QALUMGURU
Play to learn
Where learning begins


Microsoft Visual Studio
Detecting dynamic parameters...
Cancel


Find and Replace in Request...
Generate Code...
Parameterize Web Servers...
Promote Dynamic Parameters...
Run Test
Properties
Alt+Enter
Run Test
Debug Test
Run Test (Pause Before Starting)
Debug Test (Pause Before Starting)
Administrator@TALAL-PC 2019-0
Run
Deb
un completed
Results: 1/1 passed; Item(s) checked: 0
Test Name
ID
Error Message


WebTest1 [8:56 PM]
WebTest4 [8:13 PM]
WebTest4 [8:09 PM]
WebTest4Coded.cs
Passed
Click here to run again
Internet Explorer 9.0 LAN
Edit run settings
Request
Status
Total Ti
http://test.qalumguru.com/
200 OK
21.792
http://test.qalumguru.com/Login/Json
ogin
200 OK
1.760
http://test.qalumguru.com/Home
200 OK
8.770
http://test.qalumguru.com/Logout
302 Found
8.269
http://test.qalumguru.com/Login
200 OK
Web Browser
Request
Response
Context
Details
Sign In to your Account



- Now these test cases are available as test suite
- They can be run on a click
- We can rerun them anytime and see if there is any regression in the overall system quality.

Module 116:

Load Testing

- Load Testing is:
  - A type of non-functional testing.
  - A load test is type of software testing which is conducted to understand behaviour of SUT under a specific expected load.
  - Load testing is performed to determine a system's behaviour under both normal and at peak conditions
- Load Testing is performed considering requirement such as:
  - Type of load test
  - User load estimation
  - User load distribution
  - User activity analysis
  - Production environment analysis
  - Database size
  - Report generation
  - Simple metric collection
- We determine
  - Goal of the test
  - Do we intent to find the synchronization problems?
  - Do we intend to perform a load test? Or a stress test? Or a performance test?
- We shall cover load testing in detail.
- For the moment, we discuss load testing in continuation of web test recording using Visual Studio

Module 117:

Types of Load Tests

- We determine end goal of the test:
- We ask ourselves questions such as:
    - Do we intent to find the synchronization problems?
    - Do we intend to perform a load test? Or a stress test? Or a performance test?

We discuss a few load test types

- **User load estimation:**
    - We try to find out system behavior when subjected to expected load.
    - Example: You are developing system for Uber, you would like to see how many parallel transactions your SUT would support
- **User load distribution**
- Considering the same Uber application, in load distribution, you would like to see system behavior since:
    - The user load may be peak in the morning or afternoon and very less in the evening.
    - This may also help determine the concurrent users and simultaneous users.
- **User activity analysis**
    - There are a variety of functions that your SUT can perform,
    - Example: In morning there may be 200 forms submitted or in evening most of the users may login and perform results generation activity so in morning scenario out of 1000 concurrent user, 70% may be performing this activity.
    - You would like to see system behavior under different sets of activities performed by users.
- **Production environment analysis:**
    - Each connection requires X amount of RAM and disk quota for your application.
    - This may input valuable information as:
        - does the environment have enough hardware?
        - Is it running in a clustered environment?

- **Database size analysis:**
    - The database size does matter in the load test.
    - Bulky the database, more the latency.
    - DBAs update PCT-Used, PCT-Free depending upon how frequent updates or addition/deletion activities are

Module 118:

Load / Stress / Performance Testing

- Load test determines how SUT performs under the concurrent user sessions for typical user scenario.
- Load testing subjects system under a calculated load to see performance degradation
- It is:
- Process of exercising the system under test by feeding it the largest tasks it can operate with.
- Constantly increasing the load on the system via automated tools to simulate real time scenario with virtual users

Examples:

- Testing a word processor by editing a very large document.
- For Web Application load is defined in terms of concurrent users or HTTP connections.
- Stress testing examines how application behaves under maximum load.
- In simple terms find the upper threshold for the application below which it can work normally.
- We try to break the system under test by overwhelming its resources or by taking resources away from it
- Our purpose is to make sure that the system fails and recovers gracefully.
- Performance test indicates response time for the entire application from the user's perspective.
- Performance testing is the process of determining speed or effectiveness of a computer, network, software program or device.
- We consider factors:
    - Throughput
    - Response Time
    - Tuning
    - Benchmarking

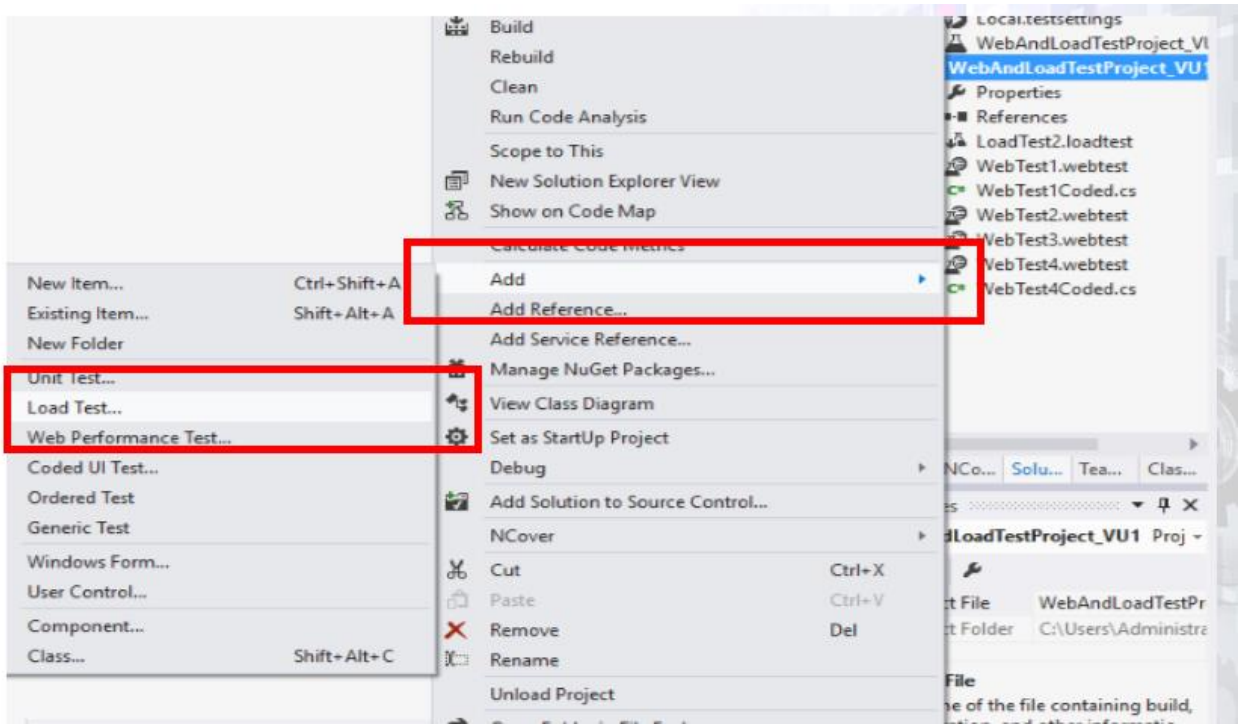
- **Throughput:**
    - Is the capability of a product to handle multiple transactions in a give period.
    - represents number of requests/business transactions processed by the product in a specified time duration.

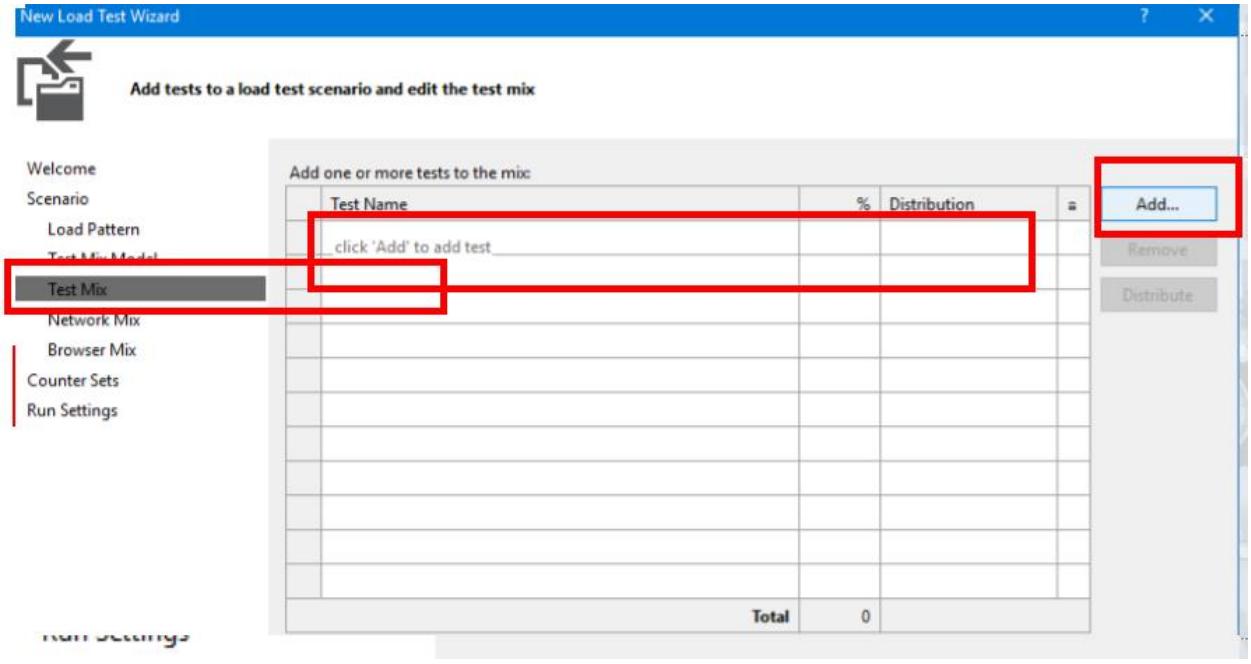
- **Response time is:**
    - equally important to find out how much time each of the transactions took to complete.
    - defined as the delay between the point of request and the first response from the product.
- **Benchmarking is:**
    - System boundaries with respect to number of transactions, parallel connections etc.
- **Tuning is:**
    - is the procedure by which product performance is enhanced by setting different values to the parameters of the product, operating system and other components.
    - improves the product performance without having to touch the source code of the product.

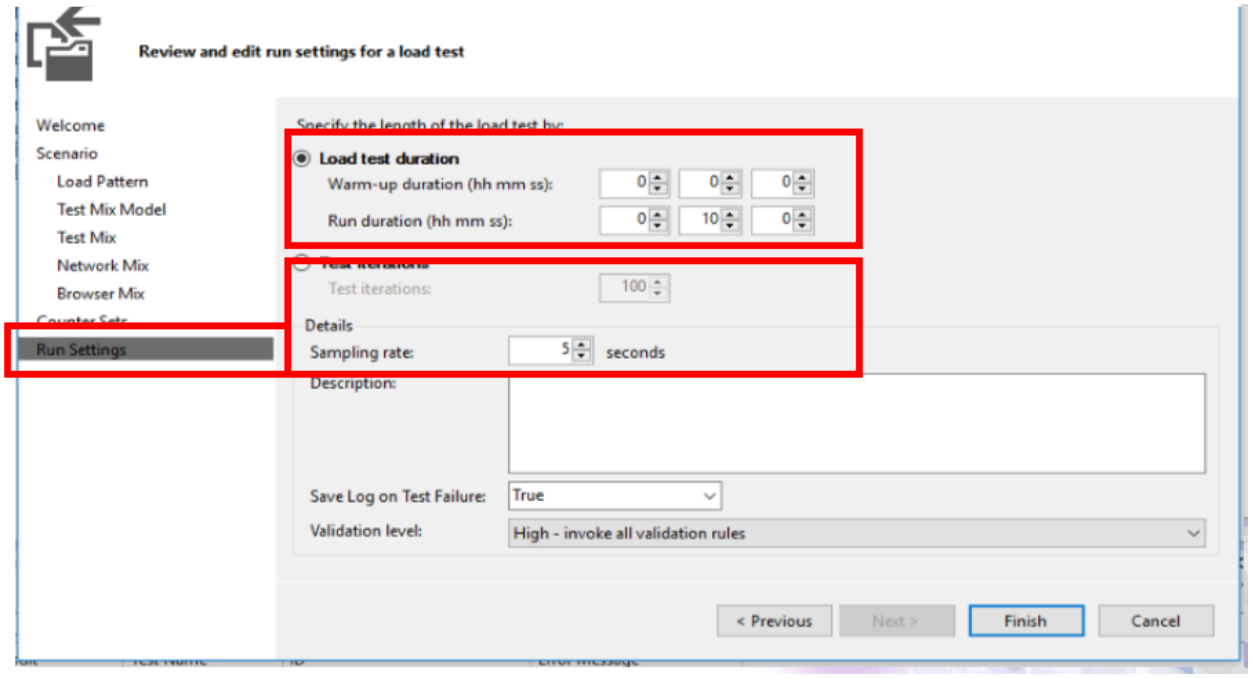
Module 119:

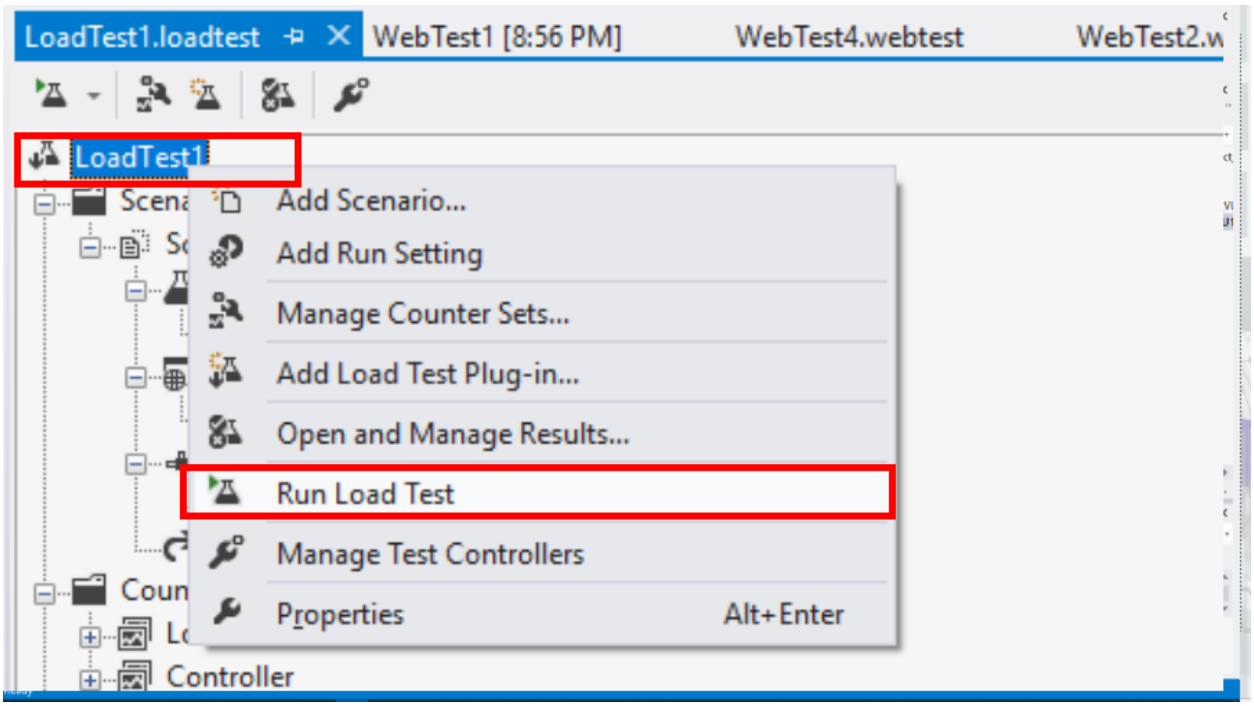
Load Testing with Microsoft Visual Studio

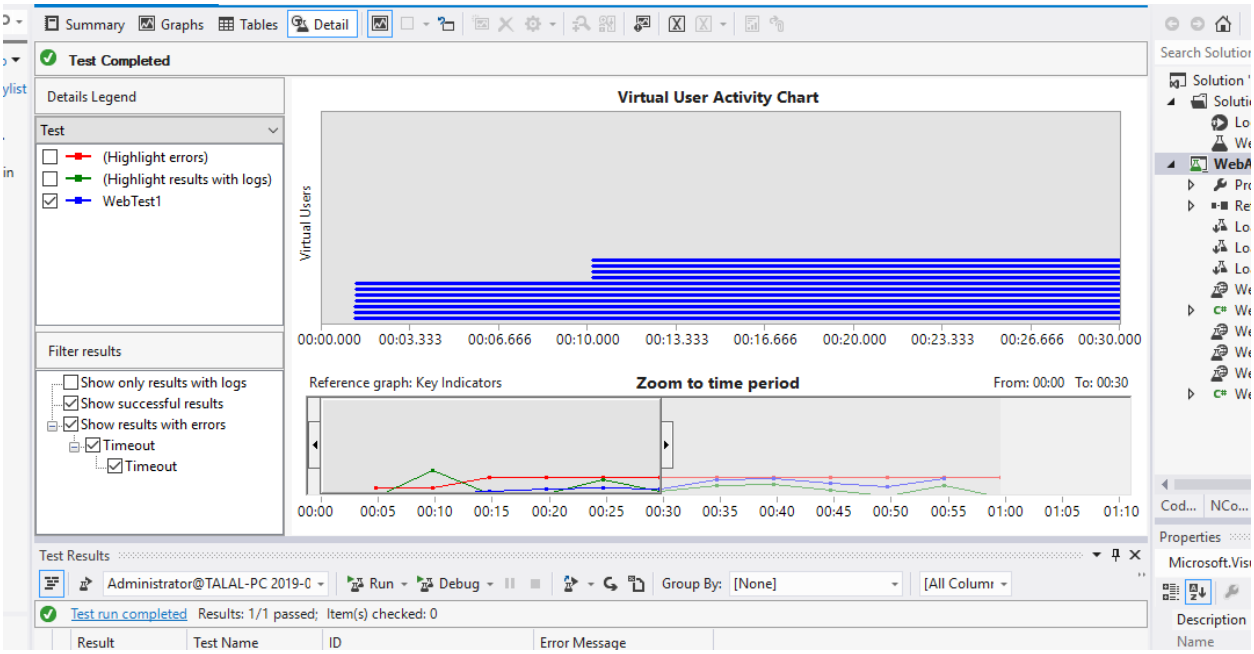
- We have recorded web test using visual studio in Module 114.
- We use the same tool to come up with load test.
- We shall cover load / stress / volume testing in detail in an upcoming module.


Build
Rebuild
Clean
Run Code Analysis
Scope to This
New Solution Explorer View
Show on Code Map
Add
Add Reference...
Add Service Reference...
Manage NuGet Packages...
View Class Diagram
Set as StartUp Project
Debug
Add Solution to Source Control...
NCover
Cut Ctrl+X
Paste Ctrl+V
Remove Del
Rename
Unload Project
New Item... Ctrl+Shift+A
Existing Item... Shift+Alt+A
New Folder
Unit Test...
Load Test...
Web Performance Test...
Coded UI Test...
Ordered Test
Generic Test
Windows Form...
User Control...
Component...
Class... Shift+Alt+C
WebAndLoadTestProject_VU1
Properties
References
LoadTest2.loadtest
WebTest1.webtest
WebTest1Coded.cs
WebTest2.webtest
WebTest3.webtest
WebTest4.webtest
WebTest4Coded.cs


New Load Test Wizard
Add tests to a load test scenario and edit the test mix
Welcome
Scenario
Load Pattern
Test Mix
Network Mix
Browser Mix
Counter Sets
Run Settings
Add one or more tests to the mix:
Test Name
%
Distribution
click 'Add' to add test
Total
0
Add...
Remove
Distribute


Review and edit run settings for a load test
Welcome
Scenario
Load Pattern
Test Mix Model
Test Mix
Network Mix
Browser Mix
Run Settings
Load test duration
Warm-up duration (hh mm ss):
Run duration (hh mm ss):
Test iterations:
Details
Sampling rate:
seconds
Description:
Save Log on Test Failure:
True
Validation level:
High - invoke all validation rules
< Previous
Next >
Finish
Cancel


LoadTest1.loadtest
WebTest1 [8:56 PM]
WebTest4.webtest
LoadTest1
Add Scenario...
Add Run Setting
Manage Counter Sets...
Add Load Test Plug-in...
Open and Manage Results...
Run Load Test
Manage Test Controllers
Properties
Alt+Enter
Controller



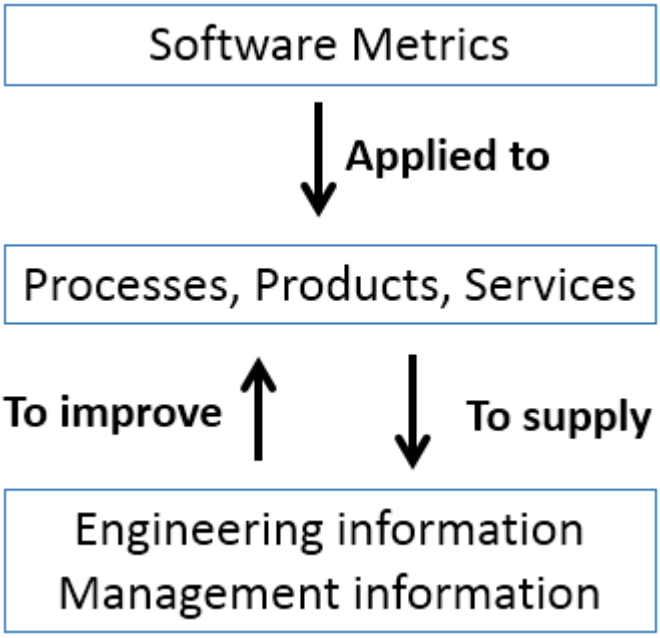
Module 120:

Software Test Metrics

- Definition: Metric is a standard unit of measurement that quantifies results.
- Metric used for evaluating the software processes, products and services is termed as Software Metrics.
- Software Metrics is:
    - A Measurement Based Technique
    - applied to processes, products and services
    - to supply engineering and management information
    - and working on the information supplied to improve processes, products and services, if required.

- The aim of collecting test metrics is to use the data for improving the test process, rather than to just show fancy reports.
- This includes finding tangible answers to the questions:
    - How long will it take to test?
    - How much money will it take to test?
    - How bad are the bugs?
    - How many bugs found were fixed? reopened? closed? deferred?

    - How many bugs did the test team did not find?
    - How much of the software was tested?
    - Will testing be done on time? Can the software be shipped on time?
    - How good were the tests? Are we using low-value test cases?
    - What is the cost of testing?
    - Was the test effort adequate? Could we have fit more testing in this release?



- We gather information regarding:
    - Total number of test cases
    - Test cases passed / failed / blocked
    - Total defects found
    - Defects accepted / rejected
    - Defects deferred
    - Critical defects
    - Number of planned test hours and actual test hours
    - Bugs found after shipping
- We derive important information to measure:
    - how effective was our test process
    - How can be further improved

Module 121:

Importance of Test Metrics

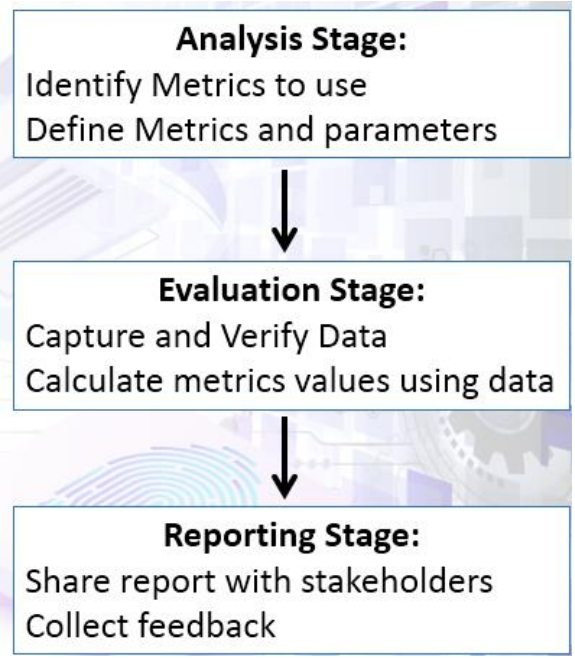
- We need test metrics to:
    - Track and control phases of testing and fixing
    - Find out if the system is ready for release
    - Improve our inhouse process
    - Find out if our testing process is efficient
    - Find out when to stop testing
    - Do trend analysis
    - Meet client expectations and customer needs

- While collecting data and while performing analysis, we need to find out if:
    - The whole process was meaningful within the context of our organization and SUT
    - Understand impact of the metric and the variance
    - Reliability and consistency
    - It was economical to measure
    - We have means for comparing our results to some benchmark or process capability baseline
- We also need to classify the Test metric with respect to:
    - Quality
    - Productivity
    - Cost
    - Defect
    - Coverage
    - Effort
    - Stability
    - Effectiveness

Module 122:

Test Metrics Lifecyle

- Metrics is used to improve the quality and productivity of products and services thus achieving Customer Satisfaction.
- Easy for management to digest one number and drill down, if required.
- Different Metric(s) trend act as monitor when the process is going out-of-control.
- Metrics provides improvement for current process.



Module 123:

Test Metrics Classification

- **Manual Testing Metrics**
  - Test Case productivity
  - Test execution summary
  - Defect acceptance
  - Defect rejection
  - Bad fix defect
  - Test execution productivity
  - Test efficiency
  - Defect severity

- **Performance Testing Metrics**
  - Performance scripting productivity
  - Performance execution summary
  - Performance Efficiency
  - Performance severity Index
- **Automation Testing Metrics**
  - Automation scripting productivity
  - Automation test execution productivity
  - Automation coverage
  - Cost comparison

Module 124:

Defect Related Metrics

- **Defect Acceptance** helps determines number of valid defects that testing team identified during execution.
- Value of this metric is compared with previous releases for a clearer picture

$$Defect\ Acceptance = \left[\frac{Number\ of\ Valid\ Defects}{Total\ Number\ of\ Defects} * 100 \right] \%$$



- **Defect Rejection**
    - This metric determine the number of defects rejected during execution.
    - It gives the percentage of the invalid defect the testing team has opened and one can control, whenever required.

$$Defect\ Rejection = \left[\frac{Number\ of\ Defect(s)\ Rejected}{Total\ Number\ of\ Defects} * 100 \right] \%$$

- **Bad Fix Defect**
    - Defect whose resolution give rise to new defect(s) are bad fix defect.
    - This metric determine the effectiveness of defect resolution process
    - It gives the percentage of the bad defect resolution which needs to be controlled

$$Bad\ Fix\ Defect = \left[\frac{Number\ of\ of\ Bad\ Fix\ Defect(s)}{Total\ Number\ of\ Valid\ Defects} * 100 \right] \%$$

- **Defect Severity Index**
    - This metric determine the quality of the product under test and at the time of release, based on which one can take decision for releasing of the product i.e. it indicates the product quality.

$$Defect\ Severity\ Index = \left[\frac{\sum (Severity\ Index * No.\ of\ Valid\ Defect(s)\ for\ this\ severity}{Total\ Number\ of\ Valid\ Defects} \right]$$

- One can divide the Defect Severity Index in two parts: -

1. **DSI for All Status defect(s):** - This value gives the product quality under test.
2. **DSI for Open Status defect(s):** - This value gives the product quality at the time of release. For calculation of DSI for this, only open status defect(s) must be considered.

$$DSI(Open) = \left[\frac{\sum (Severity\ Index * No.\ of\ Open\ Valid\ Defect(s)\ for\ this\ severity)}{Total\ Number\ of\ Open\ Valid\ Defects} \right]$$

Module 125:

Testing Efficiency Metrics

- Test Execution Summary:
  - Gives classification of the test cases with respect to execution status along with reason, if available, for various test cases.
  - Gives statistical view of the release.
  - Data for number of test case executed and status:
    - Pass / Fail (reason for failure)
    - Unable to Test with reason e.g., postponed defect, setup issue, out of scope.
- Test Case Productivity
  - This metric gives the test case writing productivity based on which one can have a conclusive remark
  - 120 / 8 = TCP = 15 steps/hour

$$Test\ Case\ Productivity\ = \left[\frac{Total\ Raw\ Test\ Steps}{Efforts\ (hours)} \right] Step(s)\ /\ hour$$

Test Case Name	Raw Steps
1	30
2	32
3	45
4	13
Total Raw Steps	120
Total Hours	8

- **Test Execution Productivity**
    - This metric gives the test cases execution productivity which on further analysis can give conclusive result.

$$Test\ Execution\ Productivity = \left[\frac{Total\ No.\ of\ TC\ executed\ (Te)}{Execution\ Efforts\ (hours)} * 8 \right] Execution(s) / Day$$

    - One can compare the productivity with previous release and can have an effective conclusion.

- **Test Efficiency**
    - This metric determines the efficiency of the testing team in identifying the defects. It also indicated the defects missed out during testing phase which migrated to the next phase.

$$Test\ Efficiency = \left[\frac{DT}{DT + DU} * 100 \right] \%$$

    - Where,
        - DT = Number of valid defects identified during testing.
        - DU = Number of valid defects identified by user after release of application.
        - In other words, post-testing defect

Module 126:

Performance Testing Metrics

- Performance Scripting Productivity
    - This metric gives the scripting productivity for performance test script and have trend over a period of time.

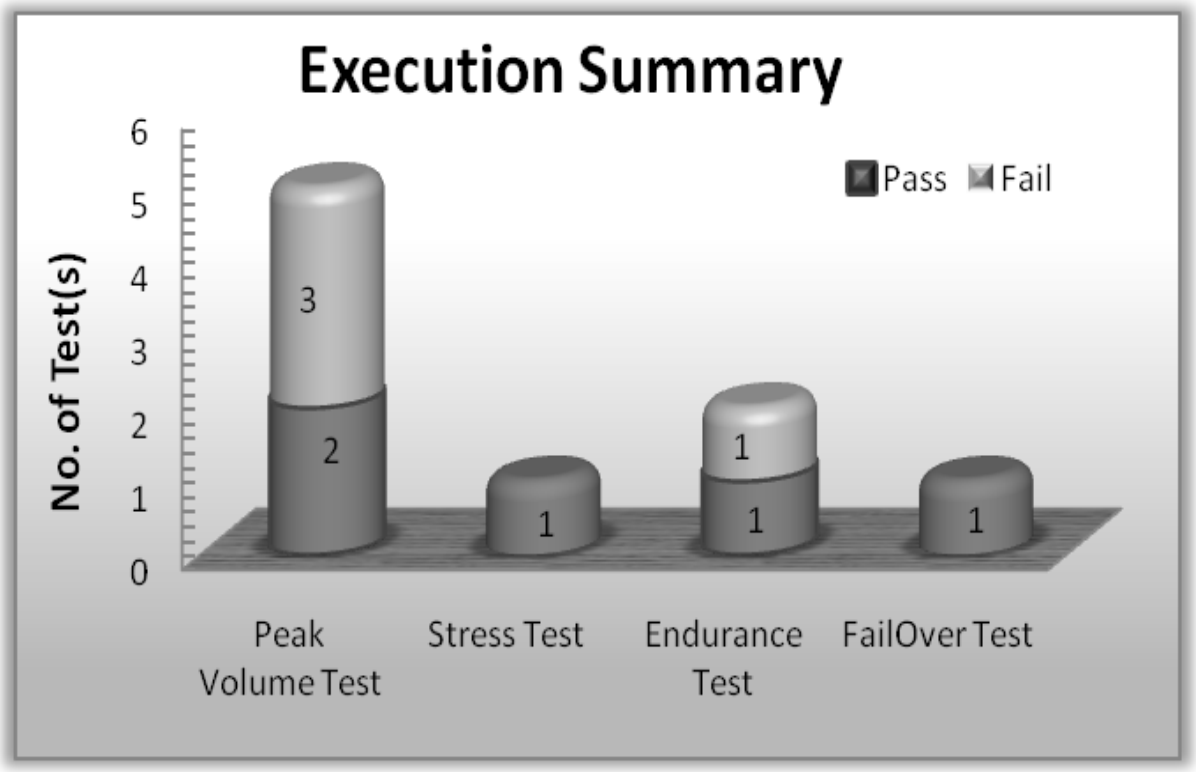
$$Performance\ Scripting\ Productivity = \left[\frac{\sum Operations\ Performed}{Efforts\ (hours)} \right] Operation(s) / hour$$

    - Where Operations performed is: -
    - No. of Click(s) i.e. click(s) on which data is refreshed.

- No. of Input parameter
- No. of Correlation parameter

- Above evaluation process does include logic embedded into the script which is rarely used.

- **Performance Execution Summary**
  - gives classification with respect to number of test conducted along with status (Pass/Fail), for various types of performance testing.
  - Some of the types of performance testing: -
    1. Peak Volume Test.
    2. Endurance/Soak Test.
    3. Breakpoint/Stress Test.
    4. Failover Test



- **Performance Execution Data - Client Side**
    - This metric gives the detail information of Client side data for execution. Following are some of the data points of this metric
    - Running Users
    - Response Time
    - Hits per Second
    - Throughput
    - Total Transaction per second
    - Time to first byte
    - Error per second

- **Performance Execution Data - Server Side**
    - This metric gives the detail information of Server side date for execution. Following are some of the data points of this metric:
        1. CPU Utilization
        2. Memory Utilization
        3. HEAP Memory Utilization
        4. Database connections per second

Module 127:

Performance Testing Efficiency Metrics

- **Performance Test Efficiency**
    - This metric determine the quality of the Performance testing team in meeting the requirements which can be used as an input for further improvisation, if required.

$$Performance\,Test\,Efficiency = \left[\frac{(Req\ met\ during\ PT) - (Req\ not\ met\ after\ Signoff\ of\ PT)}{Req\ met\ during\ PT} * 100 \right] \%$$

    - To evaluate this one need to collect data point during the performance testing and after the signoff of the performance testing.

- **Performance Severity Index**
    - This metric determine the product quality based performance criteria on which one can take decision for releasing of the product to next phase i.e. it indicates quality of product under test with respect to performance.

$$Performance\ Severity\ Index = \left[\frac{\sum (Severity\ Index * No.\ of\ \ Req.\ not\ met\ \ for\ this\ severity)}{Total\ No.\ of\ Req.\ not\ met} \right]$$

    - If requirement is not met, one can assign the severity for the requirement so that decision can be taken for the product release with respect to performance.

Module 128:

Automation Testing Metrics

- Automation Scripting Productivity
    - This metric gives the scripting productivity for automation test script based on which one can analyze and draw most effective conclusion from the same.

$$Automation\ Scripting\ Productivity = \left[\frac{\sum Operations\ Performed}{Efforts\ (hours)} \right] Operation(s)\ /\ hour$$

    - Where Operations performed is: -
    - No. of Click(s) i.e. click(s) on which data is refreshed.
    - No. of Input parameter
    - No. of Checkpoint added
- Above process does include logic embedded into the script which is rarely used.

- **Automation Test Execution Productivity**
    - This metric gives the automated test case execution productivity.

$$Auto.\ Execution\ Productivity = \left[\frac{Total\ No.\ of\ Automated\ TC\ executed\ (ATe)}{Execution\ Efforts\ (hours)} * 8 \right] Execution(s) / Day$$

Where ATe is calculated as,

$$ATe = Base\ Test\ Case + ((T\ (0.33) * 0.33) + (T\ (0.66) * 0.66) + (T\ (1) * 1))$$

Evaluation process is similar to Manual Test Execution Productivity

- **Automation Coverage**
    - This metric gives the percentage of manual test cases automated.

$$Automation\ Coverage = \left[\frac{Total\ No.\ of\ TC\ Automated}{Total\ No.\ of\ Manual\ TC} * 100 \right] \%$$

Example

If there are 100 Manual test cases and one has automated 60 test cases then Automation Coverage = 60%

- **Cost Comparison**
    - This metrics gives the cost comparison between manual testing and automation testing. This metrics is used to have conclusive ROI (return on investment).
- If Script is re-used the script development cost will be the script update cost.

Module 129:

Common Testing Metrics

- **Effort Variance**
  - This metric gives the variance in the estimated effort.

$$Effort\ Variance = \left[\frac{Actual\ Effort - Estimated\ Effort}{Estimated\ Effort} * 100 \right] \%$$

- **Schedule Variance**
  - This metric gives the variance in the estimated schedule i.e. number of days.

$$Schedule\ Variance = \left[\frac{Actual\ No.\ of\ Days - Estimated\ No.\ of\ Days}{Estimated\ No.\ of\ Days} * 100 \right] \%$$

- **Scope Change**
  - This metric indicates how stable the scope of testing is.

$$Scope\ Change = \left[\frac{Total\ Scope - Previous\ Scope}{Previous\ Scope} * 100 \right] \%$$

  Where,

  - Total Scope = Previous Scope + New Scope, if Scope increases
  - Total Scope = Previous Scope - New Scope, if Scope decreases

- We have seen how metrics are collected and how do they play a vital role in decision making and process improvement
- In earlier slides, we discussed importance of metrics as follows:
  - Track and control phases of testing and fixing
  - Find out if the system is ready for release
  - Improve inhouse process
  - When to stop testing
  - Do trend analysis
  - Meet client expectations and customer needs

Module 130:

Penetration Testing

- **Penetration (Pen) Testing is the process of:**
    - finding and reporting security vulnerabilities in
        - network,
        - computer or
        - software application
    - in order to secure it with any sort of attacks.
- Our scope for this course is software applications under test (SUT).
- With the increased cyber attacks, performing security testing is important.
- Penetration testing helps identify vulnerable areas of the system.
- It involves intentional attacks on SUT to identify weak areas, which might provide a passage to malicious or unauthorized users for attacking the system and altering their integrity and veracity.
- Helpful in fixing security related defects/loopholes.

- **Benefits:**
    - It Exposes Weaknesses Before Real Hackers Do
    - It Can Reveal Which Areas of Security we Need to Invest In
    - It Provides an Outsider Perspective on our Security
    - It Simulates a Real Attack Scenario
- **Different forms of Pen Testing are:**
    - Network Vulnerability Testing
    - Web Vulnerability Testing
    - Wireless War Driving / Walking
    - Phone Network Testing
    - Social Engineering Testing
    - Walk-throughs and Dumpster Diving
    - Physical Security Auditing..

Module 131:

Penetration Testing Phases

- **White Box Penetration Testing:** Here, the tester has complete access and in-depth knowledge of the system to be tested. This is very helpful in carrying out extensive penetration testing.
- **Black Box Penetration Testing:** In black box penetration testing approach, high-level of information is made available to the tester. The tester is totally unaware of the system/network. However, this approach might miss some areas while testing.
- **Gray Box Penetration testing:** Gray box penetration testing makes only limited information available to the tester to attack the system externally.

Steps:

- **Scanning / Reconnaissance**
    - Getting to know the target using passive methods.
    - The idea of this phase is to gather as much info about the subject as we possibly can
- **Methods:**
    - Search engine queries to gather data about the personnel, systems, or technologies of the client.
    - Social Engineering to find out positions, technologies, email addresses
    - Internet foot-printing looking for email addresses, social accounts, names, positions
    - Dumpster diving to find valuable data that may be used for attacks or social engineering
    - Tailgating to get physical access or pictures with hidden cameras
- **Threat Modelling**
- A description or model of all the security concerns and why they should be resolved.
    - Once we feel we have sufficient info about the client's systems, we start modelling the threats that the client would realistically face and identify vulnerabilities that will allow for those attacks.
    - It's kind of a pre-attack phase in which we get everything ready.

- **Vulnerability Analysis**
    - Identifying vulnerabilities and determining their severity
- **Exploitation**
    - Gaining access by breaching security of a system or finding an bug to exploit in the software.
- **Post-Exploitation Reporting**
    - Detailing the vulnerabilities found and providing information on potential impact
- **Post-Exploitation Reporting**
- Detailing the vulnerabilities found and providing information on potential impact..

Module 132:

Types of Penetration Testing

- Manual Penetration Testing: It involves a standard approach with different activities to be performed in a sequence.
    - Penetration testing planning
    - Reconnaissance
    - Vulnerability Analysis
    - Exploitation
        - External attacks
        - Internal attacks
    - Post Exploitation
    - Reporting

- Automated Penetration Testing: Automated penetration testing can be performed using various performance testing tools / scanners
- Some of the most commonly used tools include the following.
    - NMap.
    - Nessus.
    - Metasploit.
    - Wireshark.
    - Veracode..

Module 133:
Penetration Testing Tools

- Types of Pen Testing/Testers
    - Black Hat: With no prior knowledge of network, computer or application that needs to be tested.
    - White Hat: With complete knowledge of network, computer or application that needs to be tested.
    - Gray Hat: With limited knowledge of network, computer or application that needs to be tested
- Tools are specific to either mimicking a subset of attacks.
- Examples include:
    - Sqlninja, as the name, indicates it is all about taking over the DB server using SQL injection in any environment.
    - John the Ripper is password cracker works on most of the environments, although it's primarily for UNIX systems.
- We present a wide division of these tools
- Network Vulnerability Assessment Tools
    - Nmap
    - OpenVAS
    - Wireshark
    - Metasploit

    - MBSA (Microsoft baseline security analyzer)
- Web Vulnerability Assessment Tools
    - OWASP ZAP
    - Acunetix
    - Burp suite
    - Nikto
    - Sqlmap..

Module 134:
Penetration Testing or Hacking

- Difference between a Pen Tester and a Hacker?
    - Pen Tester's have prior approval from Senior Management
    - Hackers have prior approval from themselves.
    - Pen Tester's social engineering attacks are there to raise awareness
    - Hackers social engineering attacks are there to trick systems into divulging sensitive information about the whereabouts of their estranged ex-spouse.
- Vulnerability Assessment
    - Typically is general in scope and includes a large assessment.
    - Predictable. (I know when those darn Security guys scan us.)
    - Unreliable at times and high rate of false positives. (I've got a banner)
    - Vulnerability assessment invites debate among System Admins.
    - Produces a report with mitigation guidelines and action items.
- *Penetration Testing:*
    - Focused in scope and may include targeted attempts to exploit specific vectors (Both IT and Physical)
    - Unpredictable by the recipient. (Don't know the "how?" and "when?")
    - Highly accurate and reliable. (I've got root!)
    - Penetration Testing = Proof of Concept against vulnerabilities.

- Produces a binary result: Either the team owned you, or they didn't.

Ethical Hacking	**Penetration Testing**
“Ethical hacking focuses on using all techniques to find and exploit the vulnerabilities.”	“Pen testing focuses on finding all vulnerabilities in the network, computer or application.”
Main target is to break in the system	Main target is to defend the system against any threat
Ethical hacking is an offensive measure	Pen testing is defensive measure

- Benefits of Penetration Testing
  - Enables a proactive security approach.
  - Allows exploration of real risks
  - accurate representation of a company’s IT infrastructure security posture at any given time.
  - Helps investigate data breaches or network intrusions to discover any leads to the leakage of data or theft of intellectual property.

- Benefits of Penetration Testing
  - able to assess and validate the efficacy of one’s defensive mechanisms
- Benefits of Penetration Testing
  - It is a time consuming activity.
  - It is high-labor intensive and can therefore represent an increased cost
  - It might give a false sense of security. Being able to withstand most penetration testing attacks might give the sense that systems are 100% safe..

Module 135:
Penetration Testing: Worked Example

Qalum Guru

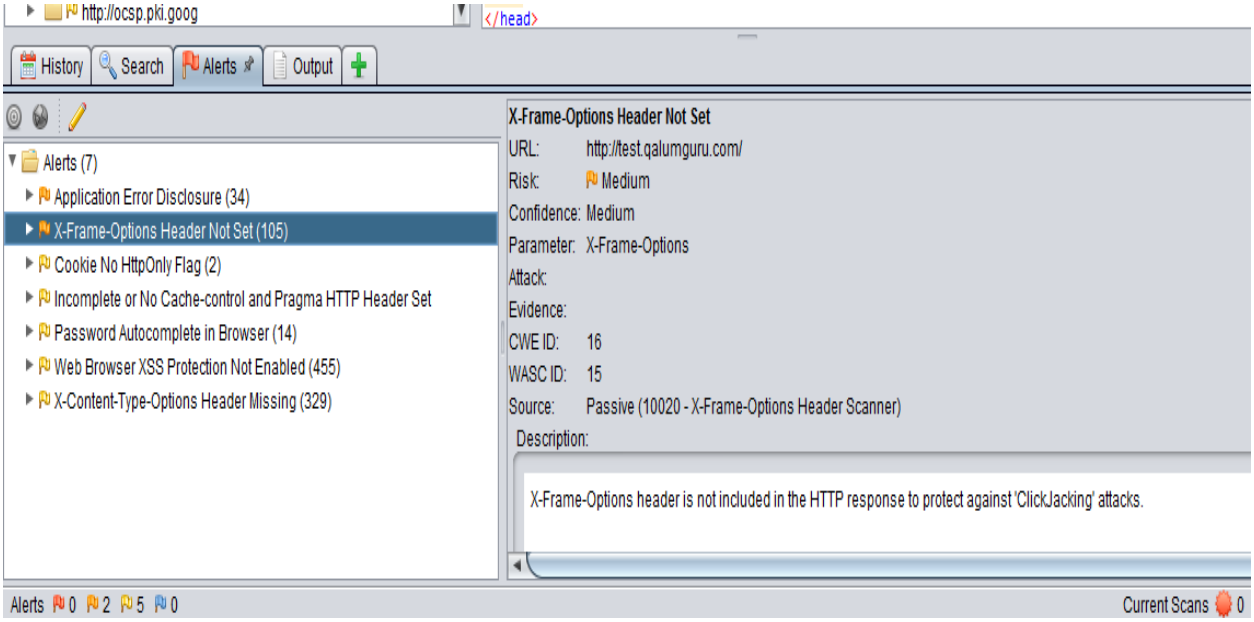
Zed Attack Proxy is an open-source web application security scanner.

Application was taken and

scanned against ZAP OWASP scanner.

As a result, scanner provide list of potential vulnerabilities that can cause potential harm to system, if exploited.





Module 136:
Exploratory Testing

- Exploratory testing is a style of software testing that:
    - emphasizes personal freedom and responsibility of individual tester to continually optimize the quality of his/her work
    - by treating test-related learning, test design, test execution, and test result interpretation as mutually supportive activities that run in parallel throughout the project."

  (Cem Kaner)
- It is about exploring, to understand that what software can do or cannot do.
- It emphasizes personal freedom and responsibility of the individual tester.
- Test cases are not created in advance but testers check system on the fly.
- Both test design and test execution are performed simultaneously
- There is no formal documentation for test scripts, test conditions, or test cases.
- Test logging is performed at the same time as test execution.
- During test logging, we document the key aspects of what functionality is tested, any defects found, and any thoughts regarding further testing.

- It is not a technique but it is an approach. What actions you perform next is governed by what you are doing currently

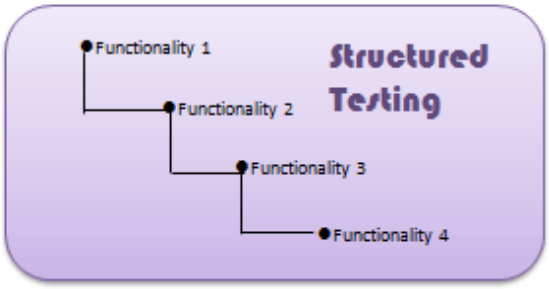
Module 137:
Exploratory Testing Process

Four phases in exploratory testing:

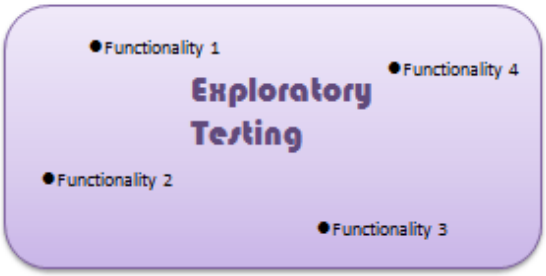
1. Learn
    - What to test?
    - How to test?
    - How to find a problem?
2. Test Design
    - Brief test description rather than test cases
3. Test Execution
    - Test Execution
    - Gather Results
4. Analysis
    - What has been learned about the product?
    - What has been learned about the testing?

Module 138:
Exploratory Testing vs Structured Testing

- In structured testing,
- We design test cases first and later proceed with test execution.
- While doing exploratory testing, test design and test execution is a simultaneous process which is done in parallel.



Functionalities are checked in a structured manner



Functionalities are checked in a ad-hoc manner

Structured Testing	**Exploratory Testing**
▪ Directed from requirements	▪ Directed from requirements and exploring during testing
▪ Determination of test cases well in advance	▪ Determination of test cases during testing
▪ Confirmation of testing with requirements	▪ Investigation of system or application
▪ Emphasizes prediction and decision making	▪ Emphasizes adaptability and learning
▪ Involves confirms testing	▪ Involves investigation
▪ Is about controlling tests	▪ Is about improvement of test design

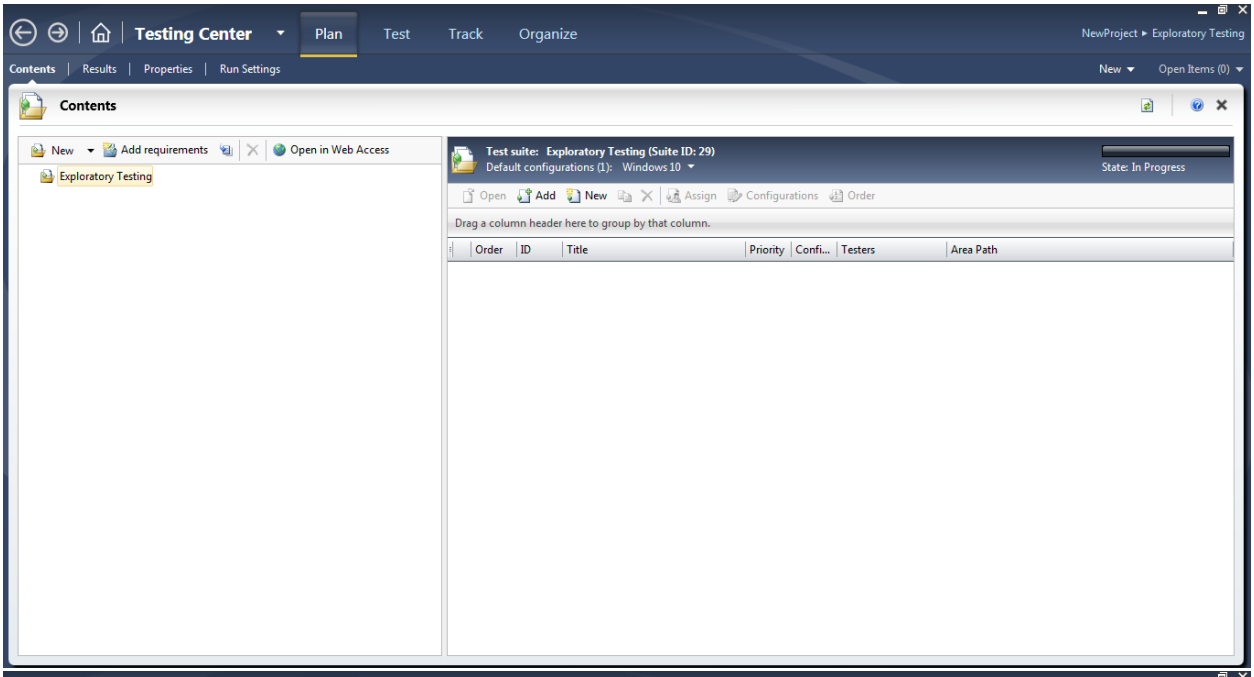
Module 139:
Exploratory Testing Benefits

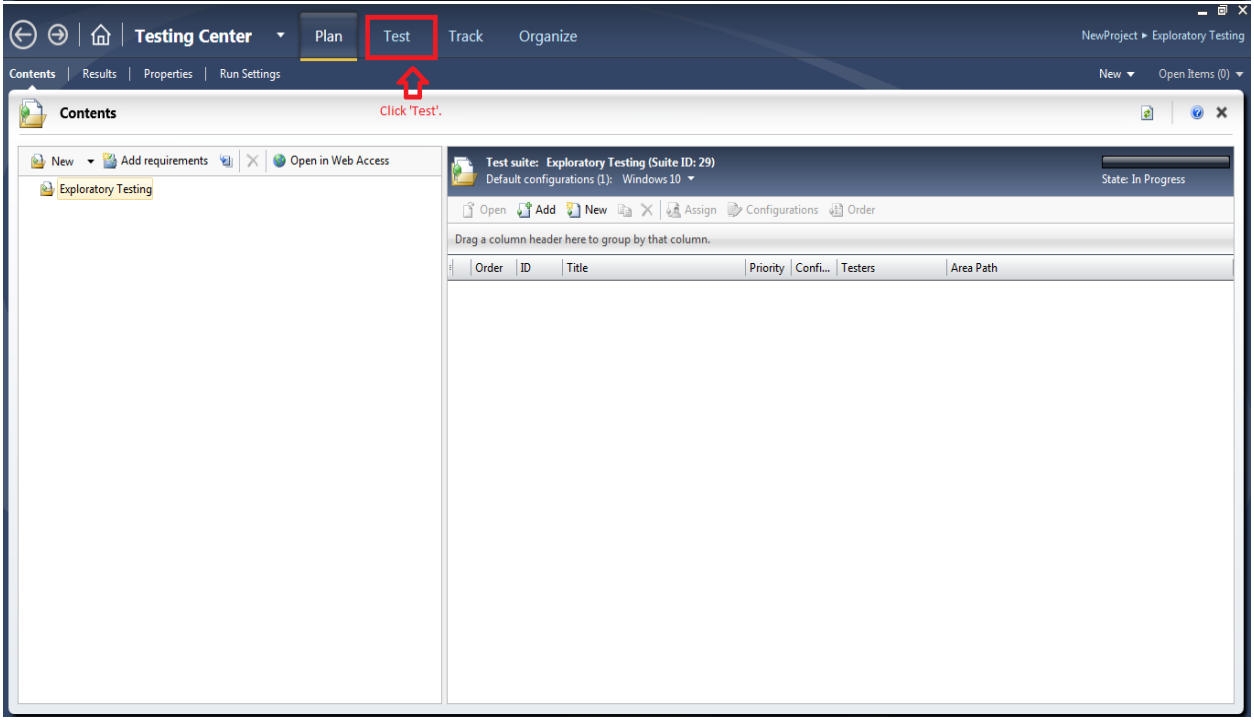
- Benefits of Exploratory Testing:
  - Requires a minimal amount of planning.
  - Testers continuously decide for their next step of testing, and the decision depends upon the tester's thought process.

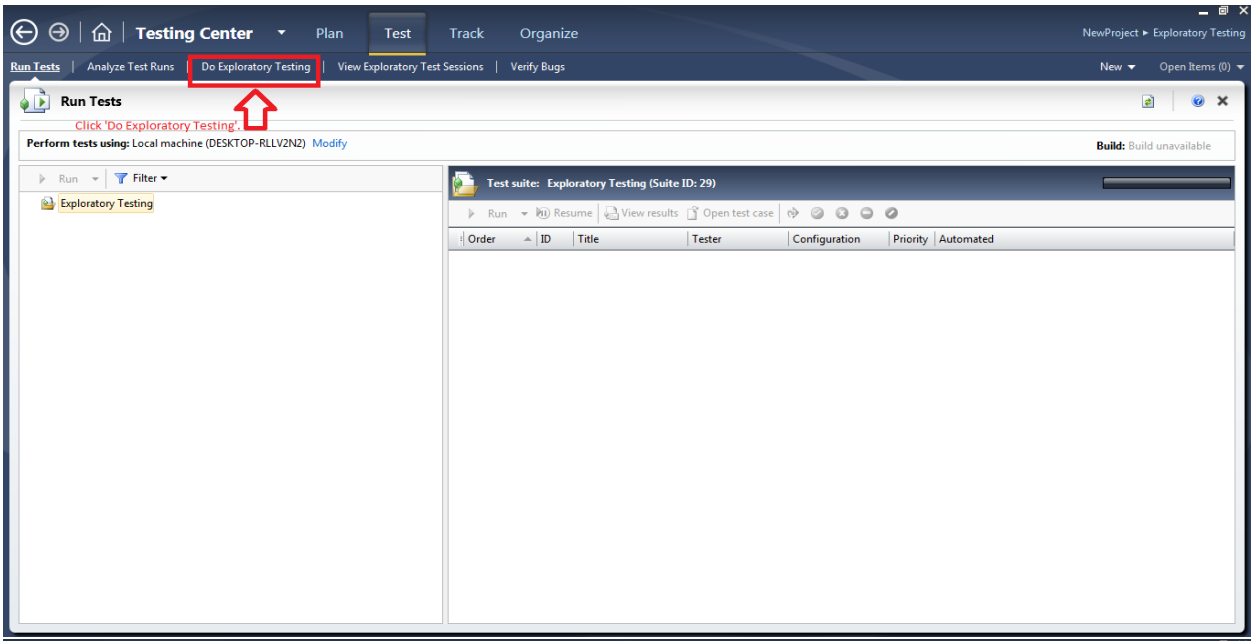
        - It helps to provide feedback to developers as soon as possible.
        - A larger number of bugs or defects can be uncovered using this way of testing.
- Benefits of Exploratory Testing:
    - Exploratory testing helps in exploring critical issues.
    - In the case of unstable requirement, ET helps in testing the new environment within a limited time.
    - It helps to promote Real-Time thinking and also helps to uncover more and more defects.
- Benefits of Exploratory Testing:
    - It promotes use cases as well as Scenario-Based testing.
    - It converts the minimal documentation process into maximum testing in a form because test cases are written by exploring an application.
- Drawbacks of Exploratory Testing:
    - Purely depends on tester skills.
    - Limited by domain knowledge of the tester.
    - Not suitable for Long execution time.
    - Some important defects may be missed.
    - Difficulties repeating the tests.
    - can sometimes test what was implemented not what was supposed to be implemented.

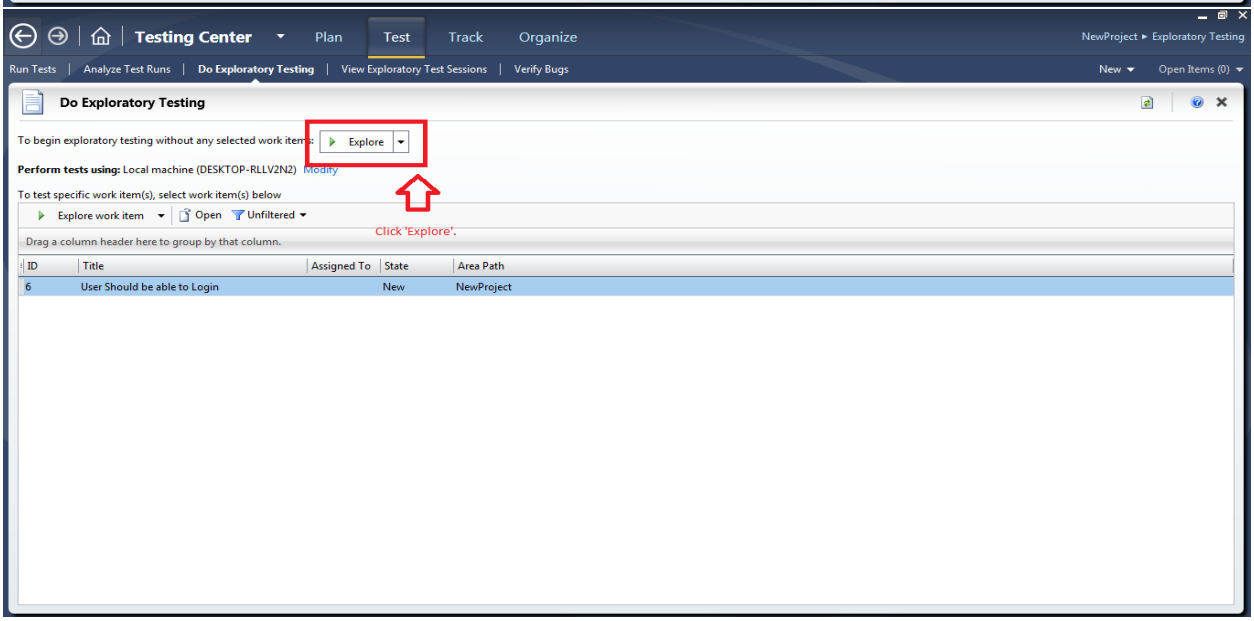
Module 140:
Exploratory Testing: Worked Example

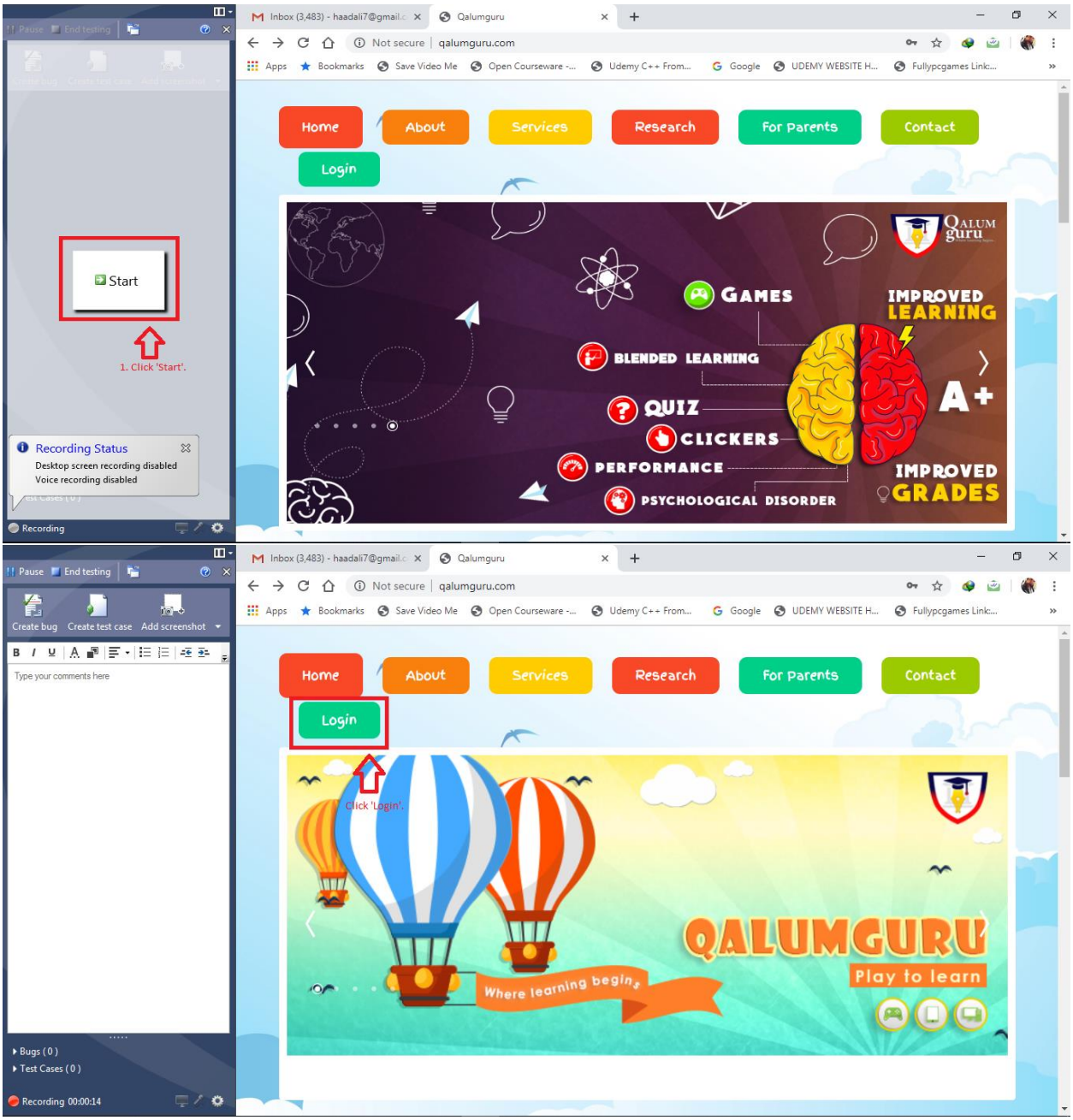
- We use Microsoft Test Manager (MTM)
- We have already used this previously to explain system testing
- We use the same application for explaining exploratory testing

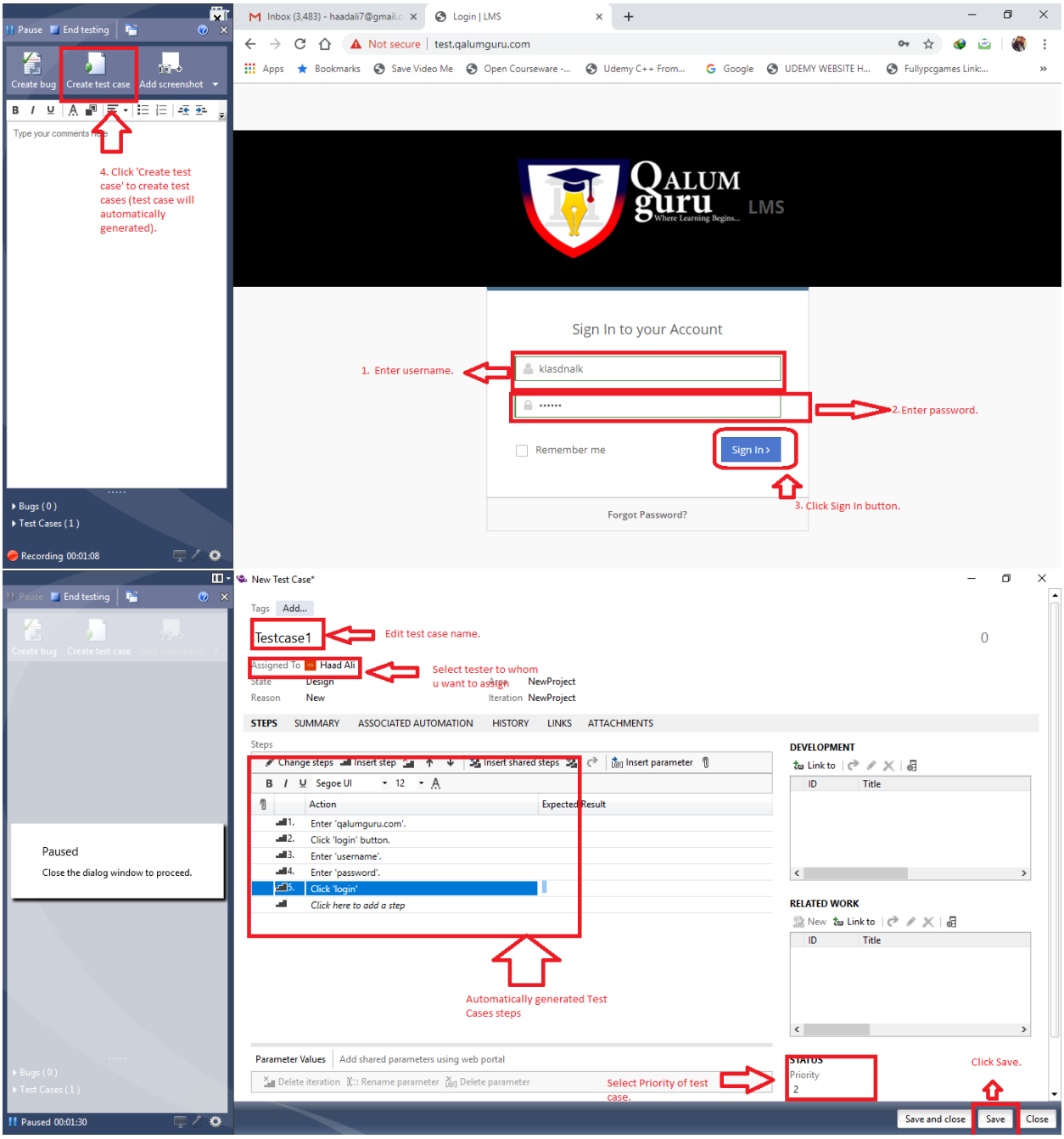

Testing Center
Plan
Test
Track
Organize
NewProject ▸ Exploratory Testing
Contents
Results
Properties
Run Settings
New
Open Items (0)
Contents
New
Add requirements
Open in Web Access
Exploratory Testing
Test suite: Exploratory Testing (Suite ID: 29)
Default configurations (1): Windows 10
State: In Progress
Open
Add
New
Assign
Configurations
Order
Drag a column header here to group by that column.
Order
ID
Title
Priority
Confi...
Testers
Area Path

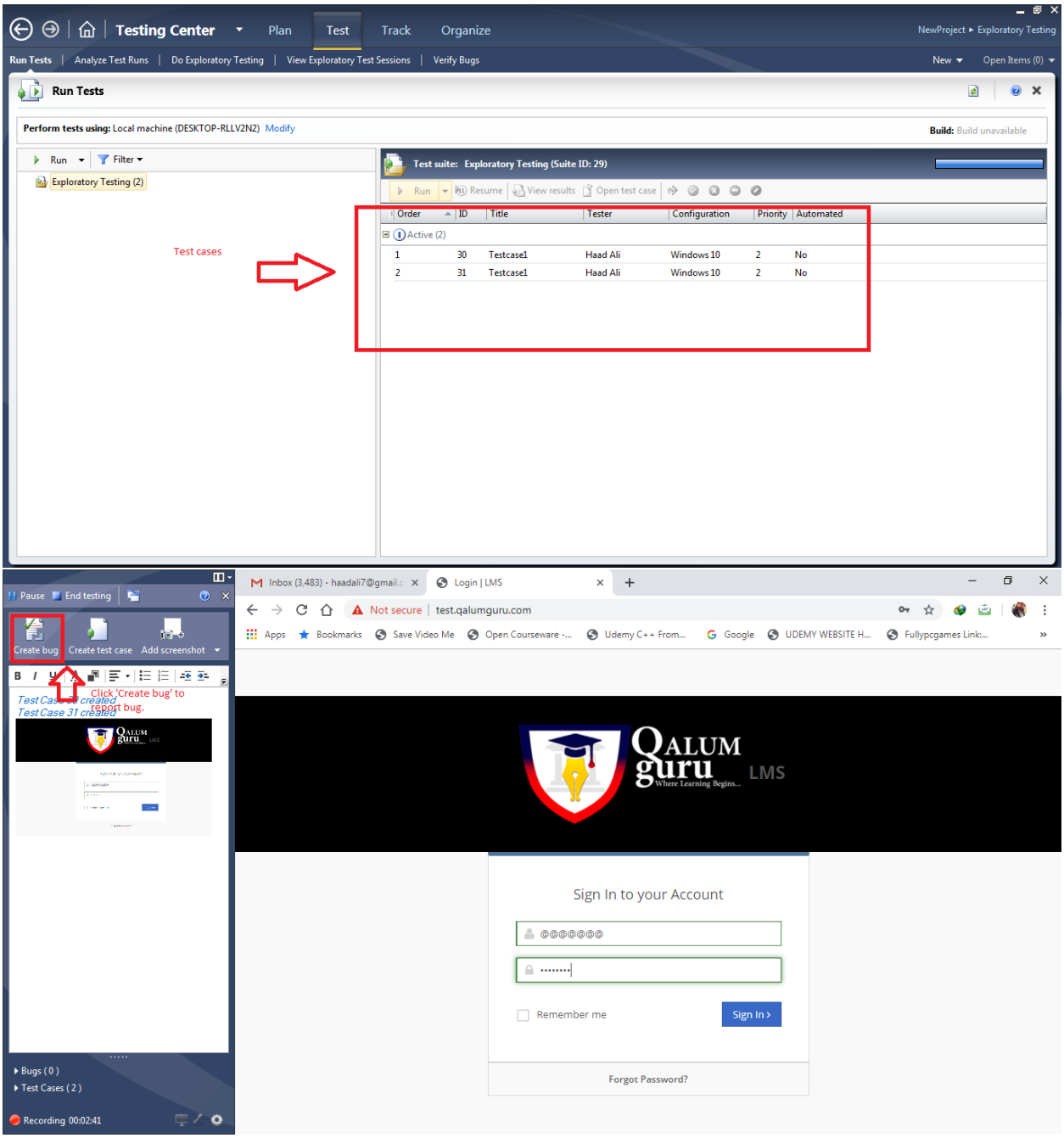

Testing Center
Plan
Test
Track
Organize
NewProject ▸ Exploratory Testing
Contents
Results
Properties
Run Settings
New
Open Items (0)
Contents
Click 'Test'.
New
Add requirements
Open in Web Access
Exploratory Testing
Test suite: Exploratory Testing (Suite ID: 29)
Default configurations (1): Windows 10
State: In Progress
Open
Add
New
Assign
Configurations
Order
Drag a column header here to group by that column.
Order
ID
Title
Priority
Confi...
Testers
Area Path

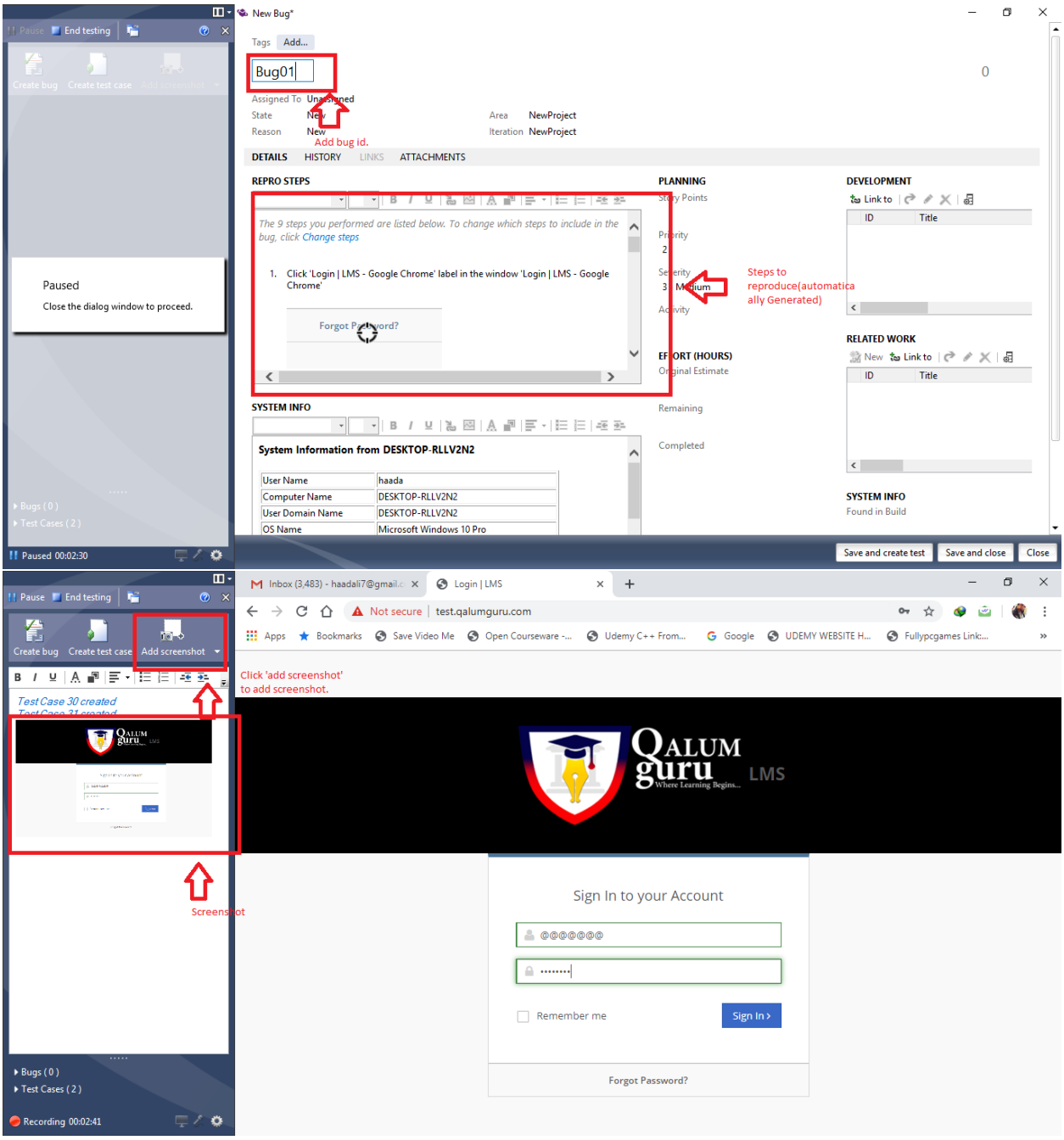

Testing Center
Plan
Test
Track
Organize
NewProject ▸ Exploratory Testing
Run Tests
Analyze Test Runs
Do Exploratory Testing
View Exploratory Test Sessions
Verify Bugs
New
Open Items (0)
Run Tests
Click 'Do Exploratory Testing'.
Perform tests using: Local machine (DESKTOP-RLLV2N2) Modify
Build: Build unavailable
Run
Filter
Exploratory Testing
Test suite: Exploratory Testing (Suite ID: 29)
Run
Resume
View results
Open test case
Order
ID
Title
Tester
Configuration
Priority
Automated


Testing Center
Plan
Test
Track
Organize
NewProject ▸ Exploratory Testing
Run Tests
Analyze Test Runs
Do Exploratory Testing
View Exploratory Test Sessions
Verify Bugs
New
Open Items (0)
Do Exploratory Testing
To begin exploratory testing without any selected work items:
Explore
Perform tests using: Local machine (DESKTOP-RLLV2N2) Modify
To test specific work item(s), select work item(s) below
Explore work item
Open
Unfiltered
Click 'Explore'.
Drag a column header here to group by that column.
ID
Title
Assigned To
State
Area Path
6
User Should be able to Login
New
NewProject


Pause
End testing
Start
1. Click 'Start'.
Recording Status
Desktop screen recording disabled
Voice recording disabled
Recording
Inbox (3,483) - haadali7@gmail.c
Qalumguru
Not secure | qalumguru.com
Apps
Bookmarks
Save Video Me
Open Courseware -...
Udemy C++ From...
Google
UDEMY WEBSITE H...
Fullypcgames Link:...
Home
About
Services
Research
For Parents
Contact
Login
GAMES
IMPROVED LEARNING
BLENDED LEARNING
QUIZ
CLICKERS
PERFORMANCE
PSYCHOLOGICAL DISORDER
A+
IMPROVED GRADES
Create bug
Create test case
Add screenshot
Type your comments here
Bugs (0)
Test Cases (0)
Recording 00:00:14
Click 'Login'.
QALUMGURU
Play to learn
Where learning begins


Pause
End testing
Create bug
Create test case
Add screenshot
Type your comments here
4. Click 'Create test case' to create test cases (test case will automatically generated).
Bugs (0)
Test Cases (1)
Recording 00:01:08
Inbox (3,483) - haadali7@gmail.c
Login | LMS
Not secure
test.qalumguru.com
Apps
Bookmarks
Save Video Me
Open Courseware -...
Udemy C++ From...
Google
UDEMY WEBSITE H...
Fullypcgames Link:...
QALUM guru
Where Learning Begins...
LMS
Sign In to your Account
1. Enter username.
klasdnalk
2.Enter password.
Remember me
Sign In ›
3. Click Sign In button.
Forgot Password?
Paused
Close the dialog window to proceed.
Paused 00:01:30
New Test Case*
Tags
Add...
Testcase1
Edit test case name.
Assigned To
Haad Ali
Select tester to whom u want to assign
State
Design
Reason
New
Area
NewProject
Iteration
NewProject
STEPS
SUMMARY
ASSOCIATED AUTOMATION
HISTORY
LINKS
ATTACHMENTS
Steps
Change steps
Insert step
Insert shared steps
Insert parameter
Segoe UI
12
Action
Expected Result
1. Enter 'qalumguru.com'.
2. Click 'login' button.
3. Enter 'username'.
4. Enter 'password'.
5. Click 'login'
Click here to add a step
Automatically generated Test Cases steps
DEVELOPMENT
Link to
ID
Title
RELATED WORK
New
Link to
ID
Title
Parameter Values
Add shared parameters using web portal
Delete iteration
Rename parameter
Delete parameter
Select Priority of test case.
STATUS
Priority
2
Click Save.
Save and close
Save
Close


Testing Center
Plan
Test
Track
Organize
NewProject ▸ Exploratory Testing
Run Tests | Analyze Test Runs | Do Exploratory Testing | View Exploratory Test Sessions | Verify Bugs
New
Open Items (0)
Run Tests
Perform tests using: Local machine (DESKTOP-RLLV2N2) Modify
Build: Build unavailable
Run
Filter
Exploratory Testing (2)
Test cases
Test suite: Exploratory Testing (Suite ID: 29)
Run
Resume
View results
Open test case
Order
ID
Title
Tester
Configuration
Priority
Automated
Active (2)
1 30 Testcase1 Haad Ali Windows 10 2 No
2 31 Testcase1 Haad Ali Windows 10 2 No
Pause
End testing
Create bug
Create test case
Add screenshot
Click 'Create bug' to report bug.
Test Case 30 created
Test Case 31 created
Bugs (0)
Test Cases (2)
Recording 00:02:41
Inbox (3,483) - haadali7@gmail.c
Login | LMS
Not secure
test.qalumguru.com
Apps
Bookmarks
Save Video Me
Open Courseware -...
Udemy C++ From...
Google
UDEMY WEBSITE H...
Fullypcgames Link:...
QALUM guru
Where Learning Begins...
LMS
Sign In to your Account
@@@@@@@
Remember me
Sign In ›
Forgot Password?


Pause
End testing
Create bug
Create test case
Add screenshot
Paused
Close the dialog window to proceed.
Bugs (0)
Test Cases (2)
Paused 00:02:30
New Bug*
Tags
Add...
Bug01
Assigned To
State
New
Reason
New
Add bug id.
Area
NewProject
Iteration
NewProject
DETAILS
HISTORY
LINKS
ATTACHMENTS
REPRO STEPS
The 9 steps you performed are listed below. To change which steps to include in the bug, click Change steps
1. Click 'Login | LMS - Google Chrome' label in the window 'Login | LMS - Google Chrome'
Forgot Password?
SYSTEM INFO
System Information from DESKTOP-RLLV2N2
User Name
haada
Computer Name
DESKTOP-RLLV2N2
User Domain Name
DESKTOP-RLLV2N2
OS Name
Microsoft Windows 10 Pro
PLANNING
Story Points
Priority
2
Severity
3 - Medium
Activity
Steps to reproduce(automatically Generated)
EFFORT (HOURS)
Original Estimate
Remaining
Completed
DEVELOPMENT
Link to
ID
Title
RELATED WORK
New
Link to
ID
Title
SYSTEM INFO
Found in Build
Save and create test
Save and close
Close
Create bug
Create test case
Add screenshot
Test Case 30 created
Test Case 31 created
Click 'add screenshot' to add screenshot.
Screenshot
Bugs (0)
Test Cases (2)
Recording 00:02:41
Inbox (3,483) - haadali7@gmail.c
Login | LMS
Not secure
test.qalumguru.com
Apps
Bookmarks
Save Video Me
Open Courseware -...
Udemy C++ From...
Google
UDEMY WEBSITE H...
Fullypcgames Link:...
QALUM guru
Where Learning Begins...
LMS
Sign In to your Account
@@@@@@@
Remember me
Sign In
Forgot Password?

Module 141:
Usability Testing

- Usability testing is an effort to ascertain the degree to which software has met usability needs of intended user base
- Usability Testing is an attempt to quantify software user-friendliness according to:
    1. Skill needed to learn the software
    2. Time required to become efficient in using software
    3. The measured increase in user productivity
    4. A subjective assessment of a user's attitude toward using the software
- It is a technique used to evaluate a product by testing it on users.
- test users will try to complete typical tasks while observers watch, listen and takes notes
- Our intent is to identify any usability problems
- ◦ Collect quantitative data on participants' performance
- ◦ Determine participant's satisfaction with the product
- Aesthetics and design are important. How well a product looks usually determines how well it works.
- There are many software applications/websites, which miserably fail, once launched, due to reasons such as:
    1. Where do I click next?
    2. Which page needs to be navigated?
    3. Which Icon or Jargon represents what?
    4. Error messages are not consistent or displayed
    5. Session time not sufficient

Usability Heuristics by J Neilson

1. Visibility of system status
2. Match between system and the real world
3. User control and freedom
4. Consistency and standards
5. Error prevention
6. Recognition rather than recall

Usability Heuristics by J Neilson

7. Flexibility and efficiency of use
8. Aesthetic and minimalistic design
9. Help users recognize, diagnose, and recover from errors
10. Help and documentation

1. Visibility of system status.

- Rationale: To make sure that users understand what information exists in the collection and how to find the information.
- Question examples
  - When searching, is it clear where you are?
  - How clearly does the web site describe contents?
  - Does formatting and organization of web site help understand where you are?

2: Match between the system and the real world

- Rationale
  - The digital library should have clear language and follow web site conventions.
- Question examples
  - Are the terms used throughout the SUT easy to understand?
  - Is it clear when icons are links?
  - Is it clear what the search and browse options are?

3: User Control and Freedom

- Rationale
  - The digital library web site should make it easy for users to return to the home page and to find pages within the site.
- Question examples
  - Can users easily move through search results?
  - The navigation was clearly visible. Users could easily find the 'back' and 'home' buttons.

- 4. Consistency and Standards
  - Rationale: Users should be able to clearly understand the format and metadata terminology should be understandable and consistent throughout.

- Question examples
    - Was the format consistent?
    - Does the site comply with conventions (such as how links look)?

5. Error prevention
    - Rationale: To prevent users from encountering errors or problems.
    - Question examples
        - Does the web site adequately prevent problems?
        - When no search results are found, interface presents user with different browsing options.

6. Recognition rather than recall
    - Rationale: The web site should not require users to remember information from previous pages to perform tasks.
    - Question examples
        - A menu or list is provided on each page highlighting the possible options available to users

7. Flexibility and Efficiency of Use
    - Rationale: Ease of searching and browsing methods ensures users will visit more than once.
    - Question examples
        - Users can easily browse the SUT.

8. Aesthetic / Minimalist Design
    - Rationale: Users should be able to view readable text and relevant information.
    - Question examples
        - SUT has a simple and clean design.
        - SUT uses readable fonts in colors that are easy read for all users.

9. Help Users Recognize, Diagnose, and Recover from Errors
    - Rationale: it is important to keep the user informed at all times
    - Question examples
        - The search results indicate when no results are found.
        - A message is displayed.

10. Help and Documentation

- Rationale: Help allows users to continue using SUT with limited knowledge and with online documentation.
- Question examples
  - If the documentation is available as a link
  - If the help is useful for making effective use of the SUT.

Module 142:
Usability Testing Process

- Usability testing process has the following steps:
  - Plan
  - Choose Participants
  - Perform Tests
  - Analyze Results
  - Document Recommendation

Plan and Prepare:

- Develop a test plan:
  - For simple testing, prepare a list of questions
  - For more detailed testing, have a script prepared

Create a Task List:

- Create lists of tasks or questions that a typical user should be able to complete in an hour
- Tasks should not be too simple nor too difficult to accomplish
  - e.g.,
  - Find a video lecture you want to study
  - Play educational game online
  - Start assessment

Find Participants

- A challenging aspect in usability testing is finding suitable participants
- Important to gather on ongoing user base
- Test outside the team—testing with people who are not associated with your company or your Web site

Conduct the Test

- User will be asked to perform a set of pre-defined tasks (but do not tell them how many or how long each will take)
- Make the user feel comfortable
- Speak only to give a new task and take notes during the process
- Analyse Results
    - Could the user complete the task?
    - Did they need help?
    - Track how much time it took them
    - Note any stumbling blocks (problems/obstacles)
    - Overall observations, commentary
    - Debrief the user, allow user to speak their mind

Examples of Usability Testing Test Cases:

- The goal of this testing is to satisfy users and it mainly concentrates on the following parameters of a system:

Example 1: Effectiveness:

- Is the system is easy to learn?
- Is the system useful and adds value to the target audience?
- Are Content, Color, Icons, Images used are aesthetically pleasing?

Examples of Usability Testing Test Cases:

Example 2: Efficiency

- Little navigation should be required to reach the desired screen or webpage, and scrollbars should be used infrequently.
- Uniformity in the format of screen/pages in your application/website.
- Option to search within your software application or website.

Example 3: Accuracy

- No outdated or incorrect data like contact information/address should be present.
- No broken links should be present.

Example 4: User Friendliness

- Controls used should be self-explanatory and must not require training to operate

- Help should be provided for the users to understand the application/website

Module 143:
Methods for Usability Testing

There are two methods available to do usability testing:

- Laboratory Usability Testing
- Remote Usability Testing

Laboratory Usability Testing:

- This testing is conducted in a separate lab room in presence of the observers.
- The testers are assigned tasks to execute.

Laboratory Usability Testing:

- The role of the observer is to monitor the behavior of the testers and report the outcome of testing.
- The observer remains silent during the course of testing.
- In this testing, both observers and testers are present in a same physical location.

Remote Usability Testing:

- Under this testing observers and testers are remotely located.
- Testers access the System Under Test, remotely and perform assigned tasks.
- Tester's voice , screen activity, testers facial expressions are recorded by an automated software.
- Observers analyze this data and report findings of the test.

Primary goal of this testing is to find crucial usability problems before the product is launched. Example checklist includes:

- Start the User experience or UX testing during early stage of design and development
- Select appropriate users to test system(Can be experts / non-experts users / 50-50 of Experts and Non-Experts users etc.

Module 144:
Benefits and Drawbacks of Usability Testing

- Benefits of Usability Testing include:
    - It helps uncover usability issues before the product is marketed.
    - It helps improve end-user satisfaction

        - It makes your system highly effective and efficient
        - It helps gather true feedback from your target audience who actually use your system during a usability test. You do not need to rely on "opinions" from random people.
- Drawbacks include:
    - Test results don't prove that a product/design/UI "works"
    - Maybe have a usability expert look at it, to find gross violations.
    - It's possible that a UI has an initial learning curve, but is then very powerful/usable. A usability test doesn't measure that.
    - Doesn't tell you if the market wants/needs a product like yours.

Module 145:
Usability Testing: Worked Example

- We first develop a checklist of questions following heuristics by J Neilson
- Checklist
    - Website:
        - What it is about?
        - What is benefit?
        - What is next?
    - Navigation:
        - Is your navigation clear?
    - Search:
        - Is search function available?
- Checklist
    - Layout:
        - Are buttons consistent?
        - Are Menus consistent?
    - Look and feel:
        - Is there enough white space?
        - Is information placed appropriately?
    - Help:
        - Is documentation / help available?

- We select testing method to be “Laboratory Usability Testing”
- Our application is Qalum guru which is available at http://www.qalumguru.com
- We start testing process and testers test where observes observe the process and report findings.
- Our findings against our checklist are:
- The website is about children basic education
- Navigation is easy and every page is connected to every page
- There is no search options
- Buttons are consistent
- Information is appropriately distributed amongst pages
- Online help is not available
- Documentation is not there

Module 146:
User Documentation Testing

- Software documentation was:
    - a readme.txt file copied onto the software’s floppy disk
    - a 1 page insert put into the shrink-wrapped package containing the software
    - comments in the source code!
    - Unix man pages are still in vogue, however …
- Testers ran a spell checker on the file and that was about the extent of testing documentation
- Much of the non-code is the software documentation, which requires much effort to produce.
- Documentation is now a major part of a software system.
    - It might exceed the amount of source code!
    - It might be integrated into the software (e.g., help system)
- Testers have to cover the code and the documentation.
    - Assuring documentation is correct is part of a software tester’s job.
- Documentation styles:
    - Loosely coupled to the code:

        - E.g., user manuals, packaging fliers.
    - Tightly coupled to the code:
        - E.g., documents are an integral part of the software, such as a training system, User manuals, CBTs etc.

Module 147:
Important Aspects of Documentation Testing

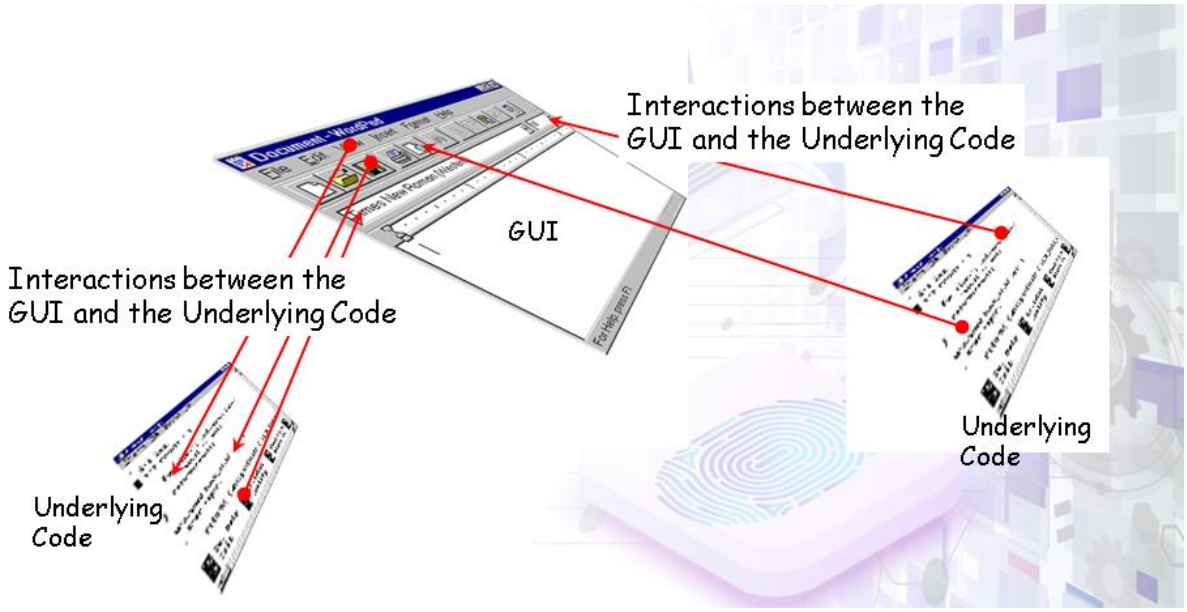
- Documentation aspects:
    - Packaging text and graphics
    - Marketing material, ads, and other inserts
    - Warranty/registration
    - End User License Agreement (EULA)
    - Labels and stickers
    - Installation and setup instructions
- Documentation aspects:
    - User's manual
    - Online help
    - Tutorials, wizards, and computer-based training
    - Samples, examples, and templates
    - Error messages
- Benefits of documentation testing include:
    - Improves usability
        - Not all software was written for the Mac :-)
    - Improves reliability
        - Testing the documentation is part of black-box testing.
        - A bug in the user manual is like a bug in the software.
    - Lowers support cost
        - The exercise of writing the documentation helped debug the system.

Module 148:
Documentation Testing Process

- We consider following aspects:
- Audience:
    - E.g., make sure documentation is not too novice or too advanced.
- Terminology:
    - Is it suitable for the audience?
    - Terms used consistently?
    - Abbreviations for acronyms?
- Content and subject matter:
    - Appropriate subjects covered?
    - No subjects missing?
    - Proper depth?
    - Missing features described accidentally?
- Just the facts:
    - All information technically correct?
    - Correct table of contents, index, chapter references?
    - Correct website URLs, phone numbers?
- Step by step:
    - Any missing steps?
    - Compared tester results to those shown in the documentation?
- Figures and screen captures:
    - Accurate and precise?
    - Are they from the latest version of the software?
    - Are the figure captions correct?
- Samples and examples:
    - Do all the examples work as advertised?
- Spelling and grammar

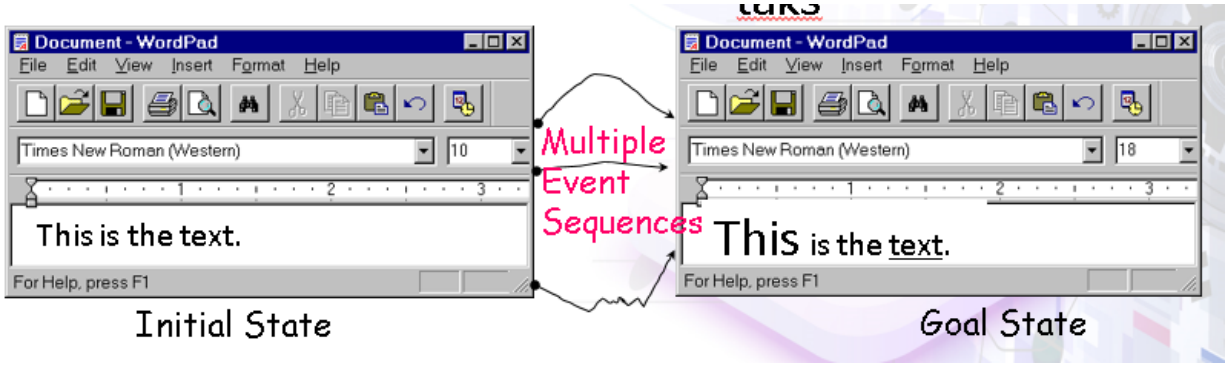
Module 149:
Graphical User Interface (GUI) Testing

- A GUI (Graphical User Interface) or simply UI is a hierarchical, graphical front end to a software system.
- GUI as a means to use/interact with software systems
- GUI are nowadays almost ubiquitous, even in safety critical systems
- Different types of device (web, pc, tablet, palm, mobile)
- GUI interacts with the underlying code by method calls or messages
- GUI testing is a process of verifying the GUI for its compliance with specifications, general principles, and specific project requirements.
- It involves simulating user actions like clicking buttons, following links, and other similar actions.
- In this way, developers check the product for its correct operation, mutual interaction of components, and the UI usability as a whole.
- GUI can exercise remote code
- GUI responds to user events (e.g., mouse clicks)
    - UIs are event-driven systems
- Testing GUI correctness is critical for system usability, robustness and safety
- The whole system can be executed by means of the GUI

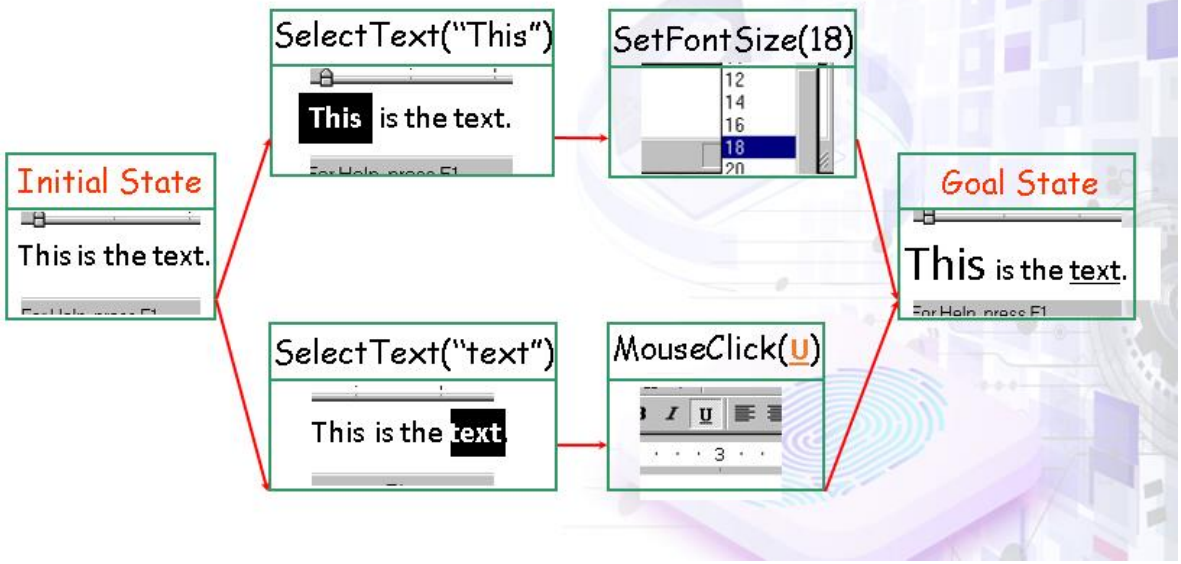


Module 150:
Graphical User Interface (GUI) Testing Challenges

- Testing correctness of a GUI is difficult for a number of reasons.
- First of all, space of possible interactions with a GUI is enormous
- The large number of possible states results in a large number of input permutations requiring extensive testing,
- Sequences can be infinitely many
- Options could be:
    - Randomly Choose Sequences
    - Expert Chooses Sequences
    - Automatically Generate Events for commonly used tasks
- Infinitely Many
- Randomly Choose Sequences
- Expert Chooses Sequences
- Automatically Generate Events for commonly used taks







- Problem of Coverage
  - For conventional software, coverage is measured using amount and type of underlying code exercised.
  - These measures do not hold good for GUI based testing.
- Another issue that we need to tackle is related to which level we are conducting the testing?
  - Unit
  - Integration
  - System

        - Regression
- How to choose inputs?
        - using specifications or use cases etc.
        - using code
        - only considering GUI (functionality, structure)
- How to identify the expected output?
        - test oracles
- How good test cases are?
        - when we can stop the testing activity
- GUIs are Event Driven
- Large space of possibilities
    - The user may click on any pixel on the screen
    - Even the simplest components have a large number of attributes and methods
        - JButton has more than 50 attributes and 200 methods
    - The state of the GUI is a combination of the states of all of its components
- Coverage:
    - Cannot use code-based coverage
        - Source code not always available
        - Event-based input
        - Different level of abstraction
- We define an event-based graph for coverage
- We call in Event-flow Graph..

Module 151:
GUI Testing Methods

- **Manual GUI Testing:**
    - GUI components are identified

- Manually tested by the tester and compared with the prototype screens or the test cases as prepared against the business requirement documents.

- **Record and Replay (Test Automation):**
  - We can automate the GUI testing with the help of tools such as QTP, Selenium, Sikuli, etc.

- **Model-based GUI testing:**

- A graphical description of the behaviour of the system is known as a Model.
  - A model helps us to determine system behaviour under test.
  - We use system requirements in order to generate the efficient test cases Develop a model.

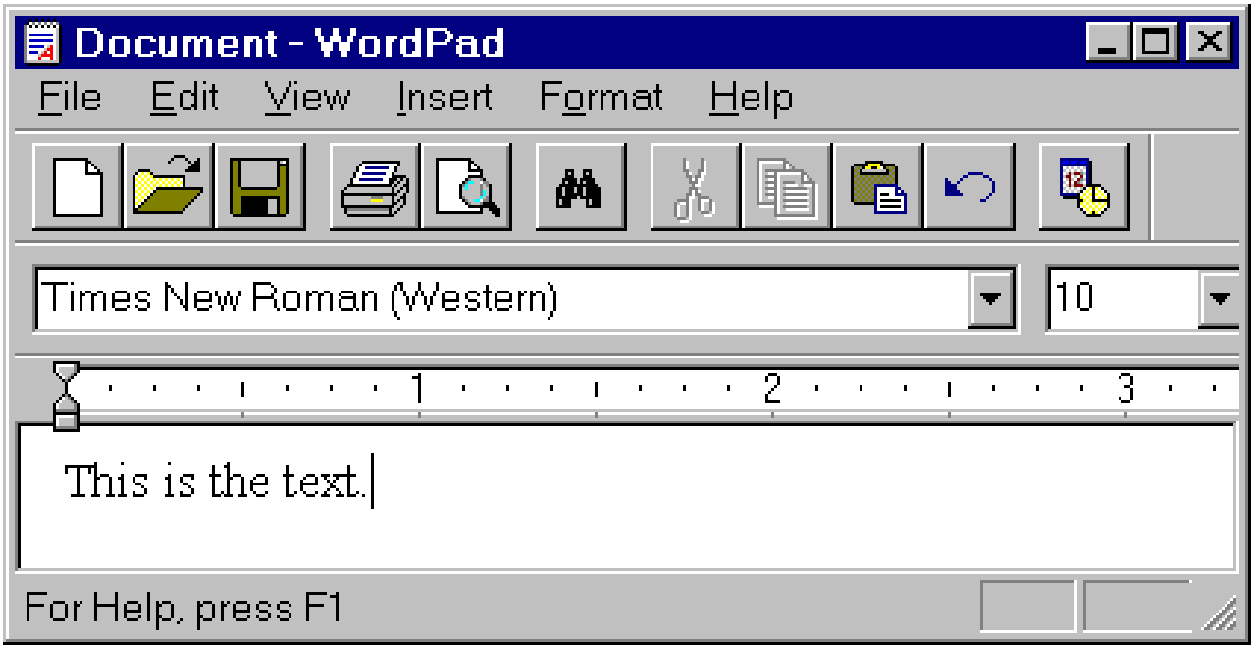
- Determine various inputs for this model.
  - Determine expected output for this model.
  - Execute the tests.
  - Compare the actual result against the expected output.
  - Make the decision on the action on the model.

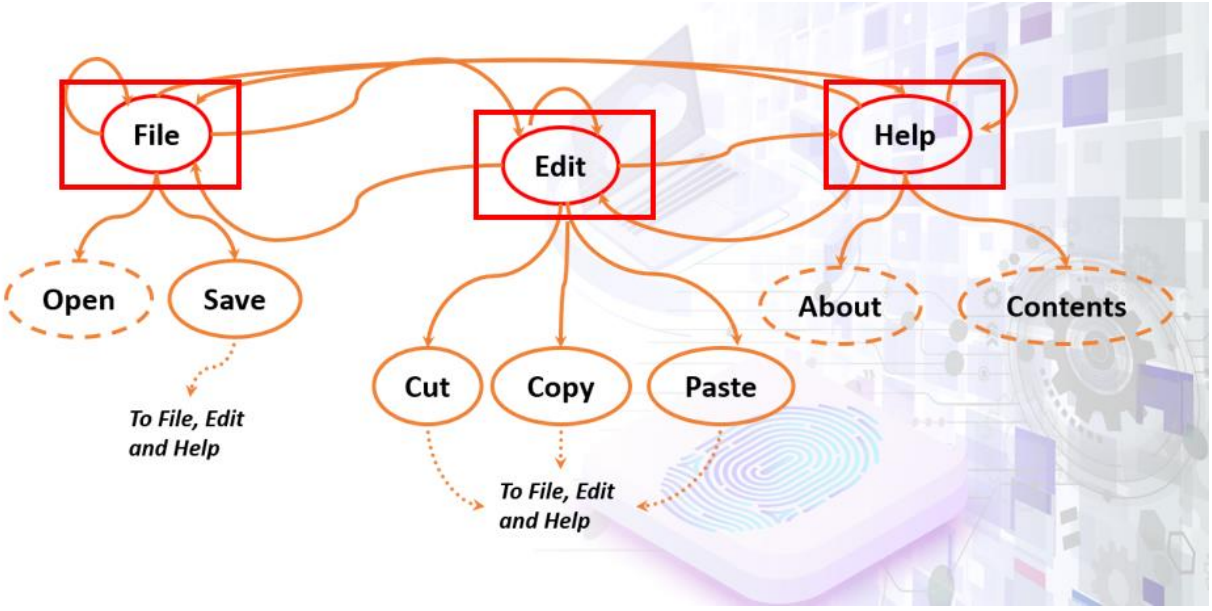
- Since GUI is event based
  - We develop model in terms of an event-flow graph

- Definition: Event-flow graph is a 4-tuple

- <V, E, B, I>

- V is the set of vertices, representing events,
- E is the set of directed edges, showing the follows relationship,
- B is the set of events first available (shown in red),
- I is the set of events that invoke other components (dotted lines).
- Considering our example of Word processing application in previous module
- We can find out:
  - Set of vertices
  - Set of edges
  - Initial events
  - Terminal events





- Approaches to GUI Testing:
  - Black box
    - Launch application
    - Simulate mouse and keyboard events
    - Compare final look to an existing screen dump

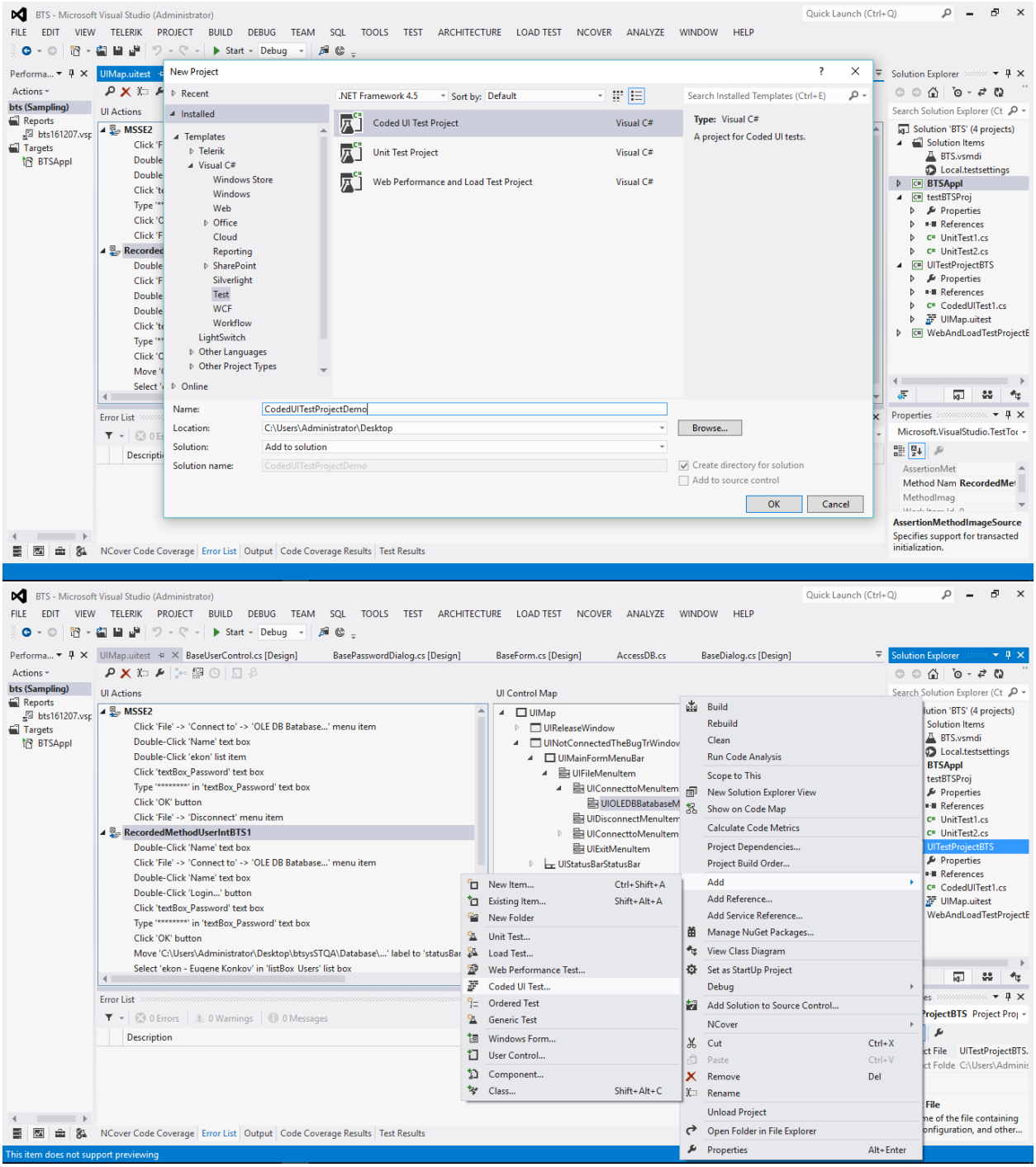
- Glass box
    - Launch application in the testing code
    - Obtain references to the various components and send events to them
    - Assert the state of components directly..

Module 152:
GUI Testing: Coverage

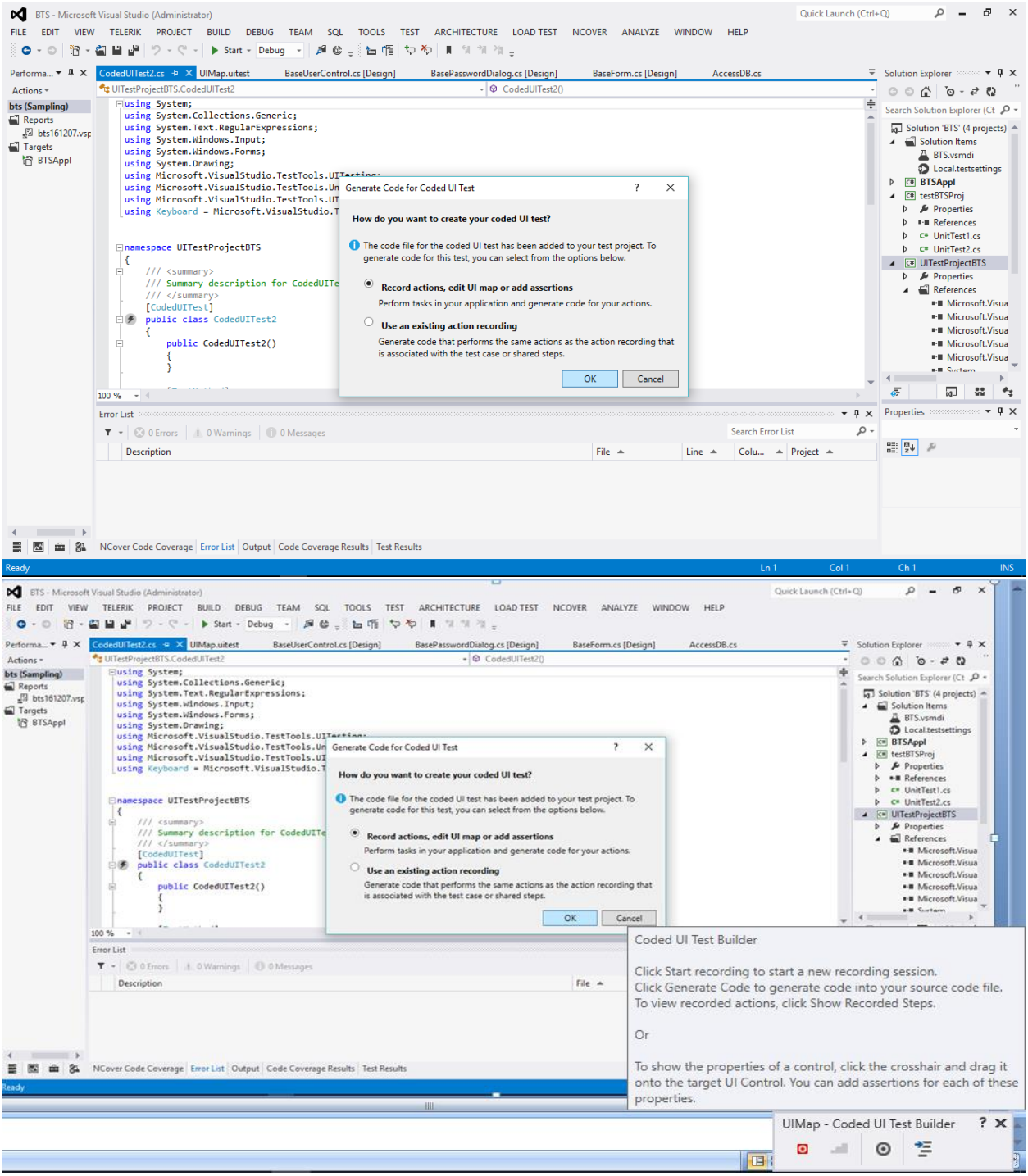
- Intra-component Coverage
    - Event coverage
        - Individual events
        - Each node in event-flow graph
    - Event-interaction coverage
        - Each pair of events
        - Each edge in event-flow graph
    - Length-n event sequence coverage
        - Sequences of events
        - Bounded by length
            - Length-1, Length-2, length-6 event sequences
        - Paths in event-flow graph
- Inter-component Coverage
    - Invocation coverage
        - Invoke each component
        - Each restricted-focus event
    - Invocation-termination coverage
        - Invoke each component and terminate it
        - Restricted-focus event followed by a termination event
    - Inter-component length-n coverage
        - Longer sequences from one component to another
        - Bounded by length..

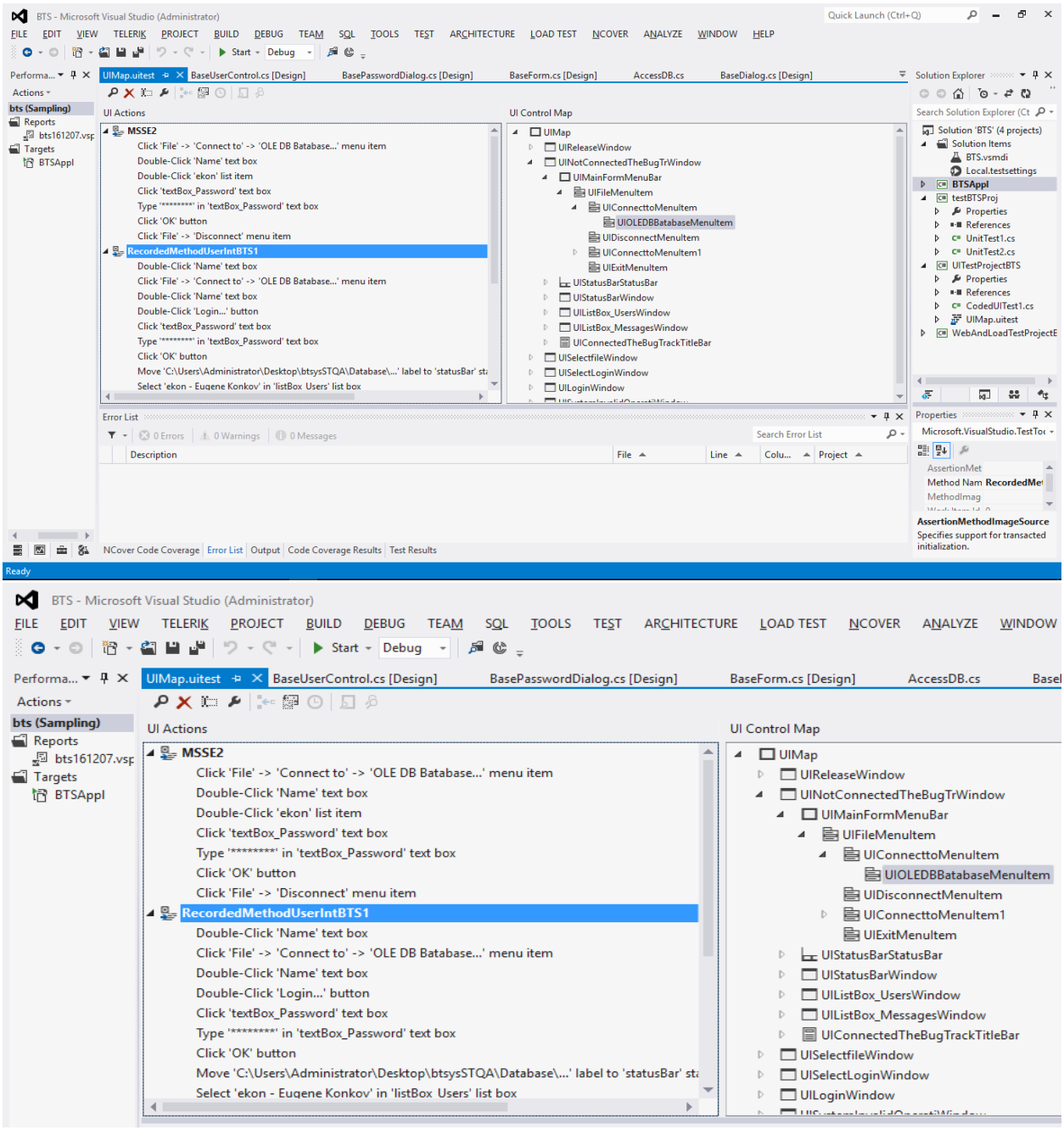
Module 153:
GUI Testing: Worked Example

- We are now writing GUI testing based test case for a client-server / desktop application
- We have already identified that we can write GUI based test cases using a variety of applications such as:
    - Selenium
    - Sikuli
    - UFT
    - Silk Test
    - TestComplete
- We use Microsoft Visual Studio

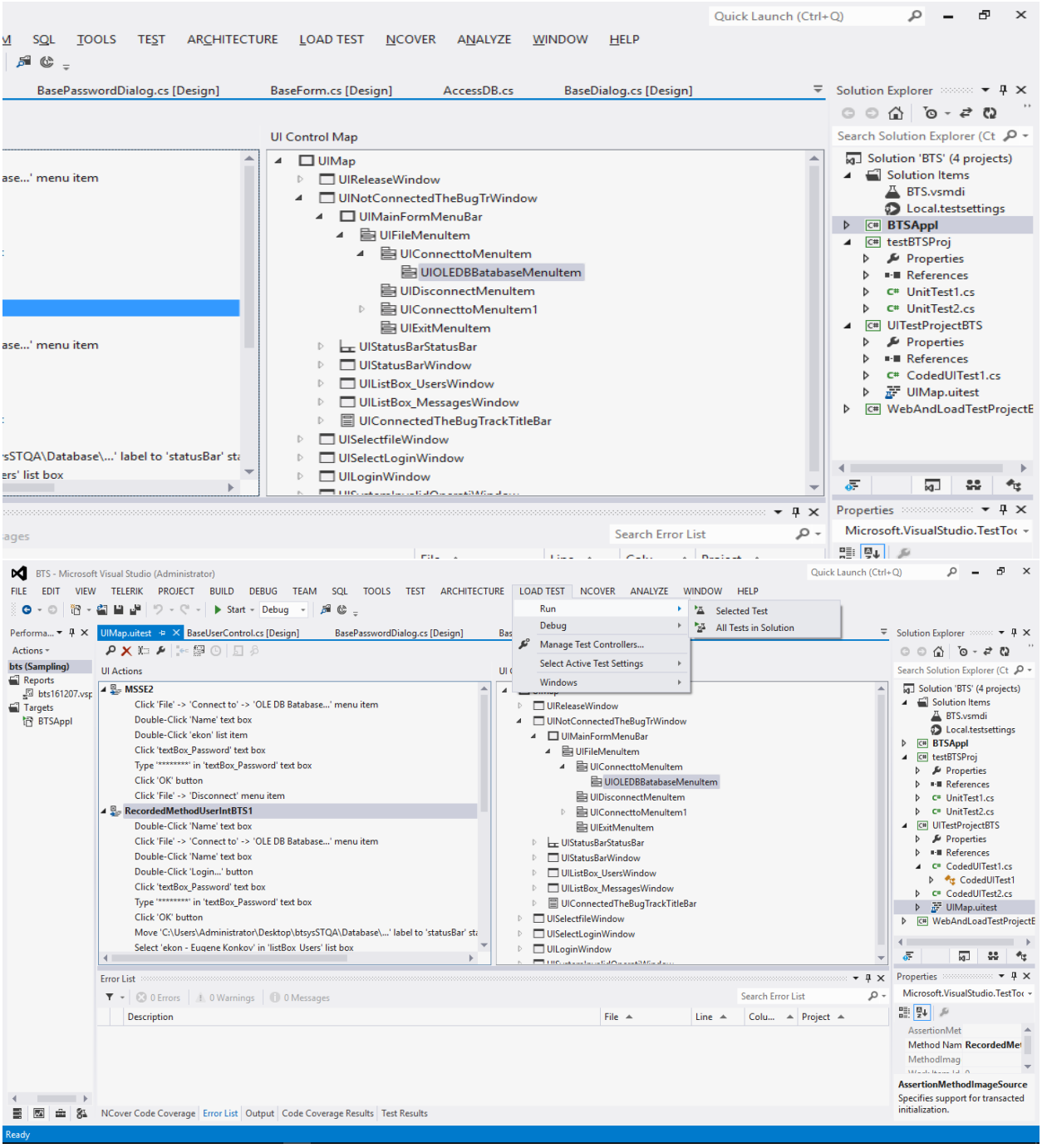

BTS - Microsoft Visual Studio (Administrator)
New Project
Recent
Installed
Templates
Telerik
Visual C#
Windows Store
Windows
Web
Office
Cloud
Reporting
SharePoint
Silverlight
Test
WCF
Workflow
LightSwitch
Other Languages
Other Project Types
Online
.NET Framework 4.5
Sort by: Default
Coded UI Test Project
Visual C#
Unit Test Project
Visual C#
Web Performance and Load Test Project
Visual C#
Type: Visual C#
A project for Coded UI tests.
Name: CodedUITestProjectDemo
Location: C:\Users\Administrator\Desktop
Solution: Add to solution
Solution name: CodedUITestProjectDemo
Browse...
Create directory for solution
Add to source control
OK
Cancel
Solution Explorer
Properties
Error List
NCover Code Coverage
Error List
Output
Code Coverage Results
Test Results

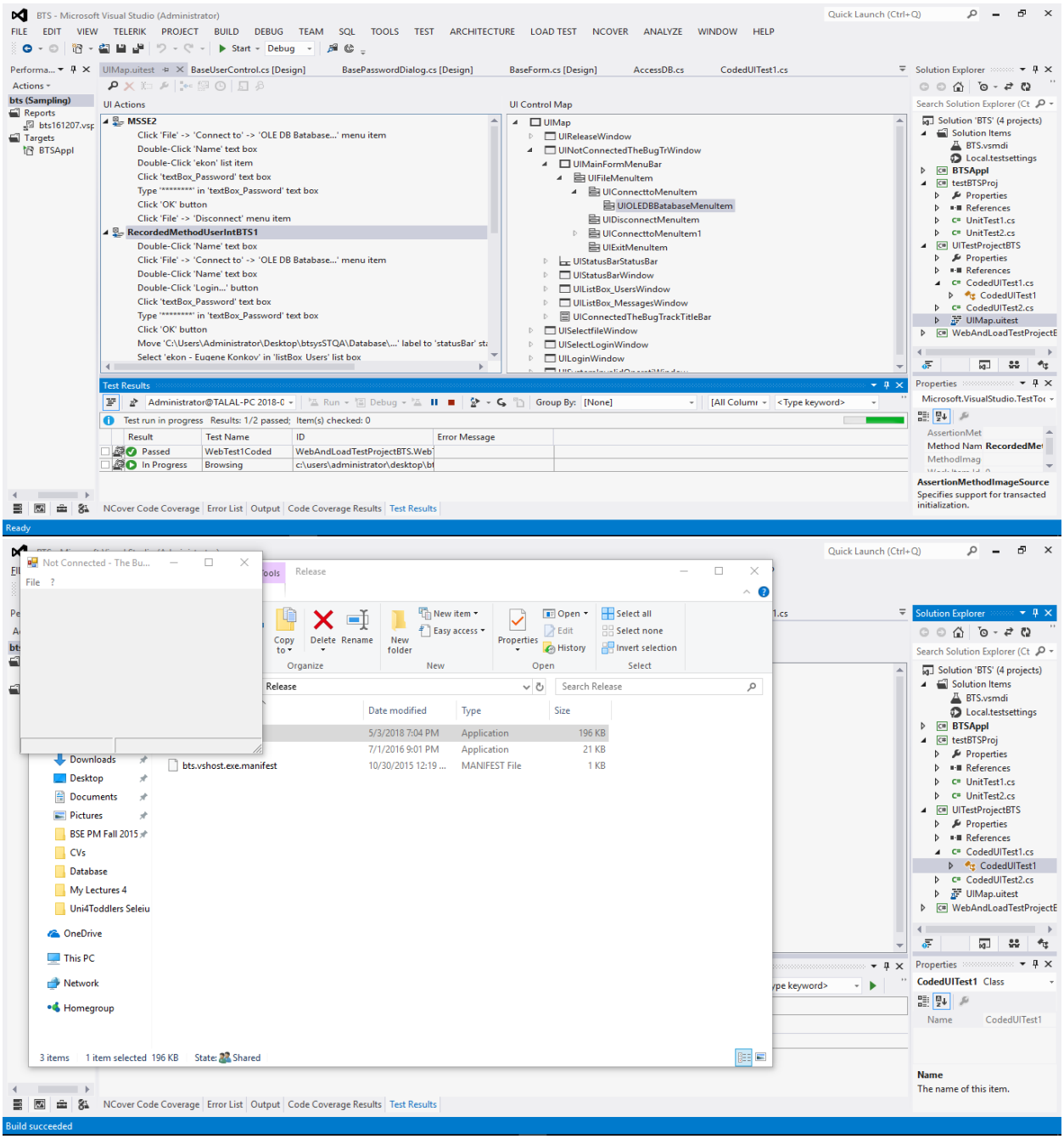
BTS - Microsoft Visual Studio (Administrator)
UIMap.uitest
BaseUserControl.cs [Design]
BasePasswordDialog.cs [Design]
BaseForm.cs [Design]
AccessDB.cs
BaseDialog.cs [Design]
UI Actions
UI Control Map
MSSE2
Click 'File' -> 'Connect to' -> 'OLE DB Batabase...' menu item
Double-Click 'Name' text box
Double-Click 'ekon' list item
Click 'textBox_Password' text box
Type '*********' in 'textBox_Password' text box
Click 'OK' button
Click 'File' -> 'Disconnect' menu item
RecordedMethodUserIntBTS1
Double-Click 'Name' text box
Click 'File' -> 'Connect to' -> 'OLE DB Batabase...' menu item
Double-Click 'Name' text box
Double-Click 'Login...' button
Click 'textBox_Password' text box
Type '*********' in 'textBox_Password' text box
Click 'OK' button
Move 'C:\Users\Administrator\Desktop\btsysSTQA\Database\...' label to 'statusBar
Select 'ekon - Eugene Konkov' in 'listBox Users' list box
UIMap
UIReleaseWindow
UINotConnectedTheBugTrWindow
UIMainFormMenuBar
UIFileMenuItem
UIConnecttoMenuItem
UIOLEDBBatabaseM
UIDisconnectMenuItem
UIConnecttoMenuItem
UIExitMenuItem
UIStatusBarStatusBar
New Item... Ctrl+Shift+A
Existing Item... Shift+Alt+A
New Folder
Unit Test...
Load Test...
Web Performance Test...
Coded UI Test...
Ordered Test
Generic Test
Windows Form...
User Control...
Component...
Class... Shift+Alt+C
Build
Rebuild
Clean
Run Code Analysis
Scope to This
New Solution Explorer View
Show on Code Map
Calculate Code Metrics
Project Dependencies...
Project Build Order...
Add
Add Reference...
Add Service Reference...
Manage NuGet Packages...
View Class Diagram
Set as StartUp Project
Debug
Add Solution to Source Control...
NCover
Cut Ctrl+X
Paste Ctrl+V
Remove Del
Rename
Unload Project
Open Folder in File Explorer
Properties Alt+Enter
Solution Explorer
Error List
0 Errors
0 Warnings
0 Messages
Description
NCover Code Coverage
Error List
Output
Code Coverage Results
Test Results
This item does not support previewing


BTS - Microsoft Visual Studio (Administrator)
Generate Code for Coded UI Test
How do you want to create your coded UI test?
The code file for the coded UI test has been added to your test project. To generate code for this test, you can select from the options below.
Record actions, edit UI map or add assertions
Perform tasks in your application and generate code for your actions.
Use an existing action recording
Generate code that performs the same actions as the action recording that is associated with the test case or shared steps.
OK
Cancel
Coded UI Test Builder
Click Start recording to start a new recording session.
Click Generate Code to generate code into your source code file.
To view recorded actions, click Show Recorded Steps.
Or
To show the properties of a control, click the crosshair and drag it onto the target UI Control. You can add assertions for each of these properties.
UIMap - Coded UI Test Builder



BTS - Microsoft Visual Studio (Administrator)
FILE EDIT VIEW TELERIK PROJECT BUILD DEBUG TEAM SQL TOOLS TEST ARCHITECTURE LOAD TEST NCOVER ANALYZE WINDOW
Start Debug
UIMap.uitest BaseUserControl.cs [Design] BasePasswordDialog.cs [Design] BaseForm.cs [Design] AccessDB.cs
Actions
bts (Sampling)
Reports
bts161207.vsp
Targets
BTSAppl
UI Actions
MSSE2
Click 'File' -> 'Connect to' -> 'OLE DB Batabase...' menu item
Double-Click 'Name' text box
Double-Click 'ekon' list item
Click 'textBox_Password' text box
Type '*********' in 'textBox_Password' text box
Click 'OK' button
Click 'File' -> 'Disconnect' menu item
RecordedMethodUserIntBTS1
Double-Click 'Name' text box
Click 'File' -> 'Connect to' -> 'OLE DB Batabase...' menu item
Double-Click 'Name' text box
Double-Click 'Login...' button
Click 'textBox_Password' text box
Type '*********' in 'textBox_Password' text box
Click 'OK' button
Move 'C:\Users\Administrator\Desktop\btsysSTQA\Database\...' label to 'statusBar' sta
Select 'ekon - Eugene Konkov' in 'listBox_Users' list box
UI Control Map
UIMap
UIReleaseWindow
UINotConnectedTheBugTrWindow
UIMainFormMenuBar
UIFileMenuItem
UIConnecttoMenuItem
UIOLEDBBatabaseMenuItem
UIDisconnectMenuItem
UIConnecttoMenuItem1
UIExitMenuItem
UIStatusBarStatusBar
UIStatusBarWindow
UIListBox_UsersWindow
UIListBox_MessagesWindow
UIConnectedTheBugTrackTitleBar
UISelectfileWindow
UISelectLoginWindow
UILoginWindow


Quick Launch (Ctrl+Q)
SQL TOOLS TEST ARCHITECTURE LOAD TEST NCOVER ANALYZE WINDOW HELP
BasePasswordDialog.cs [Design]
BaseForm.cs [Design]
AccessDB.cs
BaseDialog.cs [Design]
UI Control Map
UIMap
UIReleaseWindow
UINotConnectedTheBugTrWindow
UIMainFormMenuBar
UIFileMenuItem
UIConnecttoMenuItem
UIOLEDBBatabaseMenuItem
UIDisconnectMenuItem
UIConnecttoMenuItem1
UIExitMenuItem
UIStatusBarStatusBar
UIStatusBarWindow
UIListBox_UsersWindow
UIListBox_MessagesWindow
UIConnectedTheBugTrackTitleBar
UISelectfileWindow
UISelectLoginWindow
UILoginWindow
Solution Explorer
Search Solution Explorer (Ct
Solution 'BTS' (4 projects)
Solution Items
BTS.vsmdi
Local.testsettings
BTSAppl
testBTSProj
Properties
References
UnitTest1.cs
UnitTest2.cs
UITestProjectBTS
CodedUITest1.cs
UIMap.uitest
WebAndLoadTestProjectE
Properties
Microsoft.VisualStudio.TestTo
Search Error List
BTS - Microsoft Visual Studio (Administrator)
FILE EDIT VIEW TELERIK PROJECT BUILD DEBUG TEAM SQL TOOLS TEST ARCHITECTURE LOAD TEST NCOVER ANALYZE WINDOW HELP
Run
Debug
Manage Test Controllers...
Select Active Test Settings
Windows
Selected Test
All Tests in Solution
UIMap.uitest
BaseUserControl.cs [Design]
BasePasswordDialog.cs [Design]
UI Actions
MSSE2
Click 'File' -> 'Connect to' -> 'OLE DB Batabase...' menu item
Double-Click 'Name' text box
Double-Click 'ekon' list item
Click 'textBox_Password' text box
Type '*********' in 'textBox_Password' text box
Click 'OK' button
Click 'File' -> 'Disconnect' menu item
RecordedMethodUserIntBTS1
Double-Click 'Name' text box
Click 'File' -> 'Connect to' -> 'OLE DB Batabase...' menu item
Double-Click 'Name' text box
Double-Click 'Login...' button
Click 'textBox_Password' text box
Type '*********' in 'textBox_Password' text box
Click 'OK' button
Move 'C:\Users\Administrator\Desktop\btsysSTQA\Database\...' label to 'statusBar' sta
Select 'ekon - Eugene Konkov' in 'listBox_Users' list box
Error List
0 Errors
0 Warnings
0 Messages
Description
File
Line
Colu...
Project
NCover Code Coverage
Error List
Output
Code Coverage Results
Test Results
Ready



Module 154:
Acceptance Testing

- Acceptance testing is testing with respect to:
    - user needs,
    - requirements,
    - and business processes
    - conducted to determine whether or not a system satisfies the acceptance criteria and

    - to enable the user, customers or other authorized entity to determine whether or not to accept the system.

- Acceptance testing is a formal testing conducted to determine whether a system satisfies its acceptance criteria
    - User Acceptance Testing (UAT)
        - It is conducted by the customer to ensure that system satisfies the contractual acceptance criteria before being signed-off as meeting user needs.
    - Business Acceptance Testing (BAT)
        - It is undertaken within the development organization of the supplier to ensure that the system will eventually pass the user acceptance testing.
        - Also known as operational acceptance testing
    - Contract and regulation acceptance testing
    - Alpha / Beta (Field) Testing
- Objectives:
    - Confirm that the system meets the agreed upon criteria
    - Identify and resolve discrepancies, if there is any
    - Determine the readiness of the system for cut-over to live operations
- The acceptance criteria defined on the basis:
    - Functional Correctness and Completeness
    - Backup and Recovery
    - Usability, Performance
    - Start-up Time,
    - Stress / Load / Volume
    - Reliability and Availability
    - Maintainability and Serviceability
    - Robustness
    - Timeliness, Compliance
    - Documentation
    - etc.
- The acceptance criteria discussed are too many and very general

- The customer needs to select a subset of the quality attributes
- The quality attributes are prioritize them to specific situation
- Involvement of inhouse QA Team:
- No Involvement
- Assist in this testing
- Perform UAT and present Results..

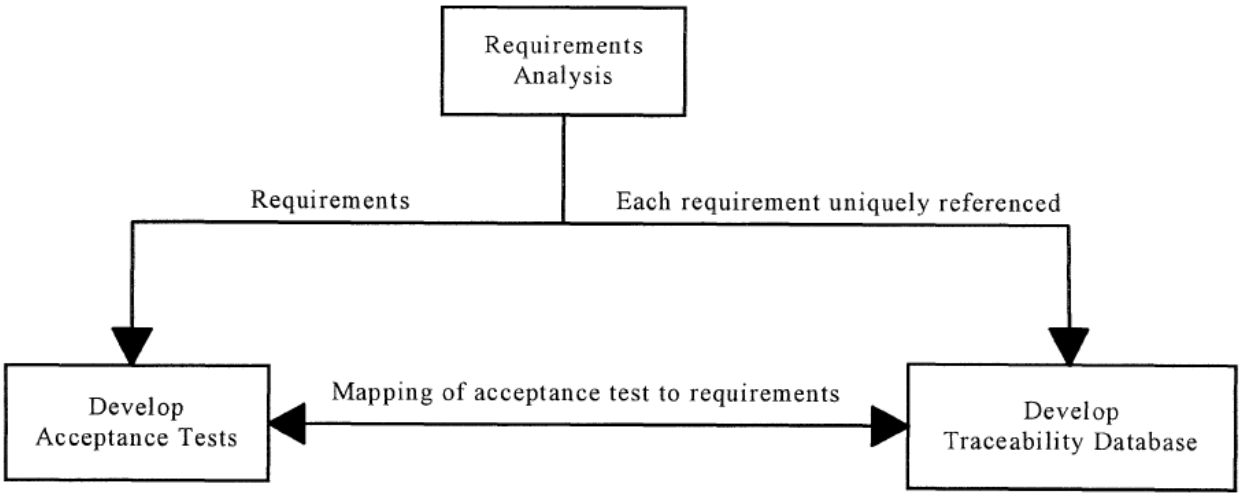
Module 155:
Acceptance Testing Phases

- The acceptance test cases phases:
    - Phase 1: execute basic test cases
        - If the test results are satisfactory then the second phase, in which the complex test cases are executed, is taken up.
    - Phase 2: complex scenarios or test cases
        - a subset of the system-level test cases are executed to confirm test results independently
- Acceptance test execution activities:
    - Train customer (representative) on usage of system
    - Development team and customer representative team work on a predefined exit criteria and execute test cases.
    - Issues are discussed and resolved
- Question: How to set predefined exit criteria
- Prerequisites for Acceptance Testing
    - Business Requirements must be available.
    - Application Code should be fully developed
    - Unit Testing, Integration Testing & System Testing should be completed
    - No Showstoppers, High, Medium defects in System Integration Test Phase -
    - Only Cosmetic error is acceptable before UAT
- Prerequisites for Acceptance Testing
    - Regression Testing should be completed with no major defects
    - All the reported defects should be fixed and tested before UAT

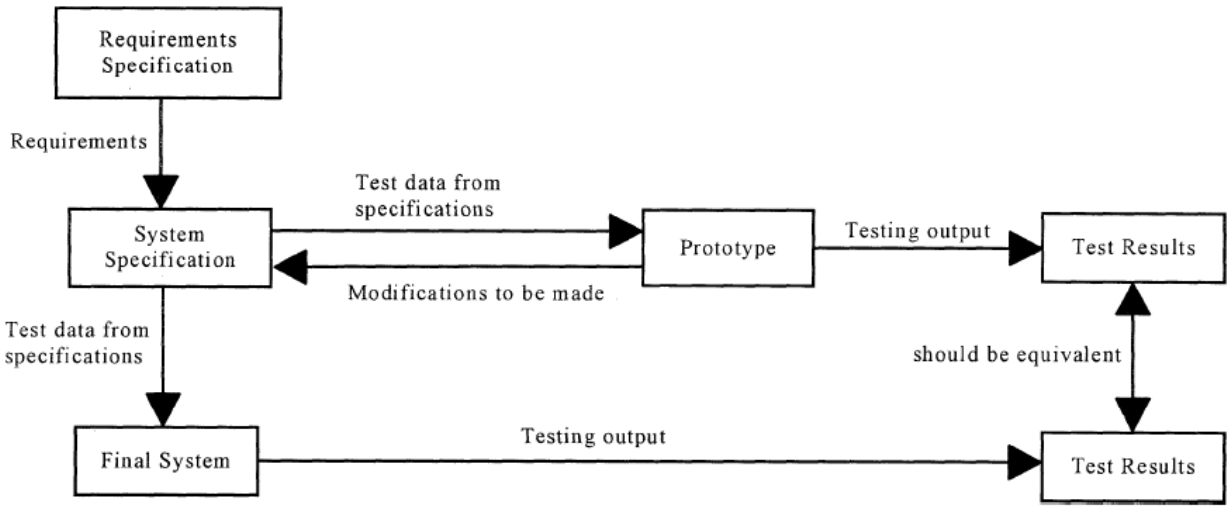
- Traceability matrix for all testing should be completed
- UAT Environment be ready
- communication from System Testing Team that system is ready for UAT..

Module 156:
Acceptance Testing Exit Criteria

- There are a number of exit criteria:
- To name a few:
  - Traceability method
  - Formal method
  - Prototyping method
  - etc
- Traceability matrix:
  - Work by creating a cross reference between requirements and acceptance test cases
  - Fabricating relationships between test cases and requirements.
- A straightforward way to match the test cases to the requirements:
  - allow testers to easily check the requirements not yet tested.
  - show the customer which requirement is being exercised by which test case(s).
- However, each cross reference is created by the tester subjectively
- Requirements in SRS are tagged:
  - We have identifiers against each requirement, Use-case, description, user-story, sprint, etc.
  - This is already done if we have requirement ID in our test case specification
- Use a database to automate traceability task
  - A matrix or checklist ensures there is a test covering each requirement
  - Keep track of when tests completed/failed.



- Apply mathematical modeling to test case generation.
- They include:
  - application of formal languages,
  - finite state machines,
  - various graphing methods,
  - and others to facilitate the generation of test cases for acceptance testing
- Prototyping methods begin by developing a prototype of the target system.
- This prototype is then evaluated to see if it satisfies the requirements.
- Once the prototype does satisfy the requirements, the outputs from the prototype become the expected outputs from the final system.
- Therefore, prototypes are used as test oracles for acceptance testing.



- Once, we have some mechanism to arrive at a conclusion such as:
    - accept the system as delivered
    - accept the system after modifications are made
    - do not accept the system
    - If the test results are unsatisfactory, changes be made to the system before acceptance testing can proceed to the next phase
- Finally, the exit Criteria:
    - No critical defects open
    - Business process works satisfactorily
    - UAT Sign off meeting with all stakeholders..

Module 157:
Acceptance Testing Activities

- UAT Management
    - Create UAT Strategy
      Ensure effective collaboration between IT and Business BA and PMO
    - Participate in requirements walkthrough meetings
    - Review Effort Estimation, Test Plan
    - Ensure Requirement Traceability
    - Drive metrics collection to quantify the benefits derived out of the updated testing methodology, tools and environment usage
- Team activities

- Verify & Validate Business Requirement against Business process
- Estimation for UAT
- Create & Execute UAT test Plan
- Participate in requirement JAD session
- Prepare test scenarios, test cases and test data based on Business Process

- Team activities
  - Maintain Traceability
  - Execute test cases and prepare test logs
  - Report defects in test management tool and manage them throughout their lifecycle
  - Produce UAT End of test report
  - Provide Business Readiness Support and Live proving..

Module 72:
Fault Seeding

- Fault Seeding, is used to estimate the "number of **errors**" remaining in a program.
- In Fault Seeding Technique, a predefined number of artificially generated errors is "sown" in the program code.
- Before testing, "seed" the program with a number of "typical errors" keeping careful track of the changes made.
- After a period of testing, compare the number of seeded and non-seeded errors detected.
- Before testing, "seed" the program with a number of "typical errors," keeping careful track of the changes made.
- After a period of testing, compare the number of seeded and non-seeded errors detected.
- If **S** is the total number of faults seeded, **n** is the number of seeded faults detected, and **N** is the number of non-seeded errors detected, the number of remaining (non-seeded) faults in program is:

  N(S/n – 1)

- Fault seeding as a technique introduced by Mills (1972)
- This helps in calculating confidence C
- **C = 1** if n > N and **C = S/(S - N + 1)** if n $\leq$ N
- where
  - **S** is the number of seeded faults,

    - **N** is the total number of non-seeded (indigenous) faults which can be found
- We calculate N by
    - **N = S*n / s**
    - where **n** is actual number of non-seeded faults and **s** is the number of seeded faults detected during testing.
- Fault seeding provides an alternative to structural testing techniques providing us with a measure of sufficiency of testing or effectiveness of our test suite.
- One or more faults are deliberately introduced into a code base, without informing the testers.
- The discovery of seeded faults during testing can be used to calibrate the effectiveness of the test process.
- Let *S* is the total number of seeded faults, and *s*(*t*) is the number of seeded faults that have been discovered at time *t*.
- *s*(*t*)/*S* is the seed-discovery effectiveness of testing to time *t*.
- If seeded faults are assumed are to be representative of actual faults, then seed-discovery effectiveness can be assumed to be representative of overall testing effectiveness
- Question:
    - Seed 100 faults into a project at time 0.
    - Testing continues to time 30, at which point 73 of the seeded faults have been detected.
    - If 219 actual faults were discovered, what is the expected number of total faults prior to seeding?
    - How many latent faults are expected to remain in the software at time 30?
- Answers:
    - The discovered original faults are three times the numbered of discovered seeded faults, so 300 original faults are expected.
    - The latent faults are those remaining and not removed: (300 - 219) original faults plus (100 - 73) seeded faults, that is 81 + 27 = 108 latent faults.
- Fault types:
- Domain faults
- Computational faults
- …

TABLE I. FAULT DISTRIBUTION

Class of Errors	%age
Computational	13%
Initialization	13%
Logic/Control	32%
Interface	18%
Data	24%

1	No mutants are present here.	Mutants are developed for testing.
2	Here source code is tested within itself.	Here mutants are combined, compared for testing to find error introduced.
3	Errors are introduced directly.	Special techniques are used to introduce errors.
4	Test cases which detect errors are used for testing.	Here, test cases which kill mutants are used for testing.
5	It is less efficient error testing technique.	It is more efficient than error seeding.
6	It requires less time.	It is more time consuming.
7	It is economical to perform.	It is expensive to perform.
8	It is better method for bigger problems.	It is a better method for small size programs.

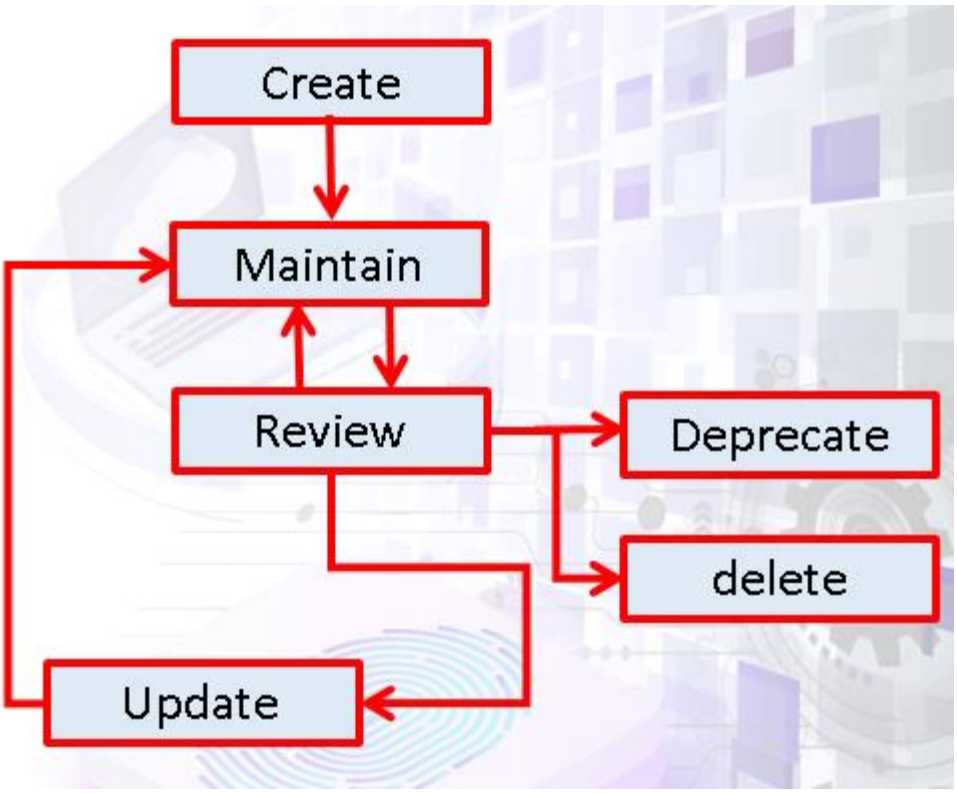
- Mutation Testing
  - Best suitable for smaller systems
  - Critical systems
  - Fault localization is easy
- Fault Seeding
  - Economical yet fault localization is difficult

Module 88:
Test Case Lifecycle

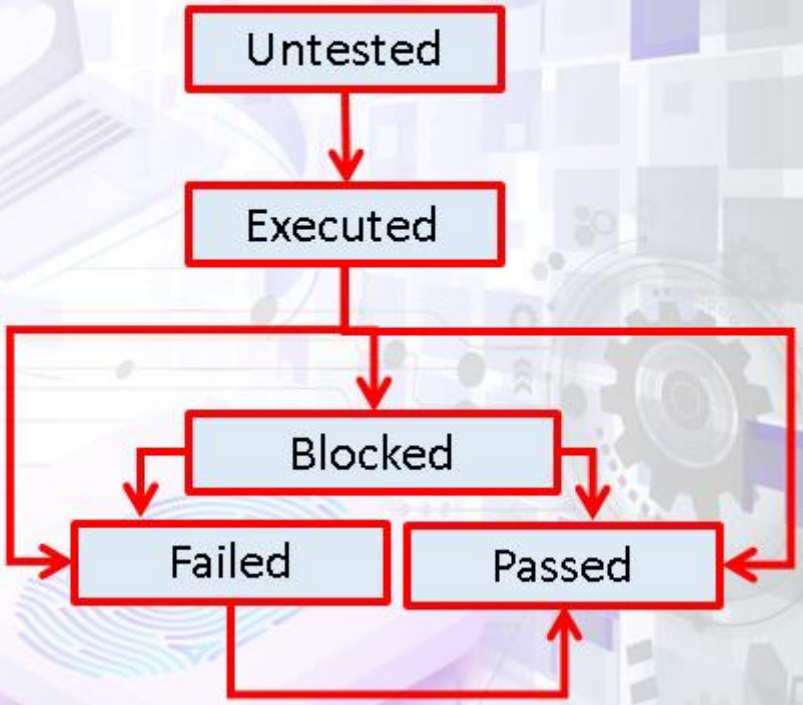
A test case undergoes a series of steps from its creation to being deleted or deprecated.

- A test case is a process rather than a single activity i.e. designing a scenario, preparing for execution and evaluating status till the test closure.
- A test case activity can be divided into fundamental test process with following steps:
- Planning and Control step determine scope and risks and identify objectives of testing.
- Analysis and Design step is used to review the information we need in order to start the test analysis and create our own test cases.
- Implementation and Execution represent set of conditions under which a tester will determine whether an application is working correctly or not and test case is pass or fail

- Evaluating exit criteria and Reporting is set the criteria for each test level against which we will measure the “enough testing”.
- Test Closure activities are done when software is delivered to qa testing companies. The testing can be closed when all the information has been gathered which are needed for the testing.
- Test case lifecycle
    - Created with an ID and other details
    - Maintained in the repository for use
    - Reviewed periodically or upon a release
        - Either kept as is
        - Or updated and kept for use
        - Or deprecated / deleted depending upon situation



- State transition of a test case



Module 35:
Coverage Criteria for Control flow testing

- Coverage:
- Node Coverage:
- Edge Coverage:
- Statement Coverage:
- Condition Coverage
- Decision Coverage
- MC/DC
- …

- Node Coverage: all nodes executed
- Edge Coverage: all edges executed
- Statement Coverage: Achieved when all statements in a method have been executed at least once
- Example Code:

```
IF ((x || y) && z)
        a = b + c;
```

- Condition Coverage:
  - each boolean expression X,Y and Z in above statement should be evaluated to TRUE and FALSE at least one time
- Test cases:

```
T1: X=T, Y=T, Z=T
T2: X=F, Y=F, Z=F
```

- Example Code:

```
IF ((x || y) && z)
        a = b + c;
```

- Decision Coverage:
  - we need to ensure that the IF statement evaluates to TRUE and FALSE at least once. So the test set will be: Test cases:

```
T1: X=T, Y=T, Z=T
T2: X=F, Y=F, Z=F
```

- Example Code:

```
IF ((x || y) && z)
    a = b + c;
```

- Modified/Multiple Condition Decision Coverage (MCDC):
  - each Boolean variable should be evaluated to TRUE and FALSE at least once and also affect the decision outcome cases:

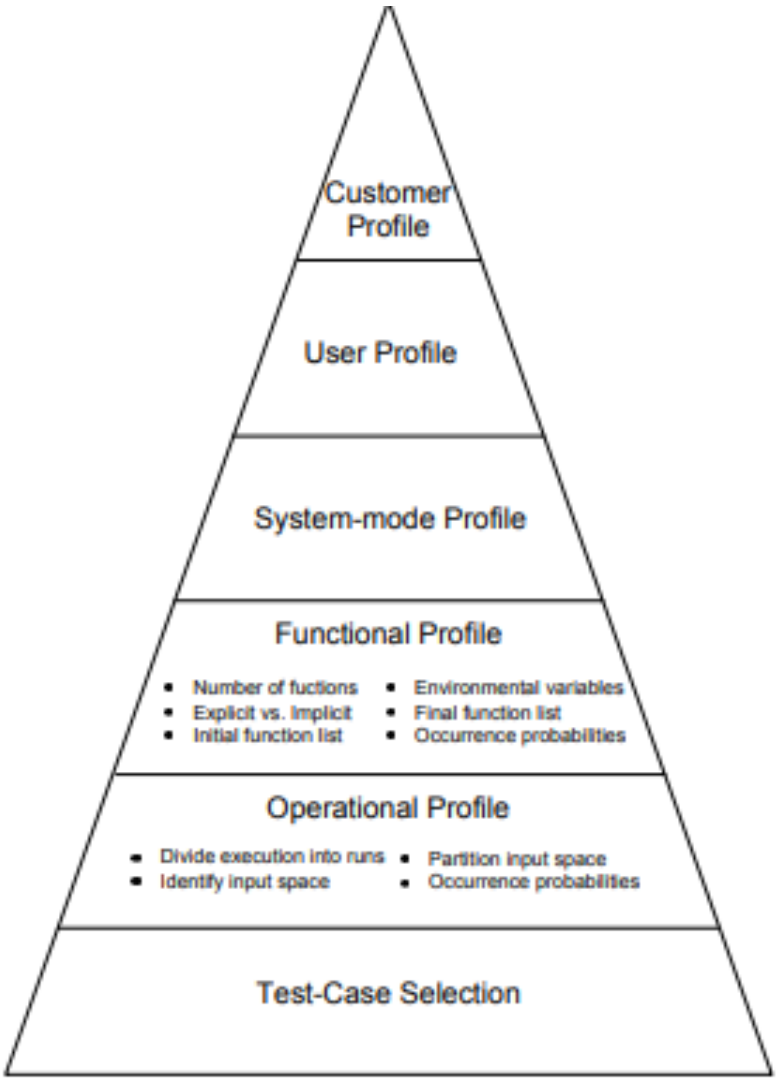
T3: X=F, Y=F, Z=T
T4: X=F, Y=T, Z=T
T5: X=F, Y=T, Z=F
T6: X=T, Y=F, Z=T

- We define subsumption
- Criteria subsumption : A test criterion C1 subsumes C2 if and only if every set of test cases that satisfies criterion C1 also satisfies C2
- In our case Edge Coverage subsumes Node coverage
- However, Coverage does not depend on the number of test cases..

Module 159:
Reliability Testing Characteristics

- As discussed:
  - Reliability is probability of **failure-free** operation of a computer program for a **specific time**
- Reliability testing is dependent upon:
  - Fault density: number of faults presents in the software
  - Time:
    - Execution time: CPU time taken by an application.
    - Calendar time: normal view of time
  - Clock time: Execution time + Idle time
- Operational profile: The way users operate the system
- We need to revisit definitions of error, fault and failure
- Error: difference between
  - actual observed output

    - and the correct expected output
    - refers to human action resulting faults or defects.
- Fault:
    - is the static condition that causes a system to fail in performing its required functionality
- Failure:
    - inability of a system to perform its required function according to its specification.
    - Failure is a characteristic of program's behaviour and it is a dynamic attribute.
- The objective behind performing reliability testing are to find out:
    - structure of repeating failures.
    - number of failures occurring is the specified amount of time.
    - main cause of failure
    - Performance testing of various modules of software application after fixing defect
- Operational profile of a system
    - Concept developed at AT&T Bell Labs.
- An operational profile describes how actual users operate a system.
- Many ways to represent operational profiles e.g.,
    - Tabular
    - Graphical
- Operation profiles example from "Musa J.D., Operational profiles in software reliability engineering, In IEEE Software 10 (1993), No. 2, pp. 14-32"



Module 160:
Reliability Testing Types

- Reliability testing types include:
  - Feature testing
  - Load testing
  - Regression testing
- Feature Testing:
  - check feature provided by software and is conducted such that:
    - Each operation in SUT is executed at least once.
    - Interaction between two operations is reduced.

        - Each operation have to be checked for proper execution
- Load testing:
    - check how well SUT performs when compared to competition system or performance. It is also based on number of concurrent users who are using SUT and behaviour of SUT to users.
    - SUT must respond to user commands with better response time (then the competition system) and meet user expectations.
- Regression testing:
    - check if SUT is performing well and no bugs have been introduced as a result of the addition of new functionality in the software.
    - It is also done when a bug has been fixed and the tester needs to test it again
    - Fundamental reliability testing types:
        - Test-retest reliability
        - Parallel or alternate forms reliability
        - …
- Test-retest reliability: A reliable test must show similar result over time
    - Consider Daewoo Service example, we do booking at 9:30 am and testing the same functionality at 1 pm again.
    - We compare both results.
    - If we are getting a high correlation in results, we find out that test is ‘Reliable'.
- Parallel or alternate forms reliability: Different versions of the same test must produce similar behaviour.
    - Consider Daewoo Service example, we do booking at 9:30 am and testing the same functionality at 1 pm again.
    - We compare both results.
    - If we are getting a high correlation in results, we find out that test is ‘Reliable'.
- Inter-rater Reliability: Rating by multiple judges should be similar and high
- Inter-Rater Reliability is otherwise known as Inter-Observer or Inter-Coder Reliability.
- It is a special type of reliability which consists of multiple judges.
- It deals with the consistency of the rating put forward by different observers.

Module 161:
Models of Software Reliability

- Many types of models with broad categorization:
    - Prediction model
    - Estimation model
- In Predictive testing,
    - we predict result with the historical data, statistics, machine learning data etc.
    - We construct plots such as scatterplot and draw an extrapolate line to the existing historical data and we can predict the upcoming data.
    - This type of model is performed before the development or testing stage itself
- Estimation Testing
    - In addition to using historical data,
    - we use current data.
    - We predict reliability of a product in the present time or future time.
    - This type of testing is performed during last stages of software development
- Key assumptions while developing models:
    - Faults in the program are independent.
    - Execution time between failures is large w.r.t. instruction execution time.
    - Potential test space covers its use space.
    - The set of inputs per test run is randomly chosen.
    - The fault causing a failure is immediately fixed or else its re-occurrence is not counted again

Module 162:
Software Reliability Metrics

- Probability of failure:
    - Likelihood SUT would behave unexpectedly
- Rate of occurrence of failure:
    - Frequency of occurrence of unexpected behaviour of SUT
- Mean time to failure (MTTF)
    - Measure of time interval between observed failures, Useful to find how long a system remains stable

- Availability:
    - Measure of likelihood that the system is available to the users
- MTTF: Mean Time To Failure
- MTTR: Mean Time To Repair
- MTBF: Mean Time Between Failures

  = MTTF + MTTR
- Availability =

[MTTF /(MTTF+MTTR)]*100

- Two factors are important
    - Intrinsic quality
        - Measured in terms of number of “bugs” in the software
        - How long a system can run before encountering serious faults, e.g., crash
        - Measured in terms of:
            - Fault density: measure Faults relative to software size
            - Mean time to failure (MTTF): measure time between failures
    - Customer satisfaction
        - Measured in terms of customer survey data
        - Usability metrics
- For intrinsic quality, we calculate fault removal effectiveness
    - Faults are injected during different phases
    - Faults at dev cycle m = Faults escaped in (n – 1) + Faults injected in (n) – Faults removed in (n)
    - Measure of overall fault removal ability
    - When used for early phases, it is call early fault removal
    - When used for a particular phase, it is called phase effectiveness
    - Higher is the metric, better is the dev process

      = fault removal in a development phase X 100

      fault latent in the product
- Where

        - Fault latent (step i)=faults escaped (step i-1)+defects injected (step i)
- Fault arrival pattern during machine testing
- Positively co-relates with faults rate in production environment
- Higher faults during testing rate shows
    - Low software quality
    - Development process and developers' ability
    - Fault density i.e., Faults per KLOC, FP is also an indicator of produced software's quality

Module 163:
Cleanroom Software Engineering

- The name "cleanroom" is derived from the process used to fabricate semi-conductors
- The philosophy focuses on defect avoidance rather than defect removal
- It combines
    - formal methods and
    - software quality methods
- We have studied later in this course only while former is covered in some other course
- Cleanroom is a concept of Shifting practice
- From - To
    - Individual efforts to peer reviewed
    - Sequential development to incremental
    - Informal coverage to statistical measures
    - Unknown reliability to measured reliability
    - Informal design to a formal design
- Cleanroom software engineering involves integrated use of
    - software engineering modeling
    - program verification
    - statistical software quality assurance.
- Verifies design specification using mathematically-based proof of correctness
- Relies on statistical use testing to uncover high impact errors
- Generally follows an incremental development process

- Benefits include:
    - Failure reduction
    - Less development time
    - Better product life
- Overall conclusion:
    - Primary goal is:
        - to avoid dependence on defect-removal processes
        - by writing code increments right
        - the first time and verifying their correctness before testing.
- Our focus shifts
    - from detection / correction
    - to defect prevention
- We aim Zero-defect software is the goal.

Module 164:
Cleanroom Software Engineering Principles

- We have
    - Small team with
    - independent specification, development,
    - and certification sub-teams
- We do
    - Incremental development
    - under statistical quality control
- We assess performance
    - during each increment
    - using measure like:
    - errors per KLOC,
    - rate of growth in MTTF,
    - or number of sequential error-free test cases

        - Etc.
- feedback is used for
    - process improvement and
    - development plan is adjusted as needed
- Specification team
    - develops and maintains the system specification
- Development team
    - develops and verifies software
    - the software is not compiled or executes during verification
- Certification team
    - develops set of statistical test to exercise software after development
    - reliability growth models used to assess reliability
- Software development based on mathematical principles
    - the box principle is used for specification and design
    - formal verification is used to confirm correctness of implementation of specification
    - program correctness is verified by team reviews using questionnaires
- Testing based on statistical principles
    - operational usage profiles needed
    - test cases are randomly generated from the usage model
    - failure data is interpreted using statistical models

Module 165:
Cleanroom Software Engineering Process

- Increment planning.
    - The project plan is built around the incremental strategy.
- Requirements gathering.
    - Customer requirements are elicited and refined for each increment using traditional methods.
- Box structure specification.
    - Box structures isolate and separate the definition of behavior, data, and procedures at each level of refinement.
- Formal design.
    - Specifications (black-boxes) are iteratively refined to become:
        - architectural designs (state-boxes)
        - and component-level designs (clear boxes).
- Correctness verification.
    - Correctness questions are asked and answered,
    - formal mathematical verification is used as required.
- Code generation, inspection, verification.
    - Box structures are translated into program language
    - inspections are used to ensure
        - conformance of code and boxes,
        - syntactic correctness of code;
        - correctness verification of the code.
- **Statistical test planning.**
    - A suite of test cases is created
        - to match the probability distribution of the projected product usage pattern.
- Certification Steps:
    - Usage scenarios must be created
    - Usage profile is specified
    - Test cases generated from the usage profile

- Tests are executed and failure data are recorded and analyzed
- Reliability is computed and recorded

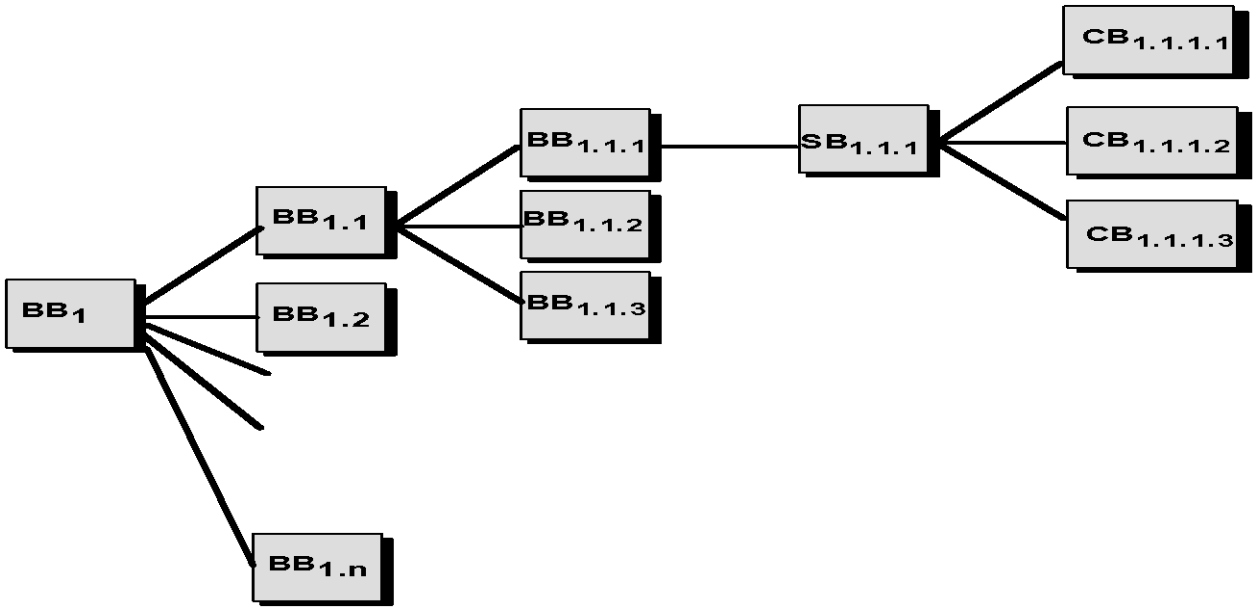
Module 166:
Cleanroom Software Engineering – Benefits

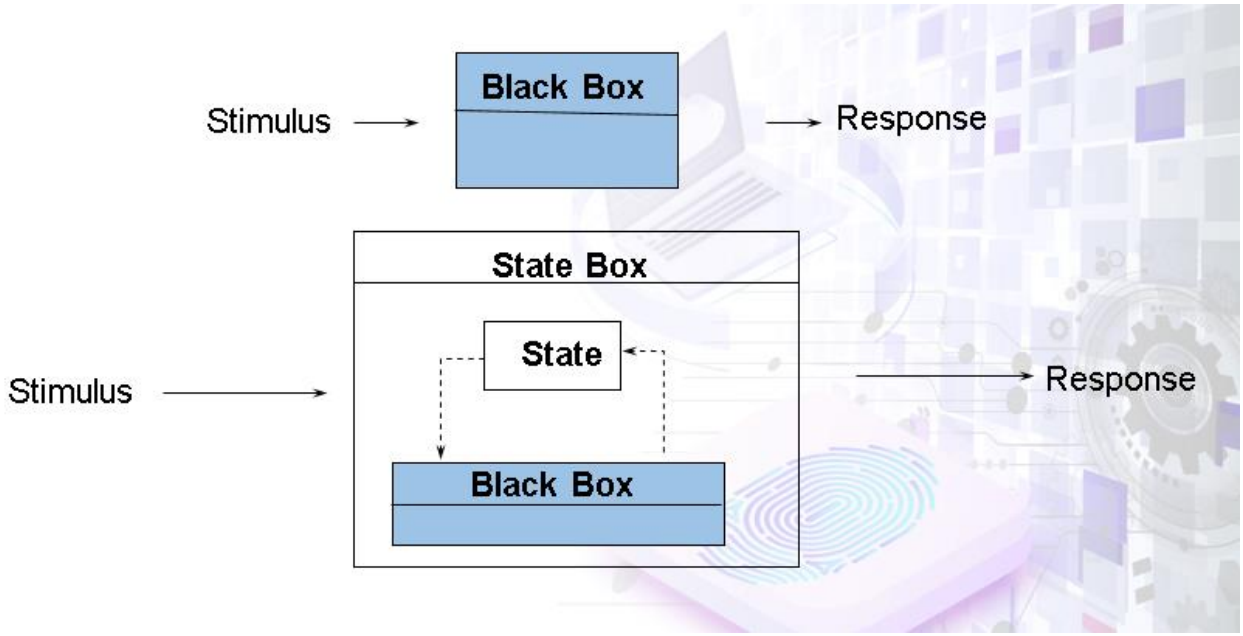
- Benefits:
  - Developing the right systems the first time,
  - requires customer involvement and feedback throughout the development process
  - Requires management control of resources and technical control of complexity
  - Product quality requires process measurement and control throughout the SW development cycle
- Benefits:
  - Concurrent engineering by scheduling parallel development and certification
  - Stepwise integration through testing cumulative increments
  - Continuous quality feedback from statistical process control
  - Continuous customer feedback from actual use
  - Change management by systematic accommodation of changes
  - Represents the first practical attempt to put software development under statistical quality control with a well-defined strategy for continuous process improvement

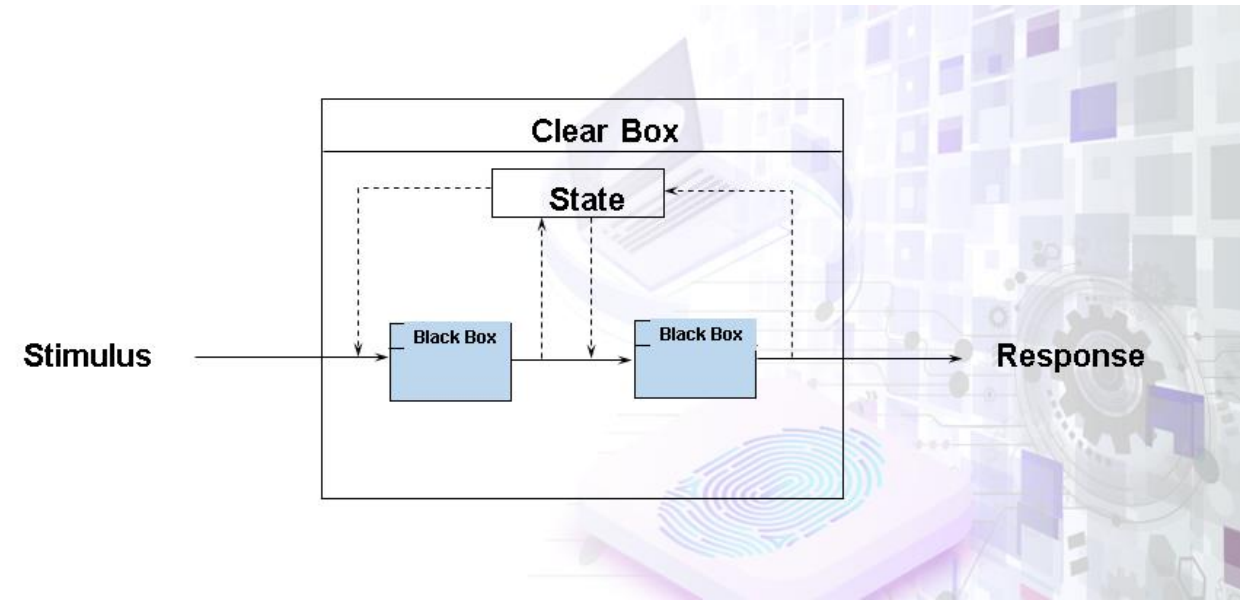
Module 167:
Cleanroom Software Engineering - Box Principles

- We need to understand box structure associated with cleanroom SE.
- We need to understand this to get to testing focus.
- Box are essentially a design concept:
  - The development team carries out a design and correctness verification cycle for each increment.
- We need to understand box structure associated with cleanroom SE…
  - The certification team works in parallel, using the usage specification to generate test cases that reflect expected use of the accumulating increments.

- Box structures are used to systematically move from an abstract specification to a detailed design providing implementation detail.
- Box structures model system components as abstractions in three detailed forms:
    - Black Box
        - Gives an external view of the component.
        - Provides description of functional requirements without details on the internal structure and operations.
        - Describes the user-visible system inputs and responses.
    - State Box
        - Gives an intermediate view of the component.
        - Decomposes the black box into an internal state representation and an internal black box.
    - Clear Box
        - Gives a detailed view of the component.
        - Replaces internal black box with detailed design using structured programing constructs.
- System development is a process of stepwise box decomposition.







- Box structure principle:
- Referential Transparency
    - The behavior of a black box is the same regardless of where in the system it is referenced.
    - The implementation of a black box is independent of the implementation of other parts of the system.
    - Referencing a black box is equivalent to referencing its corresponding clear box representation throughout the system.

- Box structure principle:
- Transaction Closure
    - Ensures that a sound and complete set of transactions is identified to achieve the required system behavior.
    - Black box level - system stimuli are necessary and sufficient to generate the required responses.
- Box structure principle:
- Transaction Closure
    - …
    - State box level - defined transactions must be necessary and sufficient for the acquisition and preservation of all state data
    - state data must be necessary and sufficient for the completion of all transactions.
- Box structure principle:
- Transaction Closure
    - …
    - Clear box level - procedural design and the internal black boxes must include all transactions.
    - We can mark a start-state and an end-state
    - This is helpful while writing test cases.
- Box structure principle:
- State Migration
    - State data should be stored at as low a level as possible, but as high as necessary or expedient.
- Since we are using concept of state and state transition, we can corelate this directly with our previous knowledge of extracting test cases from system represented as a state machine
- Referring to our model-based testing lectures…
- Box structure principle:
- …
- Common Services
    - Reusable boxes
    - May be created or referenced from a library of reusable components.

- If from a library, the common service is a pre-certified component.
- Can reduce system size and complexity.

Module 168:
Cleanroom Software Engineering - Testing Focus

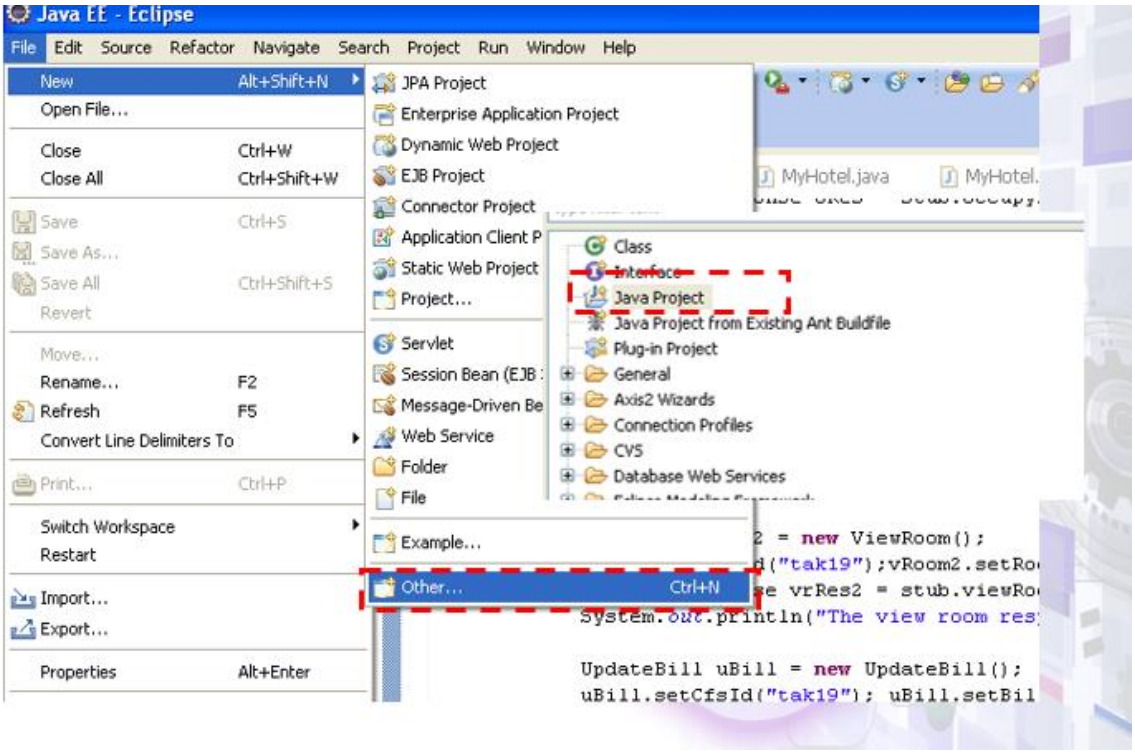
- Generation of test cases
  - SUT is treated as a state-machine
  - each test case begins in a start state and represents a random walk through the usage model ending at a designated end state.
  - Our test sequences correspond to paths through the state-machine.
- Control of statistical testing
  - a well-defined procedure is performed under specified conditions
- Generation of test cases
  - …
  - each performance is a trial and can be used as part of an empirical probability computation
- Stopping criteria for testing
  - when testing goals or quality standards are achieved
  - when the difference between the predicted usage chain and the actual testing chain becomes very small
- There are five major activities involved in a Cleanroom process
  - Specification
  - Increment Planning
  - Design and Verification
  - Statistical Testing
  - Certification
- Two to three independent teams may exist and work concurrently
  - Development Team

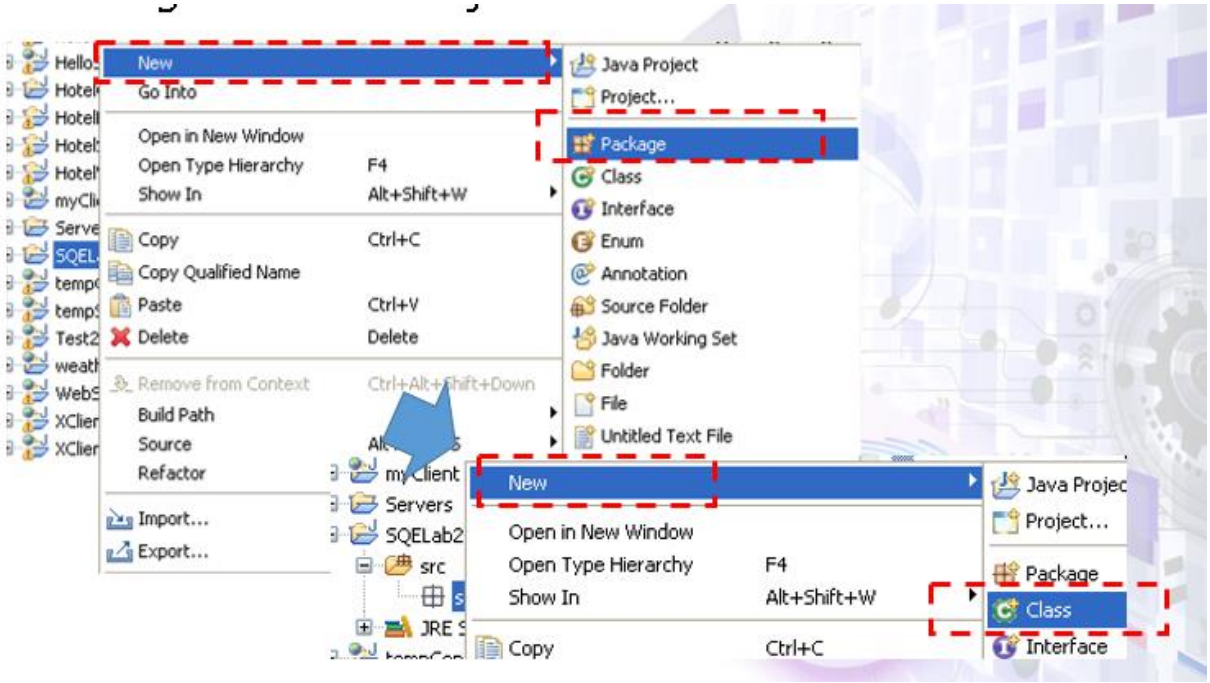
    - Testing (or Certification) Team
    - Documentation Team
- Two specifications are produced: functional and usage.
- Functional Specification
    - Defines the required external system behavior in all circumstances of use.
    - Forms the basis for incremental software development.
- Usage Specification
    - Defines usage scenarios considering:
        - *User* - person, hardware device or other software; subclasses may exist
        - *Use* - a particular work session or transaction; bounded by specific start and end events
        - *Environment* - platform, OS environment, system load, etc.
    - Forms the basis of statistical testing and quality certification.
- The procedural control structures of structured programming are single-entry, single-exit structures,
- Therefore, it produces no side-effects in control flow.
- Example: For integer x >= 0, the program function of the iteration control structure below is, in English, “Set odd x to 1, even x to 0.”
- In designing clear box procedures, we define:
    - an *intended function*,
    - We then refine it into a control structure and
    - new intended functions.
- Intended functions are recorded in design and are attached to the corresponding control structure refinements.
- So, clear boxes are composed of a finite number of control structures, each of which can be checked for correctness against its intended function.
- To verify the correctness of each control structure,
    - we derive its program function (the function it actually computes) and
    - compare it to its intended function, as recorded in the design.
- This corresponds to test case structure we have already studied.

- A correctness theorem formally defines how to do this for each control structure in terms of language independent correctness conditions.
- During a team review, every correctness condition of every control structure is verified in turn.
- Each team member must agree that each condition is correct. Thus, an error is possible only if every team member incorrectly verifies a condition.
- If an informal approach cannot produce a unanimous decision, formal proofs of correctness can be employed.
- This is more efficient and produces better code than unit testing.
- Testing software according to the way users intend to use it.
- The entire focus is on external system behavior, not the internals of the design or implementation.
- The certification team's goal is not to debug, but to certify the software's quality. This requires deep knowledge of expected usage but no knowledge of design or implementation information.
- Three steps
    - Specify usage probability distributions
    - Derive test cases that are randomly generated from usage probability distributions.
    - Execute test cases, assess results, and compute quality measures.
- Based on the data gathered during statistical testing, the software can be given a certified reliability.
- Reliability is expressed as MTTF and is computed according to specific mathematical reliability models
    - Sampling model
    - Component model
    - Certification model

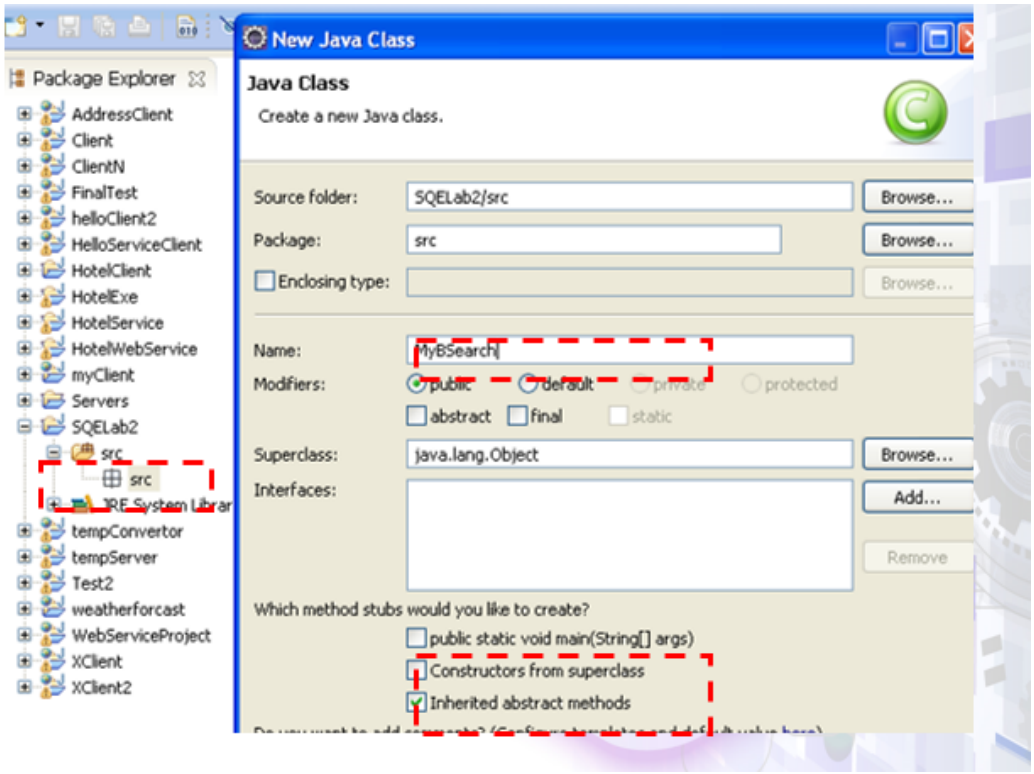
Module 169:
Software Testing Support for Java related IDE

- We have already seen unit test development using Microsoft Visual Studio
- We now have a look at how to use Eclipse for the same purpose
- We intend to see how we can make use of Java related environments for the same purpose


Java EE - Eclipse
File Edit Source Refactor Navigate Search Project Run Window Help
New Alt+Shift+N
Open File...
Close Ctrl+W
Close All Ctrl+Shift+W
Save Ctrl+S
Save As...
Save All Ctrl+Shift+S
Revert
Move...
Rename... F2
Refresh F5
Convert Line Delimiters To
Print... Ctrl+P
Switch Workspace
Restart
Import...
Export...
Properties Alt+Enter
JPA Project
Enterprise Application Project
Dynamic Web Project
EJB Project
Connector Project
Application Client P
Static Web Project
Project...
Servlet
Session Bean (EJB
Message-Driven Be
Web Service
Folder
File
Example...
Other... Ctrl+N
Class
Interface
Java Project
Java Project from Existing Ant Buildfile
Plug-in Project
General
Axis2 Wizards
Connection Profiles
CVS
Database Web Services
MyHotel.java
MyHotel.
= new ViewRoom();
("tak19");vRoom2.setRo
e vrRes2 = stub.viewRo
System.out.println("The view room res
UpdateBill uBill = new UpdateBill();
uBill.setCfsId("tak19"); uBill.setBil

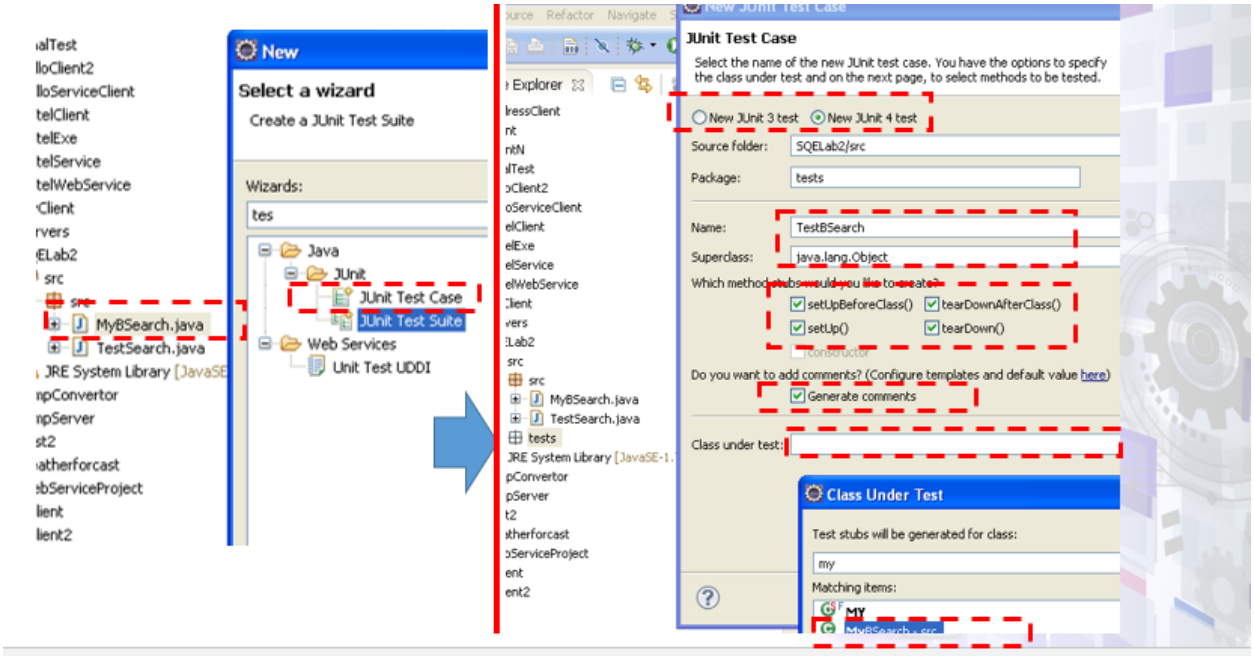

New
Go Into
Open in New Window
Open Type Hierarchy F4
Show In Alt+Shift+W
Copy Ctrl+C
Copy Qualified Name
Paste Ctrl+V
Delete Delete
Remove from Context Ctrl+Alt+Shift+Down
Build Path
Source
Refactor
Import...
Export...
Java Project
Project...
Package
Class
Interface
Enum
Annotation
Source Folder
Java Working Set
Folder
File
Untitled Text File
myClient
Servers
SQELab2
src
JRE
New
Open in New Window
Open Type Hierarchy F4
Show In Alt+Shift+W
Copy Ctrl+C
Java Projec
Project...
Package
Class
Interface

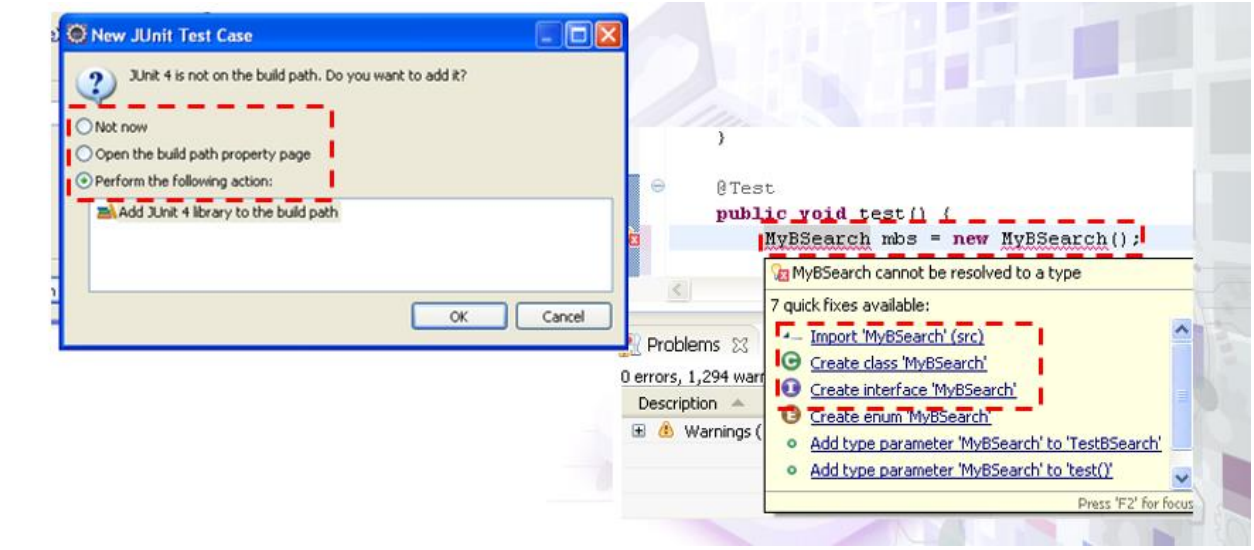
Right Click on Package and Add Class



Software Testing support in Java related IDEs

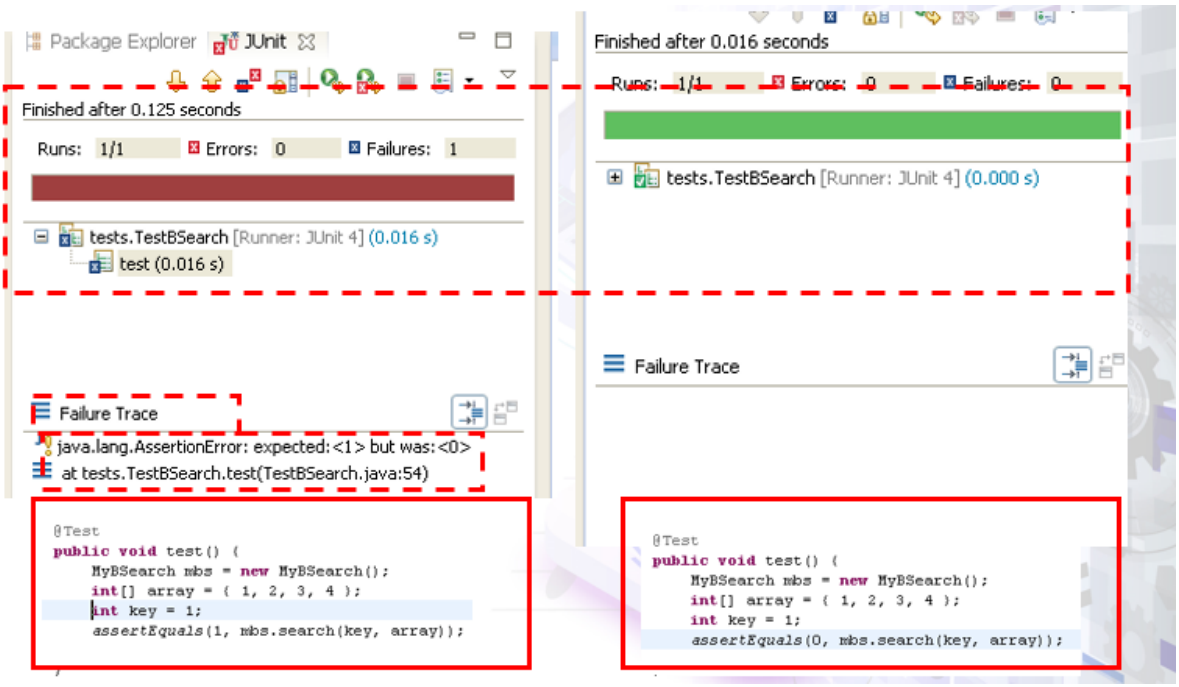
```
package src;
public class MyBSearch {
   public int search(int key, int[] elemArray) {
      int bottom = 0;  int top = elemArray.length - 1;
      int mid;   int index = -1;  boolean found = false;
      while (bottom <= top && found == false)  {
       mid = (top + bottom) / 2;
       if (elemArray[mid] == key) {
              index = mid;  found = true;    return index;       }
      else  {        if (elemArray[mid] < key)
                          bottom = mid + 1;
       else               top = mid - 1;
      } }     return index; }}
```

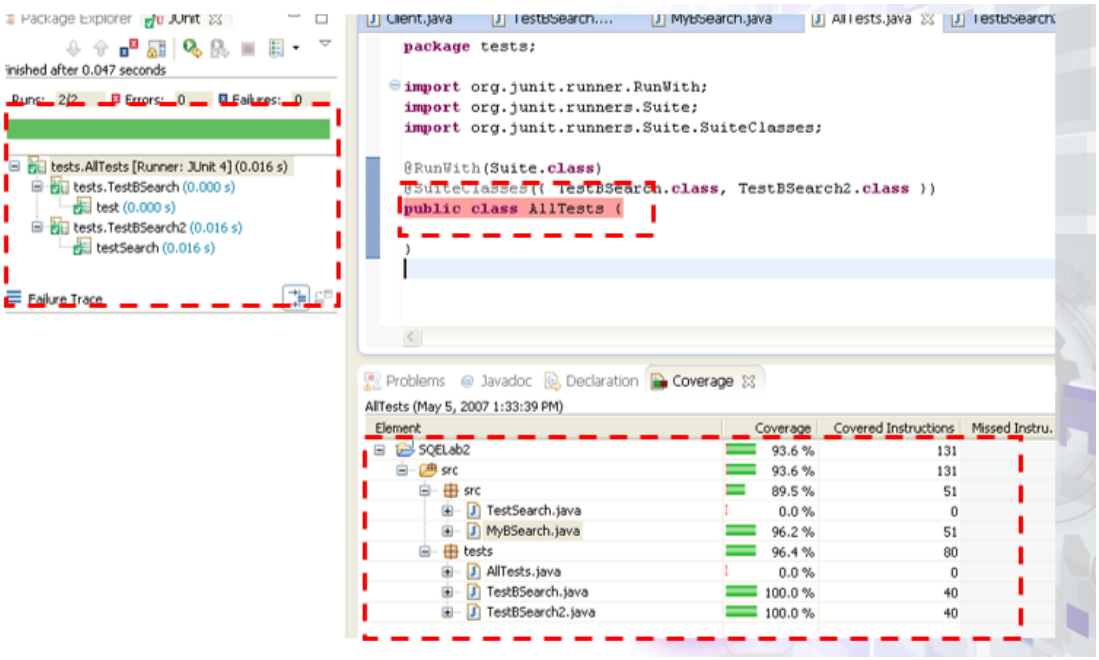

New
Select a wizard
Create a JUnit Test Suite
Wizards:
tes
Java
JUnit
JUnit Test Case
JUnit Test Suite
Web Services
Unit Test UDDI
MyBSearch.java
TestSearch.java
JUnit Test Case
Select the name of the new JUnit test case. You have the options to specify the class under test and on the next page, to select methods to be tested.
New JUnit 3 test
New JUnit 4 test
Source folder:
SQELab2/src
Package:
tests
Name:
TestBSearch
Superclass:
java.lang.Object
setUpBeforeClass()
tearDownAfterClass()
setUp()
tearDown()
Do you want to add comments? (Configure templates and default value here)
Generate comments
Class under test:
Class Under Test
Test stubs will be generated for class:
my
Matching items:


New JUnit Test Case
JUnit 4 is not on the build path. Do you want to add it?
Not now
Open the build path property page
Perform the following action:
Add JUnit 4 library to the build path
OK
Cancel
@Test
public void test() {
MyBSearch mbs = new MyBSearch();
MyBSearch cannot be resolved to a type
7 quick fixes available:
Import 'MyBSearch' (src)
Create class 'MyBSearch'
Create interface 'MyBSearch'
Create enum 'MyBSearch'
Add type parameter 'MyBSearch' to 'TestBSearch'
Add type parameter 'MyBSearch' to 'test()'
Press 'F2' for focus
Problems
Description
Warnings

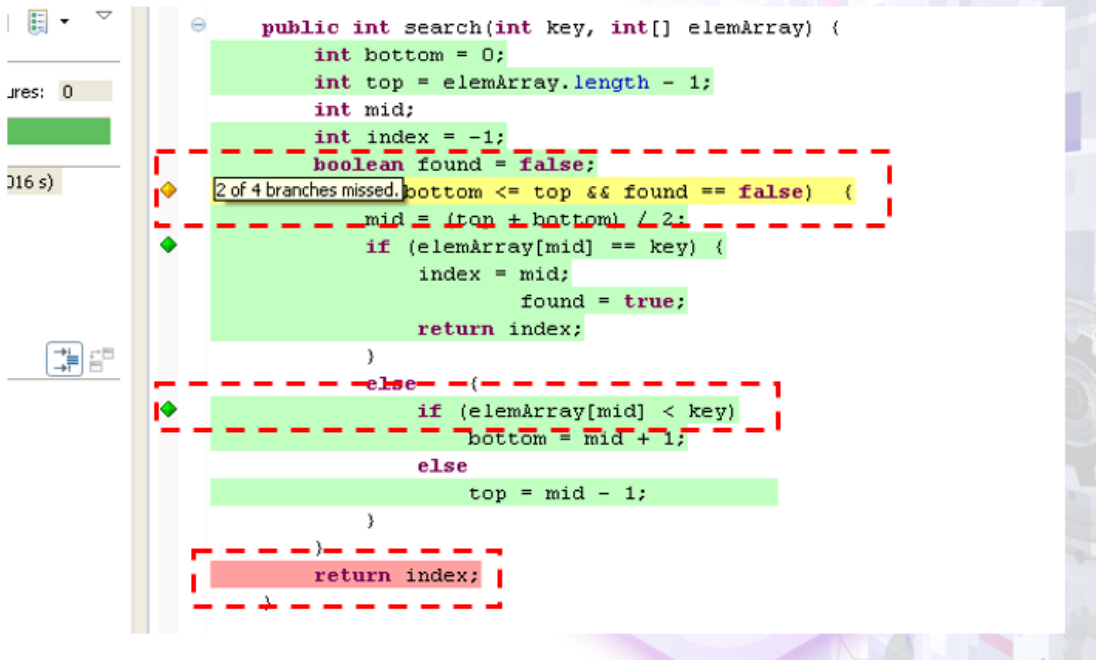
```java
@Test
public void test() {
	MyBSearch mbs = new MyBSearch();
	int[] array = { 1, 2, 3, 4 };
	int key = 1;
	assertEquals(1, mbs.search(key, array));

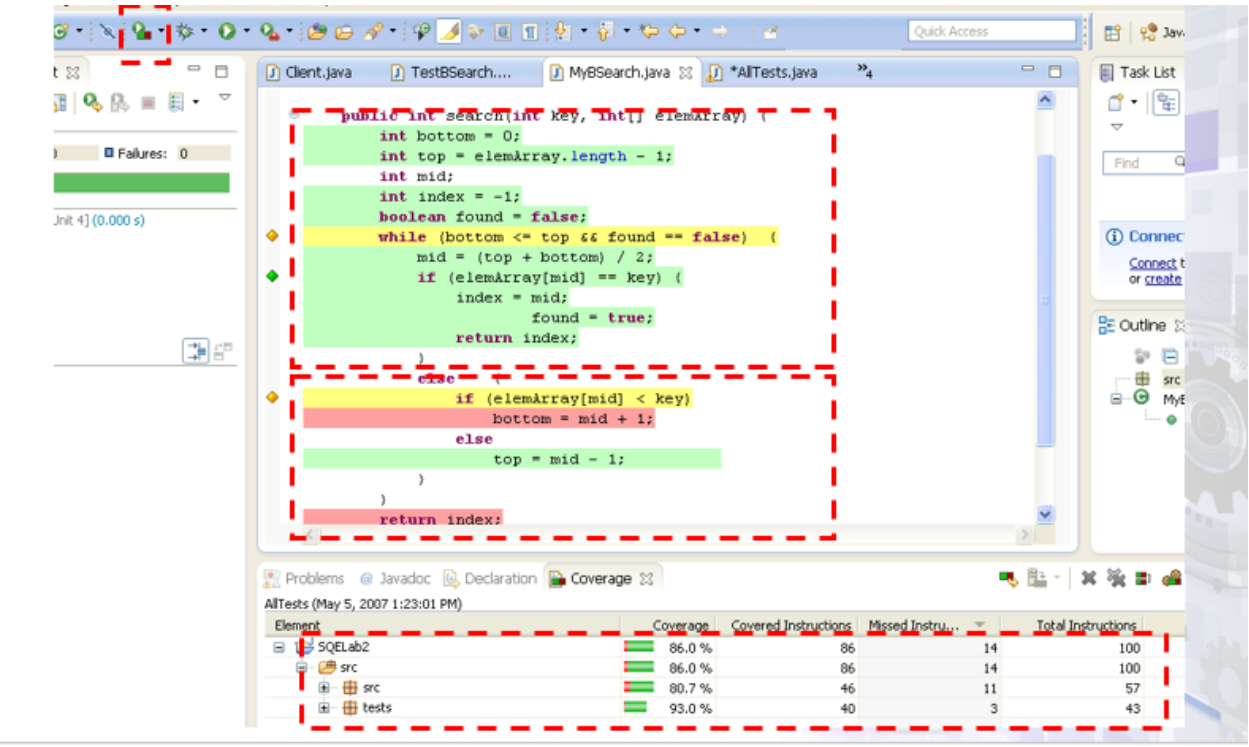
}
```





Code Analysis





- We have seen
  - How can we use Eclipse for unit testing of code written in Java
  - We have used Eclemma project for generating coverage.

Module 170:
Miscellaneous Program Representations for Testing

- Static program representations
  - Abstract syntax tree;
  - Control flow graph;
  - Program dependence graph;
  - Call graph;
  - Points-to relations.
- Dynamic program representations
  - Control flow trace, address trace and value trace;
  - Dynamic dependence graph;

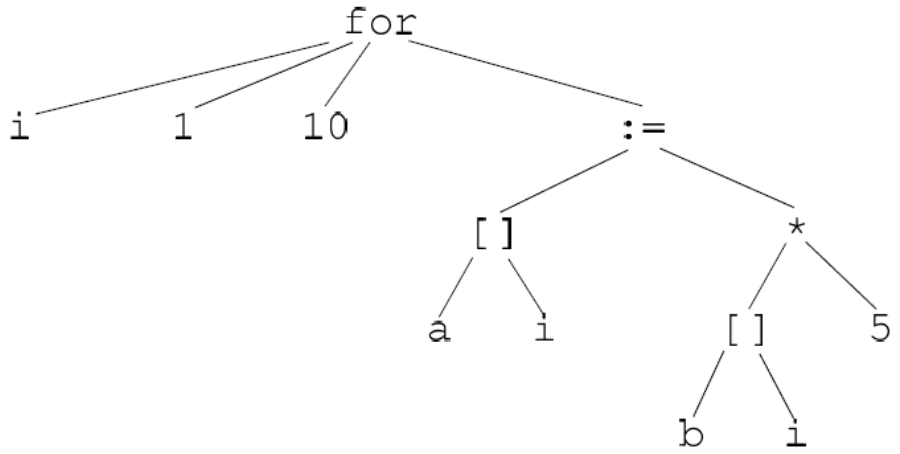
- Whole execution trace;

- Abstract Syntax Tree (AST):
- is a finite, labeled, directed tree, where the internal nodes are labeled by operators, and the leaf nodes represent the operands of the operators.

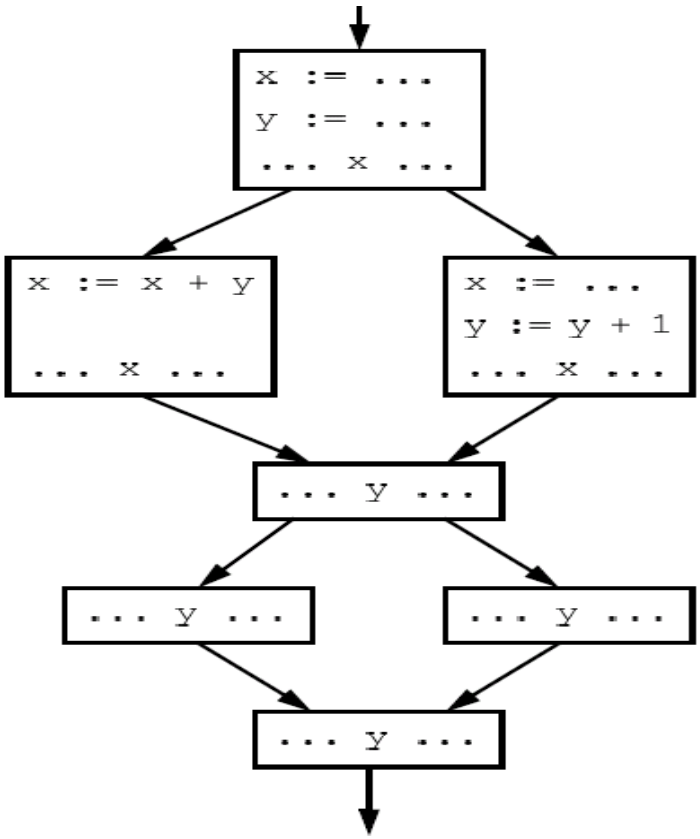
Source:

```
for i := 1 to 10 do
  a[i] := b[i] * 5;
end
```

AST:



- Program Dependence Graph:
  - S data depends T if there exists control flow path from T to S and variable is defined at T, used at S.

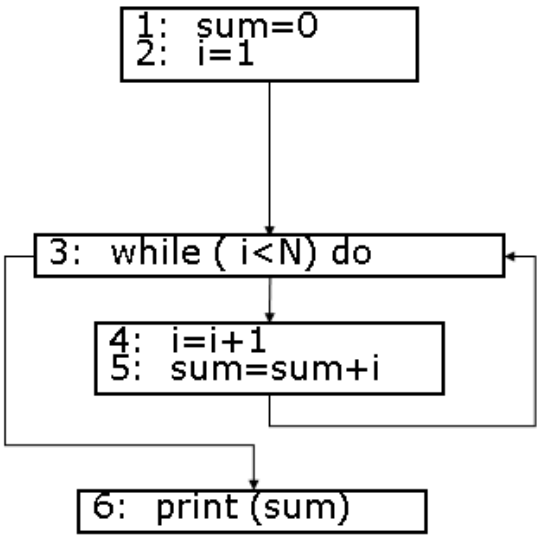


- Control Dependence graph:
- Intuitively, Y is control-dependent on X iff X directly determines whether Y executes (statements inside one branch of a predicate are usually control dependent on the predicate)
    - there exists a path from X to Y such that every node in path other than X and Y is post-dominated by Y
    - X is not strictly post-dominated by Y

- Control Dependence graph:



- Control flow trace captures complete path followed by a program during an execution.
- Analysed to determine execution frequencies of shorter program paths



N=2:

1_1: sum=0 i=1

3_1: while (i<N) do

4_1: i=i+1 sum=sum+i

3_2: while (i<N) do

4_2: i=i+1 sum=sum+i

3_3: while (i<N) do

6_1: print (sum)

$1_1\ 2_1\ 3_1\ 4_1\ 5_1\ 3_2\ 4_2\ 5_2\ 3_3\ 6_1$ → $1_1\ 3_1\ 4_1\ 3_2\ 4_2\ 3_3\ 6_1$

A More Compact CFT: < T, T, F >

- We have seen a couple of program representations for testing.

Module 171:
Software Verification and Validation – Conclusion

- We have covered basic definitions related to software verification and validation
- We have covered verification activities
- Now we know how to do inspections, walkthroughs and formal reviews
- We have seen validation related activities
- Now we can do testing a unit, integration and (sub-) system level testing
- We know difference between functional and non-functional aspects of testing
- We can perform load, volume, stress testing
- We know reliability testing
- We can perform user acceptance testing
- We can collect and analyse testing related metrics
- We have good command over related testing tools

(END OF BOOK)

(THANKS FOR VISITING VURANK)